WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Documentation
WAC (Windows Artefacts Collector)

Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

WAC — Windows Artefacts Collector

Documentation utilisateur

Version du logiciel	3.0.1
Édition du document	octobre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document décrit l'emploi de WAC : préparation, exécution d'une collecte, lecture et vérification des résultats, et traces laissées sur la machine examinée. Les mesures citées ont été relevées sur une machine virtuelle Windows 11 de test ; elles donnent des ordres de grandeur, pas des engagements.

Table des matières

1 Présentation **4**

1.1 À quoi sert WAC 4

1.2 Le principe directeur : une empreinte minimale 4

1.3 Ce que WAC ne fait pas 4

2 Principes de fonctionnement **6**

2.1 Lecture directe du disque 6

2.2 Consigne et travail 6

2.3 Ruches du registre et journaux de transaction 7

2.4 L'extraction en deux temps 7

2.5 Ce qui reste lu « à chaud » 7

3 Préparer une collecte **9**

3.1 Prérequis 9

3.2 Le support de collecte 9

3.3 Avant d'intervenir 10

4 Lancer une collecte **11**

4.1 Ligne de commande 11

4.2 Fichier de configuration : `wac.yml` 11

4.3 Options 15

4.4 Exemples 17

4.5 L'option `--binary` 17

4.6 Déroulement à l'écran 19

4.7 Durée 20

5 Collecter ici, convertir ailleurs **21**

5.1 Pourquoi séparer 21

5.2 Les réglages de la collecte 22

5.3 Les deux commandes 22

5.4 Ce que prend `--collect` 22

5.5 Ce que fait `--convert` . . . 23
5.6 Limites . . . 24

6 Les résultats **25**
6.1 Arborescence . . . 25
6.2 Le manifeste de consigne . . . 25
6.2.1 En-tête . . . 25
6.2.2 Chaque pièce . . . 26
6.3 Vérifier l'intégrité d'une collecte . . . 27
6.3.1 Le sceau . . . 27
6.3.2 Chaque pièce . . . 27
6.4 Le journal d'investigation . . . 34
6.5 Les fichiers d'artefacts . . . 34
6.5.1 Système et comptes . . . 34
6.5.2 Exécution de programmes . . . 36
6.5.3 Fichiers et navigation . . . 49
6.5.4 Périphériques . . . 52
6.5.5 Journaux d'événements (`--events`) . . . 53

7 Lire les données **55**
7.1 Horodatages . . . 55
7.2 Noms des comptes . . . 55
7.3 Noms des GUID . . . 56
7.4 Un champ absent n'est pas un champ vide . . . 56
7.5 Artefact non collecté . . . 56
7.6 Artefact absent de la version de Windows . . . 56
7.7 Couverture de MITRE ATT&CK . . . 68
7.8 Source attendue mais introuvable . . . 84
7.9 Données non décodées . . . 85
7.10 Format non pris en charge . . . 85
7.11 Chemins . . . 86

8 Traces laissées sur la machine examinée **87**
8.1 Ce que WAC fait encore . . . 87
8.2 Ce que WAC ne fait plus . . . 88

8.3 Ce qu’aucun outil ne peut éviter 88

9 Limites connues **89**

10 Dépannage **90**

A Glossaire **91**

CHAPITRE 1

Présentation

1.1 À quoi sert WAC

WAC prélève, sur un poste Windows en fonctionnement, les artefacts utiles à une investigation numérique : base de registre, journaux d'événements, fichiers Prefetch, raccourcis et listes de sauts, tâches planifiées, services, comptes, processus et sessions. Il les restitue sous forme de fichiers **JSON**, prêts à être analysés ou versés dans une chaîne de traitement.

Il s'exécute depuis une clé USB, sans installation, et écrit tout ce qu'il produit sur cette clé.

1.2 Le principe directeur : une empreinte minimale

Examiner une machine la modifie. Chaque outil lancé laisse des traces : fichiers ouverts dont la date de dernier accès change, services sollicités qui écrivent dans les journaux, clés de registre lues par l'API. Ces traces se mêlent à celles que l'on cherche.

WAC est conçu pour que ce qu'il laisse derrière lui soit aussi proche de rien que la tâche le permet, et pour que **ce qu'il ne peut pas éviter soit écrit**. Concrètement :

— le disque est lu **directement**, en lecture seule, sans ouvrir aucun fichier par le système : aucune date d'accès n'est modifiée ;

— aucune clé du registre de la machine n'est ouverte : les ruches sont copiées, puis lues sur la copie ;

— aucun cliché instantané (VSS), aucun appel COM ou WMI, aucun service de journalisation n'est sollicité ;

— rien n'est écrit sur le disque examiné ;

— chaque opération est consignée, avec la trace qu'elle est censée laisser, dans `investigation.json`.

1.3 Ce que WAC ne fait pas

— WAC **n'analyse pas** : il prélève et met en forme. L'interprétation revient à l'analyste.

— Il ne réalise pas d'image complète du disque : il extrait les fichiers porteurs d'artefacts.

— Il ne capture pas la mémoire vive. Les seules données volatiles relevées sont la liste des

processus, les sessions ouvertes et l’état des services.

CHAPITRE 2

Principes de fonctionnement

2.1 Lecture directe du disque

WAC ouvre le volume (`\\.\C:`) en lecture seule, parcourt lui-même la table des fichiers NTFS (`$MFT`) et en extrait le contenu des fichiers utiles. Le système ne voit aucune ouverture de fichier : ni date d'accès mise à jour, ni parcours de répertoire, ni verrou.

Cette lecture prend en charge les particularités de NTFS que Windows masque d'ordinaire :

Compression NTFS
: Windows 11 compresse notamment le dossier des journaux d'événements. WAC les décompresse lui-même.

Compact OS (WOF)
: Les binaires système de Windows 10 et 11 sont stockés compressés dans un flux caché ; vus par l'API, ils paraissent ordinaires. WAC les décompresse (variantes XPRESS).

Attributs éclatés
: Un fichier très fragmenté voit ses informations réparties sur plusieurs enregistrements ; WAC les réunit.

Fichiers préalloués
: Au-delà de la longueur de données valides, NTFS rend des zéros, même si le disque contient encore les restes d'anciens fichiers. WAC fait de même : le contenu extrait est celui que verrait toute acquisition ordinaire, et son empreinte aussi.

2.2 Consigne et travail

Une pièce à conviction numérique ne s'examine jamais sur elle-même : on en fait une copie, on la scelle, et l'on travaille sur une seconde copie. Si l'analyse abîme quelque chose, la copie scellée demeure et l'opération peut être refaite.

WAC applique cette procédure à **tous** les fichiers extraits, y compris ceux qu'il ne modifie pas :

`exhibits/`
: les copies brutes, telles que lues sur le volume. Une fois recopié dans la copie de travail, ce répertoire n'est plus touché, ni en écriture ni en lecture : seule la vérification de son intégrité, avant la recopie d'une conversion, le relit. Il contient le manifeste qui identifie chaque pièce, et le sceau du manifeste.

`working/`
: les copies de travail, recopiées depuis la consigne et vérifiées par empreinte — toutes, `$MFT`, journal des modifications et binaires prélevés compris, ce qui double leur place sur le support. C'est ici que sont faites les seules modifications (rejeu des journaux de transaction des ruches), et c'est ici que lisent tous les collecteurs.

> **À retenir**
> Chaque fichier extrait est donc écrit **deux fois** sur le support de collecte. Prévoyez la place correspondante (voir section 3.2).

2.3 Ruches du registre et journaux de transaction

Une ruche copiée sur une machine en fonctionnement ne contient pas les dernières modifications : elles sont encore dans ses journaux de transaction (`.LOG1`, `.LOG2`). WAC extrait ces journaux et les **rejoue sur la copie de travail**, pour que l'analyse porte sur l'état réel de la machine.

Tout ce que WAC écrit sur une copie est réversible : avant le rejeu, le contenu d'origine de chaque page remplacée est sauvegardé dans un journal d'annulation (`<ruche>.undo`). La copie brute, elle, reste intacte dans la consigne.

2.4 L'extraction en deux temps

L'emplacement des ruches de chaque utilisateur est inscrit dans la ruche `SOFTWARE`. WAC extrait donc d'abord les ruches de la machine (`SYSTEM`, `SOFTWARE`, `SAM`, `Amcache`), lit la liste des profils dans la copie de `SOFTWARE`, puis extrait les ruches de chaque profil (`ntuser.dat`, `usrClass.dat`).

Rien n'est supposé sur `C:` : le lecteur système est détecté, et un profil situé sur un autre disque (par exemple `D:\Users\...`) est lu sur son propre volume.

2.5 Ce qui reste lu « à chaud »

Quelques informations décrivent l'instant même de la collecte et ne figurent dans aucun fichier. Elles sont lues en direct, en une seule requête chacune, et enregistrées telles que lues dans la consigne (`exhibits\live`) et en même temps dans la copie de travail, où la conversion les relit :

- la liste des **processus** en cours (instantané du noyau) ;
- les **sessions** ouvertes ;
- l'**état courant des services** (la configuration, elle, est lue dans la ruche copiée) ;

— l'**horloge** et l'heure de démarrage ;

— les **réglages** de la machine en marche : lecteur système, page de code, fuseau horaire, volumes et leurs numéros de série.

CHAPITRE 3

Préparer une collecte

3.1 Prérequis

— Windows 10 ou ultérieur, 64 bits. WAC le vérifie au lancement.

— Des **droits d'administrateur** : la lecture directe d'un volume les exige. WAC le vérifie également.

— Des volumes **NTFS**, **FAT** ou **exFAT**. Un fichier situé sur un autre système de fichiers (CD-ROM) ne peut pas être lu directement ; il est signalé comme non lu, jamais ouvert autrement.

3.2 Le support de collecte

WAC s'exécute depuis le support de collecte (en général une clé USB) et y écrit tous ses résultats. Choisissez un support **vide**, de préférence en USB 3 : l'écriture est le poste le plus coûteux d'une collecte.

La place nécessaire dépend du mode et des options. Deux emplacements se remplissent : la **consigne** (copies brutes scellées) et la **copie de travail**. À la collecte, celle-ci ne reprend que ce que les collecteurs lisent sur place (ruches et leurs journaux, état de la machine en marche, dernières extractions) ; à la conversion, sur le poste d'analyse, elle est refaite depuis la consigne entière, chaque pièce recopiée et vérifiée par son empreinte, et la consigne n'est plus lue ensuite. Mesures du 29 octobre 2026 sur la machine virtuelle de test (Windows 11, un volume système et un volume de données), avec le `wac.yml` du kit :

Opération	Consigne	Copie de travail	Durée
Collecte, kit livré (`binary: unverified`)	6,8 Go	1,7 Go	10 min 26
Collecte sans les exécutables (`binary: none`)	5,9 Go	1,6 Go	3 min 46
Conversion de cette dernière, sur le poste d'analyse	—	5,9 Go	3 min 29

Les postes les plus lourds de la consigne du kit :

Contenu	Consigne
Fichiers supprimés récupérés : 10 338, au plus 4 Gio par défaut (`deleted_files`, `max_total_mib`)	4,2 Go
Binaires copiés : 2 766 non authentifiés sur 40 531 lus	850 Mo
Métadonnées NTFS : `$MFT` (400 Mo), `$LogFile` (73 Mo), `$UsnJrnl:$J` (42 Mo : NTFS a libéré le début, qui n'est pas copié)	515 Mo
Journaux d'événements (`--events`)	250 Mo
Fichiers de ressources des fournisseurs d'événements	230 Mo
Ruches et leurs journaux	130 Mo

Une machine utilisée depuis longtemps, ou chargée de logiciels, dépasse ces chiffres : la `$MFT` croît avec le nombre de fichiers, les binaires tiers non authentifiés avec les logiciels installés, les fichiers supprimés jusqu'à leur budget. Prévoir **au moins 12 Go libres** pour une collecte avec le kit, et, sur le poste d'analyse, la taille de la consigne pour sa conversion.

WAC vérifie l'emplacement **avant la toute première écriture** et refuse de démarrer :

— si le répertoire de travail contient déjà des fichiers — une collecte antérieure serait analysée à la place de celle-ci ;

— si la place libre est manifestement insuffisante : il demande deux fois l'estimation de la consigne, budget des fichiers supprimés compris.

La conversion vérifie de même, avant de refaire la copie de travail, que le poste d'analyse a la place de toutes les pièces. Si la place vient à manquer en cours de prélèvement des binaires, ceux-ci sont hachés sans être copiés, et le journal d'investigation l'indique.

3.3 Avant d'intervenir

— Utilisez un **dossier de sortie neuf** pour chaque machine (option `--output`).

— Notez l'heure de l'intervention et l'identité de l'opérateur : WAC les consigne aussi, mais une trace indépendante est utile.

— Gardez à l'esprit que brancher la clé et lancer WAC laissent des traces inévitables (chapitre 8) : elles sont connues et documentées, afin de ne pas être confondues avec l'activité étudiée.

CHAPITRE 4

Lancer une collecte

4.1 Ligne de commande

Ouvrez une invite de commandes **en tant qu'administrateur**, placez-vous sur le support de collecte, puis lancez :

```
WAC.exe [--collect | --convert=dossier] [--events] [--binary | --binary-all]
        [--threads=N] [--dump]
        [--output=dossier] [--loglevel=N] [--debug]
WAC.exe --update-trust[=dossier]
```

Toutes les options sont facultatives et désactivées par défaut.

4.2 Fichier de configuration : `wac.yml`

Version. WAC numérote ses versions `<majeur>.<mineure>.<correctif>` : majeur pour une refonte, mineure pour un ajout de fonctionnalités, correctif pour des corrections. La version se lit dans les propriétés de `WAC.exe` (onglet Détails), s'affiche au lancement, et est consignée dans `investigation.json` (`Tool.Version`) : une collecte dit par quelle version elle a été faite. Ce document décrit la version 3.0.1.

La dernière version compilée se trouve dans `bin` du dépôt : `WAC.exe`, le `wac.yml` de référence, les scripts qui vérifient une collecte (section 6.3) et `SHA256SUMS` pour vérifier le tout. Copier `WAC.exe` et `wac.yml` sur la clé de collecte suffit ; les scripts restent avec l'enquêteur.

Plutôt que de retaper les options à chaque collecte, la procédure s'écrit une fois dans `wac.yml`, à côté de `WAC.exe` : WAC l'applique d'abord, et une option donnée en ligne de commande la surcharge. `--write-config` écrit le fichier de référence, commenté — la procédure recommandée : binaires non vérifiés prélevés, journaux d'événements compris.

```
wac_version: 3.0.1      # version de WAC pour laquelle le fichier est écrit (--write-
    config)
binary: unverified      # none | unverified | all
threads: 0              # fils d'analyse des binaires, 0 : automatique
output: output
log_level: 0
dump: false
language: fr            # fr | en : langue de la console ; les fichiers restent en anglais
artefacts:              # une clé par artefact ; une clé absente vaut true
  activities_cache:        true   # Chronologie Windows : applications, fichiers, pages,
    presse-papiers (ActivitiesCache.db)
  amcache:                 true   # Amcache.hve : applications, exécutables et pilotes vus
  bam:                     true   # dernière exécution de chaque programme par utilisateur
```

```
    (BAM)
  bits:                    true   # qmgr.db : travaux BITS, URL, destinations, commandes
    de notification
  boot_records:            true   # ce qui s'exécute avant Windows : premier Mio de chaque
     disque (MBR, GPT), secteurs de démarrage, fichiers de la partition EFI
  cloud_identity:          true   # inscription Entra ID de la machine et comptes
    professionnels des utilisateurs ; caches de jetons copiés chiffrés
  defender:                true   # Defender : journaux (MPLog, traces WPP brutes),
    historique des détections, quarantaine (copiée chiffrée)
  defender_configuration:  true   # exclusions et réglages de protection de Microsoft
    Defender
  defender_endpoint:       true   # dossier local de Defender for Endpoint, là où il est
    inscrit (sorties de Live Response, données cyber), copie brute seule
  deleted_files:                  # fichiers supprimés dont la lecture brute retrouve
    encore le contenu
    enabled:               true   # false : rien n'est lu
    level:                 2      # 1 : contenu des enregistrements supprimés de la $MFT ;
     2 : et noms restés dans les index
                                   # 3 : et fichiers taillés dans l'espace libre (PE, ZIP,
    PDF) — tout l'espace libre lu
    max_file_mib:          64     # au-delà : relevé avec ses empreintes, pas copié ;
    unlimited : aucun plafond
    max_total_mib:         4096   # total copié ; les plus récents d'abord ; unlimited :
    aucun plafond
    excluded_folders:              # jamais lus, sur chaque volume
      - \Windows\WinSxS
      - \Windows\SoftwareDistribution
      - \Windows\servicing
      - \Windows\Installer\$PatchCache$
      - \Windows\System32\catroot
      - \Windows\System32\catroot2
      - \$Extend\$RmMetadata
  devices:                 true   # périphériques USB et SCSI et leurs dates, portables,
    noms de volumes
  dpapi_master_keys:       true   # clés maîtresses DPAPI copiées chiffrées ; JSON :
    propriétaire, GUID, dates, jamais la clé
  emails:                  true   # Outlook (PST, OST) et Thunderbird (mbox) : messages,
    destinataires, dates ; corps et pièces jointes exportés
  error_reports:           true   # rapports d'erreurs (WER) : programmes plantés, avec
    leur chemin
  etw_logs:                true   # fichiers de trace ETW que Windows garde sur son disque
     (.etl : sessions de démarrage, diagnostics, Windows Update), copie brute seule
  event_log_configuration: true   # état de chaque journal et session ETW de démarrage :
    actif, taille, rétention, accès (hors ligne)
  events:                  true   # journaux d'événements, avec leurs messages
  explorer_recent_docs:    true   # fichiers ouverts par chaque utilisateur (RecentDocs de
     l'Explorateur)
  fat_volumes:             true   # volumes FAT et exFAT, clés USB du suspect comprises :
    entrées, supprimées aussi
  firewall:                true   # règles et profils du pare-feu
  firewall_log:            true   # journal du pare-feu (pfirewall.log), s'il est activé :
     connexions acceptées ou rejetées, adresses, ports, processus
  hosts:                   true   # le fichier hosts : noms détournés vers une adresse
```

```
  installed_programs:      true   # programmes installés (Uninstall) de la machine et de
    chaque utilisateur
  internet_settings:       true   # proxy des utilisateurs
  intune_logs:             true   # journaux de l'extension Intune : scripts et
    applications poussés par le locataire, leur résultat
  jump_lists:              true   # listes de sauts : fichiers et lieux ouverts, par
    application
  known_networks:          true   # réseaux auxquels la machine s'est connectée (
    NetworkList)
  local_groups:            true   # membres de chaque groupe local (SAM)
  log_file:                true   # $LogFile et $TxfLog : journaux de transactions NTFS et
     TxF, copie brute seule
  mft:                     true   # $MFT : chaque fichier, présent ou supprimé, et ses
    dates
  mounted_devices:         true   # volumes et lettres de lecteur connus de la machine (
    MountedDevices)
  mount_points:            true   # volumes et partages vus par chaque utilisateur (
    MountPoints2)
  mru:                     true   # fichiers ouverts ou enregistrés par les boîtes de
    dialogue communes
  mru_apps:                true   # programmes qui ont ouvert ou enregistré ces fichiers
  muicache:                true   # noms des programmes lancés par chaque utilisateur
  network:                 true   # en direct : connexions, voisins, interfaces, routes
  network_configuration:   true   # cartes réseau, dernière configuration IPv4/IPv6, baux
    DHCP (hors ligne)
  network_shares:          true   # partages réseau de la machine, chemin et droits
  ntfs_volumes:            true   # $Boot et $UsnJrnl:$Max : numéro de série et géométrie,
     création du journal
  office_trust_records:    true   # documents Office approuvés, macros activées
  onedrive:                true   # journaux (.odl) et réglages de OneDrive de chaque
    profil, base de synchronisation comprise, copie brute seule
  operating_system:        true   # version, installation, nom de la machine, durée de
    fonctionnement
  persistence:             true   # persistance du registre : IFEO, AppInit, Winlogon, LSA
    , …COM
  powershell_history:      true   # PSReadLine : les commandes tapées dans PowerShell
  powershell_profiles:     true   # profils PowerShell de la machine et de chaque
    utilisateur : une persistance
  prefetch:                true   # Prefetch : programmes lancés, compteur, huit dernières
     exécutions, fichiers chargés
  processes:               true   # en direct
  program_usage:           true   # programmes lancés ou utilisés (FeatureUsage,
    JumplistData, compatibilité, épinglés)
  rdp_client:              true   # Bureau à distance sortant : serveurs, fichiers .rdp,
    tuiles du cache bitmap
  rdp_server:              true   # Bureau à distance entrant : authentifications,
    sessions, adresses source
  recent_documents:        true   # Recent : raccourcis vers les fichiers ouverts par
    chaque utilisateur
  recycle_bin:             true   # $I des corbeilles : quoi, par qui, quand
  registry_backups:        true   # RegBack : anciennes ruches de la machine, copie brute
    seule
  root_certificates:       true   # certificats racines de la machine et des utilisateurs
```

```
  run_keys:                true   # clés Run et RunOnce de la machine et de chaque
    utilisateur
  scheduled_tasks:         true   # tâches planifiées : commandes, déclencheurs, auteurs,
    dates
  search_index:            true   # index de recherche Windows : chemin, dates, taille, ré
    sumé de chaque élément indexé
  security:                true   # ruche SECURITY : noms des secrets LSA, connexions en
    cache, audit
  security_descriptors:    true   # $Secure:$SDS : propriétaire et droits de chaque
    fichier, chaque descripteur une fois
  services:                true   # services et pilotes
  sessions:                true   # en direct
  setupapi:                true   # setupapi.dev.log : installations de périphériques
  shadow_copies:           true   # clichés instantanés lus en brut, sans le service VSS :
     ruches et journaux de chaque cliché
  shellbags:               true   # dossiers parcourus par chaque utilisateur dans l'
    Explorateur
  shimcache:               true   # cache de compatibilité des applications (
    AppCompatCache)
  shortcuts:               true   # raccourcis hors de Recent : menu Démarrer, Bureaux,
    Quick Launch, barre des tâches
  srum:                    true   # SRUDB.dat : réseau et ressources par application et
    utilisateur, 30 à 60 jours
  startup_folders:         true   # dossiers de démarrage : ce qui s'exécute à l'ouverture
     de session
  sysmon:                  true   # Sysmon s'il est installé : sa configuration (règles
    telles que stockées) et les fichiers supprimés qu'il a archivés ; son journal est dans
     events
  thumbcache:              true   # caches des miniatures et des icônes : ce que chacun a
    vu, même supprimé ; images exportées
  typed_paths:             true   # chemins tapés dans l'Explorateur
  usbstor:                 true   # périphériques de stockage USB (Enum\USBSTOR)
  userassist:              true   # programmes lancés depuis l'Explorateur, avec compteur
    et date
  users:                   true   # comptes locaux (SAM) : connexions, échecs, indicateurs
    , condensats
  user_shell_folders:      true   # où sont vraiment le Bureau, les Documents, les Télé
    chargements de chacun
  usn_journal:             true   # $UsnJrnl:$J : créations, renommages, suppressions
  web_client:              true   # navigateurs Chromium et Firefox : visites, télé
    chargements, recherches ; leurs bases copiées brutes
  web_server:                     # IIS : sites, pools, modules natifs, journaux, chaînes
    de connexion, comptes
    enabled:               true   # false : rien n'est lu ; le dossier de chaque site copi
    é entier
    max_database_mib:      1024   # base hors des sites, données d'un moteur, au-delà :
    signalés, pas copiés ; unlimited : aucun plafond
  wmi:                     true   # dépôt WMI lu hors ligne : abonnements (filtres,
    consommateurs, liaisons)
  word_wheel_query:        true   # recherches de l'Explorateur
  zone_identifiers:        true   # flux Zone.Identifier : d'où viennent les fichiers télé
    chargés
```

— **Strict** : une clé inconnue, une valeur hors de celles admises (`true`/`false` seulement, pas `yes`), une clé en double, une liste : la collecte est refusée avant de commencer, avec la ligne fautive. Une clé absente garde sa valeur par défaut : `true`.

— **Versionné** : `wac_version` porte la version de WAC pour laquelle le fichier a été écrit (`--write-config` l'y met). Un fichier d'une autre version, ou sans version, est refusé avant toute lecture : il pourrait manquer des réglages ajoutés depuis, ou en porter qui n'existent plus. Le refaire avec `--write-config`.

— **Une clé par artefact**, aucun prélevé d'office. Les ruches, les journaux d'événements et la `$MFT` ne sont pas des artefacts mais des *dépendances* : prises dès qu'un artefact demandé les lit, et seulement alors. Les journaux servent à `events` et à `rdp_server` ; les ruches des utilisateurs à `muicache`, `userassist`, `run_keys`, `shellbags`, `mru`, `mru_apps` et aux autres artefacts lus dans `NTUSER.DAT` ; `Amcache.hve` au seul `amcache`. SYSTEM, SOFTWARE, SAM, SECURITY et DEFAULT sont toujours prises : elles donnent le fuseau horaire, les profils et le nom des comptes.

— **Consigné** : chemin, SHA-256 et contenu du fichier vont dans `investigation.json` (`Tool.Configuration`) — la procédure appliquée fait partie de la preuve.

— Un artefact désactivé n'est pas lu du tout, et son fichier JSON le dit (`CollectionStatus NotRequested`, avec la clé) : jamais confondu avec un échec ou une absence de traces.

— `--config=fichier` lit un autre fichier.

— Sans `wac.yml`, WAC collecte tous les artefacts, journaux d'événements compris, options telles que données.

4.3 Options

Option	Effet
`--collect`	Le mode par défaut, dit explicitement : collecte, sur la machine examinée — relevés en direct et extraction brute dans la consigne scellée, rien n'est converti. Voir chapitre 5.
`--convert=dossier`	Conversion seule, sur un poste d'analyse, de la collecte faite avec `--collect` dans ce dossier, après vérification du sceau et de chaque empreinte. Voir chapitre 5.
`--events`	Prend les journaux d'événements (`.evtx`, ou `.evt` sous Windows XP et Server 2003) même si `wac.yml` dit `events: false` ; sans cela, ils sont pris par défaut (clé `events`). Chaque événement reçoit son message lisible, reconstitué à partir des ressources de son fournisseur. Environ 250 Mo de copies.

Option	Effet
`--binary`	À la collecte, lit directement chaque exécutable, bibliothèque, pilote, script et document à macros des volumes fixes, vérifie sa signature en mémoire et **prélève** dans la consigne ceux qu'elle n'authentifie pas ; à la conversion, chaque fichier cité par un artefact reçoit les empreintes de sa pièce. Voir section 4.5.
`--binary-all`	Comme `--binary`, mais **tout** exécutable est prélevé, authentifié ou non ; la signature est tout de même vérifiée, et le manifeste dit pour chacun si elle est valide. Pour les dossiers où le contenu même doit figurer dans la consigne.
`--threads=N`	Nombre de fils qui analysent les exécutables de `--collect --binary` (PE, Authenticode, catalogues, empreintes). Par défaut, le nombre de fils du processeur moins un, la lecture du disque gardant le sien. Le résultat n'en dépend pas : le manifeste est le même.
`--update-trust [=dossier]`	Sur le **poste d'analyse**, **avant** la collecte, et seule : télécharge chez Microsoft la liste des racines de confiance (`authroot.stl`), celle des certificats désapprouvés (`disallowedcert.stl`) et chacune des racines, vérifie leurs signatures et leurs empreintes, ainsi que les listes de pilotes vulnérables de Microsoft et de LOLDrivers (non signées : authentifiées par HTTPS seul) et les listes de révocation (CRL) des autorités de signature de code que recense la CCADB, de celles propres à Microsoft et de celles qu'une collecte précédente a trouvées manquantes (`wanted-crls.txt`), et les écrit dans `dossier` (par défaut `trust`, à côté de `WAC.exe`), à emporter sur la clé avec `WAC.exe`. Seul mode qui utilise le réseau : la bibliothèque réseau n'est chargée que là. Refusée avec toute autre option. Sous Windows 7, les sites servis en TLS 1.2 seul (listes de pilotes, CCADB) demandent la mise à jour KB3140245 : sans elle, ces listes manquent au jeu de confiance, et `investigation.json` le dit.
`--dump`	Ajoute la valeur hexadécimale brute des éléments de shellbags et de raccourcis `.lnk`, pour un décodage ultérieur.
`--output=dossier`	Dossier de sortie, relatif au répertoire courant : un nom, sans `\`, `/` ni `:`, autre que `..` — la collecte ne quitte pas la clé. Par défaut : `output`.
`--language=fr|en`	Langue de la console : étapes, aide, messages d'erreur. Français par défaut ; `language:` dans `wac.yml` la fixe aussi. Les fichiers JSON, `investigation.json` et le journal texte restent en anglais, quelle que soit la langue : ce sont des pièces lues par des outils, et une collecte donne les mêmes fichiers quel que soit l'opérateur.

Option	Effet
`--loglevel=N`	Active un journal texte détaillé, dans le répertoire courant, nommé d'après le programme lancé (`WAC.exe.log` pour `WAC.exe`) : 0 aucun, 1 par type d'artefact, 2 par artefact, 3 par sous-fonction (mise au point).
`--debug`	Affiche sur la sortie d'erreur la trace du lecteur NTFS (résolution des chemins, blocs d'index, séquences de grappes). Réservé au diagnostic.
`--help, /?`	Affiche l'aide.

4.4 Exemples

Collecte standard :

```
E:\> WAC.exe --output=poste-comptable-01
```

Collecte complète, avec journaux, binaires et journal détaillé :

```
E:\> WAC.exe --events --binary --output=poste-comptable-01 --loglevel=2
```

4.5 L'option `--binary`

Une empreinte suffit pour interroger une base publique (VirusTotal, par exemple) **sans lui envoyer le fichier**. Mais elle ne dit rien d'un binaire que personne ne connaît — précisément celui qui intéresse l'enquête — et un binaire non prélevé peut avoir disparu quand une détection survient. C'est pourquoi `--binary` copie aussi les charges potentielles :

— **prélevés** : `exe`, `dll`, `sys`, `ocx`, `cpl`, `scr`, `drv`, `efi`, `com`, `msi` et les scripts `ps1`, `psm1`, `bat`, `cmd`, `vbs`, `vbe`, `js`, `jse`, `wsf`, `wsh`, `hta` ;

— **prélevés aussi** : les documents Office **capables de porter des macros**, vecteur d'intrusion aussi courant qu'un exécutable — formats binaires anciens (`doc`, `xls`, `ppt`, `pps`, `dot`, `xlt`, `pot`), formats « m » (`docm`, `dotm`, `xlsm`, `xltm`, `pptm`, `potm`, `ppsm`), `xlsb`, compléments (`xla`, `xlam`, `xll`, `wll`, `ppa`, `ppam`), Publisher (`pub`), Visio (`vsd`, `vsdm`, `vstm`, `vssm`) et Access (`mdb`, `accdb`, `accde`). Ils apparaissent dans les traces comme cibles de raccourcis ou de listes de sauts, ou parmi les fichiers chargés par Word ou Excel dans leur Prefetch ;

— **seulement hachés** : les autres documents et fichiers de données, y compris `docx`, `xlsx` et `pptx`, qui ne peuvent pas contenir de macros. Les copier ferait de la collecte une copie des fichiers de l'utilisateur.

Les fichiers cités proviennent des processus, services (binaire et DLL), tâches planifiées,

fichiers chargés par les programmes (Prefetch), Shimcache, Amcache et cibles de raccourcis et de listes de sauts. Chaque fichier n'est lu qu'une fois, quel que soit le nombre d'artefacts qui le citent, et **un même contenu n'est stocké qu'une fois** : les autres chemins figurent au manifeste comme pièces partageant ce contenu (`SharedExhibit`).

Binaires Microsoft authentiques : hachés, non prélevés

La plupart des binaires cités sont des composants de Windows, identiques sur toute machine de la même version. Leur origine se prouve sur la machine elle-même, sans liste à embarquer : les **catalogues** de Windows, signés par Microsoft, listent l'empreinte de chaque fichier du système. WAC lit ceux de `System32\CatRoot`, ceux des composants (`WinSxS\Catalogs`) et ceux des paquets de maintenance (`servicing\Packages`), plus de 13 000 sur Windows 11. Les binaires signés individuellement (Edge, OneDrive, Office, .NET) portent une **signature intégrée**. Un binaire authentifié ainsi est haché sans être prélevé ; tous les autres sont prélevés — y compris un binaire système remplacé par un attaquant, dont l'empreinte ne correspond plus.

Cette vérification ne laisse **aucune trace** : catalogues et binaires sont lus directement sur le disque, et la vérification cryptographique est faite en mémoire par WAC, jusqu'à des certificats racines Microsoft embarqués dans l'exécutable. Aucun service n'est sollicité, le magasin de certificats de la machine n'est pas consulté, et la révocation n'est pas contrôlée en ligne.

Seuls les signataires Microsoft pour son propre code sont acceptés (« Microsoft Windows », « Microsoft Corporation », « Microsoft Windows Publisher » et « .NET »). Les pilotes **tiers** certifiés par Microsoft (signataires « Hardware Compatibility Publisher » et « Early Launch Anti-malware Publisher ») restent prélevés : un pilote vulnérable mais signé est un moyen d'attaque classique.

Applications du Store : vérifiées par leur paquet

Les fichiers d'une application du Store ne sont pas signés un à un : c'est leur **paquet** qui l'est. Son dossier contient la signature (`AppxSignature.p7x`) et la carte des blocs (`AppxBlockMap.xml`), qui donne le SHA-256 de chaque bloc de 64 Kio de chaque fichier. WAC vérifie la signature, qui doit remonter à une racine Microsoft (le Store signe les paquets qu'il distribue), vérifie qu'elle garantit bien cette carte, puis compare chaque bloc lu à la carte. Un fichier conforme est haché sans être prélevé ; un fichier modifié, ou ajouté au dossier après l'installation, est prélevé, de même que tout fichier d'un paquet signé hors de la chaîne Microsoft.

Les catalogues, signatures et cartes de blocs qui ont justifié un non-prélèvement sont consignés, pour qu'un tiers puisse refaire la vérification. Dans les fichiers d'artefacts, le champ `Signature` (ou `ServiceDllSignature`, `SignatureTarget`…) indique l'authentification retenue, par exemple « `Microsoft (catalog C:\Windows\….cat)` », « `Microsoft (embedded signature)` » ou « `Package (package C:\Program Files\WindowsApps\…)` ».

Les **scripts** sont authentifiés de la même façon : par les catalogues de Windows, et, pour PowerShell (`ps1`, `psm1`, `psd1`, `ps1xml`…), par leur bloc de signature intégré (`# SIG # Begin signature block`). Un script Windows Script Host (`vbs`, `js`, `wsf`) signé seulement dans son

texte, et non par un catalogue, reste prélevé.

Sur la machine de test, cette vérification ramène le prélèvement de 2 131 binaires (3,3 Go) à 92 (335 Mo), sans allonger la collecte ; les scripts encore prélevés sont ceux qui ne sont pas signés.

La lecture étant directe, prélever ne laisse aucune trace de plus que hacher : seulement de la place sur le support.

4.6 Déroulement à l'écran

WAC affiche chaque étape, suivie de `OK` ou d'un message d'erreur. Une étape en échec n'interrompt pas la collecte : tout ce qui est accessible est recueilli, et ce qui manque est consigné.

```
[VÉRIFICATION DES PRÉREQUIS]
 - Droits d'administrateur : OK
 - Contrôle du support de collecte : OK
[OBSERVATION DE LA MACHINE EN MARCHE]
 - Sessions : OK
 - Processus : OK
 - État du réseau : OK
[EXTRACTION BRUTE]
 - Ruches du système (lecture brute NTFS) : OK
 - Ouverture de la ruche SYSTEM : OK
 - Ouverture de la ruche SOFTWARE : OK
 - Profils des utilisateurs (ruche SOFTWARE) : OK
 - Ruches des utilisateurs (lecture brute NTFS) : OK
 - Artefacts fichiers (lecture brute NTFS) : OK
[ARTEFACTS DU REGISTRE]
 - Clés UserAssist du registre : OK
   ...
[ARTEFACTS DES FICHIERS]
 - Prefetch : OK
   ...
[JOURNAUX D'ÉVÉNEMENTS]
 - Journaux d'événements : OK
[CONSIGNE]
 - Copie des binaires prélevés dans la copie de travail : OK
 - Scellement de la consigne (manifeste + SHA-256) : OK
[JOURNAL D'INVESTIGATION]
 - Écriture de investigation.json : OK
FIN, durée : 309 s
```

Avec `--language=en`, ou `language: en` dans `wac.yml`, les mêmes étapes s'affichent en anglais.

Le scellement de la consigne est la **dernière** opération sur celle-ci : toutes les pièces, y compris celles ajoutées pendant l'analyse des journaux, sont couvertes par le manifeste.

4.7 Durée

La durée dépend surtout du matériel (USB 2 ou 3, disque, processeur) et des options. Sur la machine virtuelle de test (disque NVMe) : environ 70 s avec `--events`, environ 310 s avec `--events` et `--binary`. Sur une clé USB, l'écriture domine.

CHAPITRE 5

Collecter ici, convertir ailleurs

5.1 Pourquoi séparer

Une exécution de WAC fait deux choses différentes :

1. **la collecte** : relever ce que seul un système en marche montre (processus, sessions, état des services, horloge), lire le disque directement, copier les preuves dans une consigne scellée. Elle a **besoin** de la machine examinée ;
2. **la conversion** : décoder en JSON des milliers de fichiers copiés (ruches, journaux, Prefetch, raccourcis…). Elle n'a besoin que de la consigne, **pas** de la machine.

WAC ne fait **jamais les deux dans la même exécution** : il collecte sur la machine examinée, et convertit ailleurs. Convertir sur la machine y ajouterait un travail que rien n'impose : des minutes de traitement dans la mémoire de WAC, que Windows peut paginer dans `pagefile.sys`, sur le disque même qu'on examine ; un processus présent plus longtemps, vu et journalisé par les protections de la machine ; en cas de plantage, un rapport d'erreur Windows écrit sur la machine ; et des bibliothèques du système chargées pour décoder (le moteur ESE pour rejouer les journaux d'une base).

C'est précisément ce que les bonnes pratiques demandent d'éviter :

— **ISO/IEC 27037** traite de l'identification, de la collecte et de la préservation des preuves numériques, **ISO/IEC 27042** de leur analyse : deux étapes distinctes ;

— **principe 1 de l'ACPO** : aucune action ne doit modifier les données de la machine examinée ; toute opération qui s'y exécute doit être nécessaire ;

— **principe 3 de l'ACPO** : un tiers indépendant doit pouvoir refaire le traitement et obtenir le même résultat.

Séparer les deux étapes apporte trois choses :

— **moins de traces** sur la machine examinée : seule la collecte s'y exécute ;

— **des preuves vérifiées avant toute analyse** : `--convert` contrôle d'abord le sceau et le SHA-256 de chaque pièce, et refuse une collecte retouchée ;

— **une analyse reproductible** : deux conversions d'une même collecte donnent des fichiers JSON identiques octet pour octet, et l'analyse peut être refaite plus tard, avec une version corrigée de WAC ou par un tiers, sans revenir sur la machine.

5.2 Les réglages de la collecte

— **Portée de `--binary`** : la collecte lit **tous les exécutables de tous les volumes fixes NTFS**, cités ou non, car elle ne sait pas encore lesquels les artefacts citeront. Ce temps (8 min 30 sur la machine de test) est de la lecture, qui n'écrit rien sur la machine.

— **`--binary-all`** : quand le contenu de chaque exécutable doit figurer dans la consigne (analyse de logiciel malveillant, demande des fichiers eux-mêmes), pour environ 20 Go.

5.3 Les deux commandes

Sur la machine examinée, en administrateur :

```
E:\> WAC.exe --collect --events --binary --output=poste-comptable-01
```

Sur le poste d'analyse, sans droits particuliers, avec la même configuration (`events`, `--binary`) que la collecte :

```
D:\> WAC.exe --convert=E:\poste-comptable-01 --events --binary
```

Les fichiers JSON sont écrits dans le dossier de la collecte, à côté de la consigne. `--convert` exclut `--collect` et `--output`.

5.4 Ce que prend `--collect`

La conversion ne pouvant plus rien demander à la machine, la collecte prend **tout ce qu'elle lira** :

— les **relevés en direct**, enregistrés tels que lus dans `exhibits\live` et scellés comme toute pièce : processus, sessions, état des services, horloge et démarrage, et les réglages de la machine (lecteur système, page de code, fuseau horaire, volumes et leurs numéros de série) ;

— les ruches et les fichiers d'artefacts, comme d'ordinaire ;

— avec les journaux d'événements (`events`), les fichiers de ressources de **tous** les fournisseurs d'événements déclarés (binaire, fichier de paramètres, satellites `.mui`), car on ne sait qu'à la conversion lesquels seront utiles. Cela représente environ 650 Mo sur Windows 11 ;

— avec `--binary`, **tous** les exécutables, bibliothèques, pilotes, scripts et documents à macros de **tous les volumes fixes NTFS** — un profil sur `D:` compris —, lus directement et vérifiés en mémoire (section 4.5) :

  — authentifié (catalogue, signature intégrée, paquet du Store), ou binaire tiers dont la signature tient face au jeu de confiance de la clé (`--update-trust` : chaîne vers une racine admise par Microsoft pour la signature de code, valide à la date

de signature, non révoquée, pas un pilote vulnérable) : **non copié** ; sans jeu de confiance utilisable, tout binaire tiers est copié, et le journal d'investigation le dit ; le manifeste garde son chemin, ses empreintes Authenticode et le verdict (`ContentStored` à `false`) ;

— non authentifié : **copié**, avec ses trois empreintes MD5, SHA-1, SHA-256, son empreinte Authenticode et la raison ;

— fichier différentiel de Windows (`PA30`/`PA31`, dossiers `r` et `f` de `WinSxS`) : ce n'est pas un exécutable ; haché, non copié.

Un même contenu n'est stocké qu'une fois, et les autres noms d'un lien physique (`System32` et `WinSxS`) sont déclarés au manifeste comme le premier. Les volumes amovibles, dont le support de collecte, et les fichiers de mémoire (`pagefile.sys`, `swapfile.sys`, `hiberfil.sys`) sont exclus.

> **Attention**
> Avec `--binary`, la collecte lit tous les volumes : sur la machine virtuelle de test (Windows 11 sans logiciel ajouté), 44 446 exécutables lus, dont 41 593 authentifiés (ou tiers à signature valide) non copiés et 2 853 copiés (1,2 Go) ; la collecte entière dure 8 minutes 30 et la consigne, journaux d'événements et métadonnées NTFS compris, pèse 2,7 Go (voir la section 3.2). Si la place vient à manquer, les fichiers restants sont hachés sans être copiés, et le manifeste l'indique.

Aucun fichier JSON d'artefact n'est écrit : le dossier ne contient que la consigne scellée, le répertoire de travail et `investigation.json`.

5.5 Ce que fait `--convert`

1. **Vérification**, avant toute lecture : le sceau doit porter l'empreinte réelle du manifeste, et chaque pièce son empreinte SHA-256. Une seule pièce altérée, et la conversion est **refusée** ; `conversion.json` en donne la raison.
2. **Reconstruction du répertoire de travail** depuis la consigne : l'ancien est supprimé plutôt que cru, les copies sont vérifiées contre le manifeste et les ruches réparées, comme à la collecte. Les exécutables et les ressources des fournisseurs sont lus dans la consigne même, sans copie.
3. **Conversion**, depuis les seules pièces : aucune lecture de la machine qui convertit, ni de son disque, ni de son registre, ni de son fuseau horaire.

La consigne n'est jamais modifiée et son sceau reste celui de la collecte. `investigation.json` reste le journal de la collecte ; `conversion.json` est celui de la conversion (opérateur, poste, opérations). Deux conversions d'une même collecte donnent des fichiers JSON identiques, ce que le harnais de test vérifie à chaque cycle.

5.6 Limites

— Les **documents à macros** (`doc`, `docm`, `xls`, `xlsm`, `ppt`…, liste de la section 4.5) sont traités comme les exécutables : lus sur tous les volumes fixes et **prélevés**, sauf s'ils sont, octet pour octet, un fichier livré par Microsoft et listé dans un catalogue — ce qu'un document écrit ou modifié par quiconque ne peut pas être.

— Un fichier cité qui ne peut porter ni code ni macro (`docx`, `xlsx`, `pdf`, donnée) n'est pas prélevé : en conversion, il n'a donc **pas d'empreinte**.

— Un exécutable authentifié n'est pas copié : à la conversion, il a son empreinte Authenticode et son verdict, mais pas de MD5, SHA-1 ni SHA-256 — ceux-ci ne sont calculés que pour ce qui est prélevé.

— Un exécutable situé sur un volume amovible n'est pas prélevé par `--collect --binary`.

— La configuration (`events`, `--binary`) de la conversion doit être celle de la collecte : ce que la collecte n'a pas pris ne peut pas être converti.

CHAPITRE 6

Les résultats

6.1 Arborescence

```
<sortie>/
  exhibits/            copies brutes, jamais réécrites
    MANIFEST.json     identification de chaque pièce et de la collecte
    MANIFEST.sha256   empreinte du manifeste : son sceau
    Windows/...        les pièces, sous leur chemin d'origine
    Users/...
  working/             copies de travail (ruches rejouées)
  investigation.json   journal de la collecte
  *.json               les artefacts
```

Un fichier d'un autre volume que le volume système est rangé sous `_volume_D\...` ; le JSON indique toujours le chemin d'origine, avec sa vraie lettre de lecteur.

Sous `exhibits/` et `working/`, un chemin dépasse souvent 260 caractères : celui d'origine, plus le dossier de sortie. WAC l'écrit en chemin étendu (`\\?\`). L'Explorateur et plusieurs commandes de Windows (`Copy-Item`, `Remove-Item` de PowerShell 5.1, une copie par glisser-déposer) s'arrêtent sur ces fichiers, parfois sans le dire. Pour copier ou effacer une collecte, utiliser `robocopy` ou `rd /s /q \\?\<chemin>` ; pour la vérifier, `verifier-consigne.ps1`, qui lit une pièce à chemin long en chemin étendu.

6.2 Le manifeste de consigne

`exhibits/MANIFEST.json` identifie la collecte et chaque pièce.

6.2.1 En-tête

`Tool`
nom de l'outil, ligne de commande, date de compilation.

`Host`
nom de la machine, lecteur système, fuseau horaire du suspect (lu dans sa ruche `SYSTEM`) et de la machine de collecte.

`Operator`
compte ayant lancé la collecte, et s'il était élevé.

`Custody`
déclaration de procédure, début de l'extraction, heure d'écriture du manifeste, nombre de

pièces et d'échecs, volume total, algorithmes d'empreinte, volumes lus (lettre, numéro de série, système de fichiers).

6.2.2 Chaque pièce

Champ	Signification
`SourcePath`	Chemin d'origine, avec lettre de lecteur.
`ExhibitPath`	Emplacement de la copie brute, relatif au dossier de sortie.
`Method`	Méthode de collecte (lecture directe NTFS, volume, motif).
`Result`	`OK`, ou le code et le message d'erreur. Les échecs figurent au manifeste : une pièce absente se lirait comme une pièce jamais cherchée.
`MD5, SHA1, SHA256`	Les trois empreintes, calculées pendant l'écriture : elles portent sur ce qui a été lu du volume.
`Bytes`	Taille extraite.
`DeclaredBytes`	Taille déclarée par NTFS, présente seulement si elle diffère de la taille extraite (extraction tronquée).
`ValidDataBytes`	Longueur de données valides d'un fichier préalloué (journaux d'événements) : au-delà, la pièce contient des zéros par définition.
`SharedExhibit`	`true` si le contenu, identique octet pour octet, est déjà stocké sous une autre pièce : `ExhibitPath` désigne celle-ci. La source reste une pièce à part entière, avec ses propres chemin, entrée `$MFT` et horodatages.
`MftEntry`	Numéro d'enregistrement NTFS : identifie le fichier indépendamment de son nom.
`Extracted(Utc)`	Instant de l'extraction de cette pièce.
`SourceCreated, SourceModified, SourceMftModified, SourceAccessed`	Les quatre horodatages NTFS du fichier d'origine (`$STANDARD_INFORMATION`), en UTC et en heure locale du suspect. Ce sont eux, et non les dates des copies, qui datent le fichier.
`SourceFileName Created, Modified, MftModified, Accessed`	Les quatre horodatages de l'attribut `$FILE_NAME` du nom lu. NTFS les tient à part, et une modification des dates par programme ne les touche pas : un `SourceCreated` antérieur à `SourceFileNameCreated` est le signe classique de dates falsifiées après coup (*timestomping*).
`ContentStored`	`false` : binaire authentifié lu mais non copié (`--collect --binary`) ; il n'a pas de `ExhibitPath`.

Champ	Signification
`AuthenticodeSHA1`, `AuthenticodeSHA256`	Empreintes Authenticode d'un PE, celles que couvrent sa signature et les catalogues ; seules empreintes d'un binaire authentifié non copié.
`SignatureVerified`	Résultat de la vérification de signature d'un exécutable : `true` avec `Signature` (ce qui l'authentifie), `false` avec `SignatureReason` (pourquoi).
`PeTimeDateStamp`, `PeSizeOfImage`, `SymbolServerKey`	Version d'un PE et clé (`nom/HORODATAGEtaille/nom`) sous laquelle le serveur de symboles de Microsoft la conserve : un binaire Microsoft authentique non copié peut en être retéléchargé, identique, et son empreinte revérifiée.

> **Pourquoi trois empreintes**
> Des collisions MD5 se fabriquent à volonté depuis 2008, et SHA-1 depuis 2017 : une empreinte seule ne tranche plus une contestation d'identité. Trois empreintes indépendantes, si.

6.3 Vérifier l'intégrité d'une collecte

6.3.1 Le sceau

Le manifeste ne peut pas contenir sa propre empreinte : elle est dans `MANIFEST.sha256`. Sous Linux :

```
$ cd sortie/exhibits
$ sha256sum -c MANIFEST.sha256
MANIFEST.json: OK
```

Le sceau relevé. `MANIFEST.sha256` prouve seulement que le manifeste n'a pas changé depuis son scellement : qui réécrit le manifeste peut réécrire son sceau. WAC affiche donc le sceau à l'écran en fin de collecte (« Sceau du manifeste (SHA-256), à reporter au procès-verbal ») : reporté au procès-verbal, c'est la seule copie hors d'atteinte de la collecte, et les scripts ci-dessous lui opposent le manifeste (`--seal` / `-Seal`).

6.3.2 Chaque pièce

Deux scripts, livrés avec `WAC.exe` dans `bin`, vérifient le sceau puis le SHA-256 de chaque pièce : `verifier-consigne.ps1` sous Windows, `verifier-consigne.sh` sous Linux ; ils lancent `verify-exhibits.ps1` et `verify-exhibits.sh`, les mêmes contrôles, avec des messages en français. Un manifeste sans sa liste de pièces est refusé, jamais pris pour une collecte vide ; une pièce illisible est comptée à part d'une pièce différente. Les scripts PowerShell demandent

PowerShell 4 ou plus (Windows 8.1 et suivants) : sous un PowerShell plus ancien, ils le disent et s'arrêtent (code 2) plutôt que de lire de travers.

Vérifier depuis une machine Linux. C'est la voie recommandée : la collecte peut contenir ce qui a infecté la machine examinée, et une machine d'investigation sous Linux constitue une rupture technologique — un exécutable ou un document à macro Windows de la collecte ne peut pas s'y exécuter. Le script ne demande que `bash`, les coreutils et `python3` (qui lit le JSON) ; les empreintes sont calculées par `sha256sum`, non par WAC :

```
$ bash verifier-consigne.sh --seal=DCFF53FA...2C3E32 /media/usb/output
Sceau conforme (DCFF53FAE709EC2E351FFFD02EC6EAD9945602265CF3363D0D928B0B452C3E32)
3360 piece(s) conforme(s), 0 differente(s)
0 binaire(s) authentifie(s), empreinte seule (pas de copie a verifier)
```

Le manifeste vient de la machine examinée : les deux scripts refusent un chemin de pièce qui sort de la collecte (`..`, chemin absolu, et pour le `.sh` un lien symbolique), et le comptent comme différent sans le lire.

Depuis Windows :

```
PS> powershell -ExecutionPolicy Bypass -File verifier-consigne.ps1 -Output E:\output -Seal
     DCFF53FA...2C3E32
Sceau conforme (DCFF53FAE709EC2E351FFFD02EC6EAD9945602265CF3363D0D928B0B452C3E32)
3360 piece(s) conforme(s), 0 differente(s)
```

Leur code de retour vaut 0 si tout est conforme, 1 sinon.

```
#!/usr/bin/env bash
# verify-exhibits.sh — checks the seal and every exhibit of a WAC collection,
# from a Linux investigation machine: the collection is examined away from the
# Windows machine it comes from, which it cannot infect there.
# Usage: bash verify-exhibits.sh [--language=en|fr] [--seal=<SHA-256>] /media/usb/output
#   --seal      the seal WAC printed at the collection, as written down in the
#               report: compared with MANIFEST.sha256 (see below)
#   --language  the messages' language; verifier-consigne.sh is this script
#               in French
# Exit code: 0 if everything matches, 1 otherwise.
# Needs: bash, coreutils (sha256sum, realpath), python3 (reads the JSON only).
#
# The fingerprints are computed by sha256sum, not by WAC: a verification that
# does not rely on the tool that produced the collection.
#
# The seal beside the manifest proves only that the manifest is unchanged
# since it was sealed: whoever rewrites the manifest can rewrite its seal.
# Against that, the value written down at the collection (--seal).
#
# The manifest comes from the examined machine, so it is treated as hostile:
# an exhibit path that leaves the collection (.., absolute path, symbolic
# link) is refused and counted as a mismatch, never read; a manifest without
# its list of exhibits is refused, never taken for an empty collection.

set -u
language=en; expected_seal=""; output="./output"
for argument in "$@"; do
```

```
    case "$argument" in
        --language=*) language="${argument#--language=}" ;;
        --seal=*) expected_seal="$(printf '%s' "${argument#--seal=}" | tr 'a-f' 'A-F')" ;;
        *) output="$argument" ;;
    esac
done

# The messages, in the language asked for. $1: the message's key; the rest
# goes into its text.
say() {
    local key="$1"; shift
    if [ "$language" = fr ]; then
        case "$key" in
            tool) echo "OUTIL ABSENT : $1" ;;
            absent) echo "MANIFESTE ABSENT : $1 ou $2" ;;
            seal) echo "SCEAU NON CONFORME : manifeste modifie" ;;
            noted) echo "SCEAU NON CONFORME : $1 au lieu de $2, releve a la collecte" ;;
            sealed) echo "Sceau conforme ($1)" ;;
            unnoted) echo "Sceau compare a MANIFEST.sha256 seul : donner --seal=<valeur
    relevee a la collecte> pour l'opposer a une reecriture" ;;
            unreadable) echo "MANIFESTE ILLISIBLE : $1" ;;
            mismatch) echo "DIFFERENTE : $1" ;;
            total) echo "$1 piece(s) conforme(s), $2 differente(s)" ;;
            binaries) echo "$1 binaire(s) authentifie(s), empreinte seule (pas de copie a
    verifier)" ;;
        esac
    else
        case "$key" in
            tool) echo "MISSING TOOL: $1" ;;
            absent) echo "NO MANIFEST: $1 or $2 absent" ;;
            seal) echo "SEAL MISMATCH: the manifest was modified" ;;
            noted) echo "SEAL MISMATCH: $1 instead of $2, written down at the collection"
    ;;
            sealed) echo "Seal verified ($1)" ;;
            unnoted) echo "Seal checked against MANIFEST.sha256 only: give --seal=<value
    written down at the collection> to hold it against a rewrite" ;;
            unreadable) echo "UNREADABLE MANIFEST: $1" ;;
            mismatch) echo "MISMATCH: $1" ;;
            total) echo "$1 exhibit(s) verified, $2 mismatch(es)" ;;
            binaries) echo "$1 binary(ies) authenticated and fingerprinted only (no copy
    to verify)" ;;
        esac
    fi
}

manifest="$output/exhibits/MANIFEST.json"
seal="$output/exhibits/MANIFEST.sha256"
for tool in sha256sum realpath python3; do
    command -v "$tool" >/dev/null || { say tool "$tool"; exit 1; }
done
[ -f "$manifest" ] && [ -f "$seal" ] || { say absent "$manifest" "$seal"; exit 1; }
root="$(realpath -e -- "$output")" || exit 1
```

```
# 1. The seal: the manifest's fingerprint must be the one recorded beside it,
# and the one written down at the collection when it is given.
recorded="$(head -c 64 -- "$seal" | tr 'a-f' 'A-F')"
actual="$(sha256sum -- "$manifest" | head -c 64 | tr 'a-f' 'A-F')"
if [ "$recorded" != "$actual" ]; then say seal; exit 1; fi
if [ -n "$expected_seal" ] && [ "$expected_seal" != "$actual" ]; then say noted "$actual"
    "$expected_seal"; exit 1; fi
say sealed "$actual"
[ -n "$expected_seal" ] || say unnoted

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
# python3 lists the exhibits kept with their content, as NUL-separated
# "S, SHA-256, path" records, and one "F" record per authenticated binary that
# was not copied (--collect --binary): no file to verify. NUL cannot occur in
# a path, whatever the manifest holds. A manifest that is no JSON, or whose
# Items is no list, ends with exit code 3: refused.
list_exhibits() {
    python3 - "$manifest" <<'PYTHON'
import json, sys
try:
    with open(sys.argv[1], encoding="utf-8-sig") as f:
        items = json.load(f).get("Items")
except (OSError, ValueError, AttributeError):
    sys.exit(3)
if not isinstance(items, list):
    sys.exit(3)
out = sys.stdout.buffer
for item in items:
    if not isinstance(item, dict) or item.get("Result") != "OK":
        continue
    if item.get("ContentStored") is False:
        out.write(b"F\0")
        continue
    sha = str(item.get("SHA256") or "").replace("\0", "")
    path = str(item.get("ExhibitPath") or "").replace("\0", "")
    out.write(b"S\0" + sha.encode() + b"\0" + path.encode("utf-8", "surrogateescape") + b
    "\0")
PYTHON
}

ok=0; ko=0; fingerprint_only=0
while IFS= read -r -d '' kind; do
    if [ "$kind" = F ]; then fingerprint_only=$((fingerprint_only + 1)); continue; fi
    IFS= read -r -d '' expected || break
    IFS= read -r -d '' exhibit || break
    if [ "$kind" = X ]; then ko=$((ko + 1)); say unreadable "$manifest"; continue; fi
    # The manifest writes Windows paths, relative to the output folder.
    relative="${exhibit//\\//}"
    file="$(realpath -e -- "$root/$relative" 2>/dev/null)" || file=""
    case "$relative" in
        ""|/*) file="" ;;
    esac
    case "$file" in
```

```
        "$root"/*) ;;
        *) ko=$((ko + 1)); say mismatch "$exhibit"; continue ;;
    esac
    hash="$(sha256sum -- "$file" | head -c 64 | tr 'a-f' 'A-F')"
    if [ -n "$expected" ] && [ "$hash" = "$(printf '%s' "$expected" | tr 'a-f' 'A-F')" ];
    then
        ok=$((ok + 1))
    else
        ko=$((ko + 1)); say mismatch "$exhibit"
    fi
done < <(list_exhibits || printf 'X\0\0\0')

say total "$ok" "$ko"
say binaries "$fingerprint_only"
[ "$ko" -eq 0 ] || exit 1
```

```
# verify-exhibits.ps1 — checks the seal and every exhibit of a WAC collection.
# Usage: powershell -ExecutionPolicy Bypass -File verify-exhibits.ps1 -Output E:\output [-
    Seal <SHA-256>] [-Language fr]
#   -Seal      the seal WAC printed at the collection, as written down in the
#              report: compared with MANIFEST.sha256 (see below)
#   -Language  the messages' language; verifier-consigne.ps1 is this script in French
# Exit code: 0 if everything matches, 1 otherwise, 2 if PowerShell is older than 4.
#
# The seal beside the manifest proves only that the manifest is unchanged
# since it was sealed: whoever rewrites the manifest can rewrite its seal.
# Against that, the value written down at the collection (-Seal).

param([string]$Output = '.\output', [string]$Seal = '', [string]$Language = 'en')

# The messages, in the language asked for: {0}, {1} take the values.
$messages = @{
    en = @{
        Version    = 'PowerShell 4 or later is needed (this is {0}): use verify-exhibits.
    sh, or a more recent Windows'
        Absent     = 'NO MANIFEST: {0} or {1} absent'
        Seal       = 'SEAL MISMATCH: the manifest was modified'
        Noted      = 'SEAL MISMATCH: {0} instead of {1}, written down at the collection'
        Sealed     = 'Seal verified ({0})'
        Unnoted    = 'Seal checked against MANIFEST.sha256 only: give -Seal <value written
     down at the collection> to hold it against a rewrite'
        Unreadable = 'UNREADABLE MANIFEST: {0}'
        Mismatch   = 'MISMATCH: {0}'
        NotRead    = 'UNREADABLE EXHIBIT: {0}'
        Total      = '{0} exhibit(s) verified, {1} mismatch(es)'
        Binaries   = '{0} binary(ies) authenticated and fingerprinted only (no copy to
    verify)'
    }
    fr = @{
        Version    = 'PowerShell 4 ou plus est nécessaire (version {0}) : utiliser
    verifier-consigne.sh, ou un Windows plus récent'
        Absent     = 'MANIFESTE ABSENT : {0} ou {1}'
        Seal       = 'SCEAU NON CONFORME : manifeste modifié'
```

```
        Noted      = 'SCEAU NON CONFORME : {0} au lieu de {1}, relevé à la collecte'
        Sealed     = 'Sceau conforme ({0})'
        Unnoted    = 'Sceau comparé à MANIFEST.sha256 seul : donner -Seal <valeur relevée
    à la collecte> pour l''opposer à une réécriture'
        Unreadable = 'MANIFESTE ILLISIBLE : {0}'
        Mismatch   = 'DIFFÉRENTE : {0}'
        NotRead    = 'PIÈCE ILLISIBLE : {0}'
        Total      = '{0} pièce(s) conforme(s), {1} différente(s)'
        Binaries   = '{0} binaire(s) authentifié(s), empreinte seule (pas de copie à vé
    rifier)'
    }
}
$text = if ($messages.ContainsKey($Language)) { $messages[$Language] } else { $messages['
    en'] }
function Say([string]$key) { Write-Output ($text[$key] -f $args) }

# Get-FileHash, which reads the seal, came with PowerShell 4:
# on an older one the reading would fail, and its failure read as a mismatch
# of the seal. Said so instead.
if ($PSVersionTable.PSVersion.Major -lt 4) { Say Version $PSVersionTable.PSVersion; exit 2
     }

# Both files first: a missing one is no tampering, and must not read as one.
$manifest = "$Output\exhibits\MANIFEST.json"
$sealFile = "$Output\exhibits\MANIFEST.sha256"
if (-not (Test-Path -LiteralPath $manifest -PathType Leaf) -or -not (Test-Path -
    LiteralPath $sealFile -PathType Leaf)) {
    Say Absent $manifest $sealFile; exit 1
}

# 1. The seal: the manifest's fingerprint must be the one recorded beside it,
# and the one written down at the collection when it is given.
$recorded = ((Get-Content -Raw -LiteralPath $sealFile) -split '\s+')[0]
$actual   = (Get-FileHash -Algorithm SHA256 -LiteralPath $manifest).Hash
if ($recorded -ne $actual) { Say Seal; exit 1 }
if ($Seal -and $Seal -ne $actual) { Say Noted $actual $Seal; exit 1 }
Say Sealed $actual
if (-not $Seal) { Say Unnoted }

# 2. Every exhibit: its SHA-256 must be the one in the manifest. A manifest
# that is no JSON, or without its list of exhibits, is refused: never taken
# for an empty collection. Read by the serializer ConvertFrom-Json is built
# on, its limit lifted as PowerShell 5 does: that of PowerShell 4, 2 MB of
# text, refused every collection's manifest (tens of MB) on Windows 8.1.
try {
    Add-Type -AssemblyName System.Web.Extensions
    $serializer = New-Object Web.Script.Serialization.JavaScriptSerializer
    $serializer.MaxJsonLength = [int]::MaxValue
    $m = $serializer.DeserializeObject((Get-Content -Raw -Encoding UTF8 -LiteralPath
    $manifest))
} catch { $m = $null }
if ($m -isnot [Collections.IDictionary] -or $m['Items'] -isnot [Collections.IList]) { Say
    Unreadable $manifest; Say Total 0 1; exit 1 }
```

```
# The manifest comes from the examined machine: an exhibit path that leaves
# the collection (.., absolute path, drive) is refused, never read. Checked on
# the string.
#
# Each exhibit is opened by CreateFileW and read through its handle: a path
# past 259 characters needs \\?\, which .NET refuses before 4.6.2 (Windows
# 8.1 has 4.5) and CreateFileW takes on every Windows. Below that, an
# ordinary path, normalised as WAC's own writing was.
Add-Type -TypeDefinition @'
using System; using System.Runtime.InteropServices; using Microsoft.Win32.SafeHandles;
public static class ExhibitFile {
    [DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
    public static extern SafeFileHandle CreateFileW(string name, uint access, uint share,
    IntPtr security,
                                                    uint disposition, uint flags, IntPtr
    template);
}
'@
$GENERIC_READ = [uint32]2147483648   # 0x80000000: as a literal, a negative Int32 before
    PowerShell 7
$FILE_SHARE_READ = 1; $OPEN_EXISTING = 3; $FILE_FLAG_SEQUENTIAL_SCAN = 0x08000000
$sha256 = [Security.Cryptography.SHA256]::Create()
# The SHA-256 of a file, in upper-case hexadecimal; throws if it cannot be read.
function Get-ExhibitHash([string]$path) {
    $handle = [ExhibitFile]::CreateFileW($path, $GENERIC_READ, $FILE_SHARE_READ, [IntPtr
    ]::Zero,
                                         $OPEN_EXISTING, $FILE_FLAG_SEQUENTIAL_SCAN, [
    IntPtr]::Zero)
    if ($handle.IsInvalid) { throw "CreateFileW: $([Runtime.InteropServices.Marshal]::
    GetLastWin32Error())" }
    $stream = New-Object IO.FileStream($handle, [IO.FileAccess]::Read)
    try { [BitConverter]::ToString($sha256.ComputeHash($stream)) -replace '-', '' }
    finally { $stream.Dispose() }
}
$root = (Resolve-Path -LiteralPath $Output).ProviderPath.TrimEnd('\') + '\'
$ok = 0; $ko = 0; $fingerprintOnly = 0
foreach ($p in ($m.Items | Where-Object { $_.Result -eq 'OK' })) {
    # An authenticated binary that was not copied (--collect --binary): no file.
    if ($p.ContentStored -eq $false) { $fingerprintOnly++; continue }
    $rel = ([string]$p.ExhibitPath) -replace '/', '\'
    if (-not $rel -or [IO.Path]::IsPathRooted($rel) -or $rel -match ':' -or ($rel -split
    '\\') -contains '..') {
        $ko++; Say Mismatch $p.ExhibitPath; continue
    }
    $path = $root + $rel
    if ($path.Length -gt 259) { $path = if ($root.StartsWith('\\')) { '\\?\UNC\' + $path.
    Substring(2) } else { '\\?\' + $path } }
    try { $h = Get-ExhibitHash $path }
    catch { $ko++; Say NotRead $p.ExhibitPath; continue }   # unread is no mismatch: said
    as such
    if ($h -eq $p.SHA256) { $ok++ } else { $ko++; Say Mismatch $p.ExhibitPath }
}
Say Total $ok $ko
```

```
Say Binaries $fingerprintOnly
if ($ko) { exit 1 }
```

6.4 Le journal d'investigation

`investigation.json` retrace la collecte : outil et ligne de commande, machine et fuseaux horaires, opérateur, début, fin et durée, puis la liste **ordonnée** des opérations. Pour chacune :

`Seq`, `TimestampUtc`, `TimestampLocal`
: rang et instant ;

`Operation`, `Target`
: ce qui a été fait, et sur quoi ;

`Result`
: l'issue ;

`Footprint`
: la trace que cette opération est censée laisser sur la machine examinée.

Ce journal permet de confronter, collecte par collecte, les traces attendues à celles effectivement observées. Il signale aussi un écart entre le fuseau horaire du suspect et celui de la machine de collecte.

6.5 Les fichiers d'artefacts

Un fichier par artefact. Les champs décrits sont ceux émis ; un champ sans valeur n'est pas émis (voir chapitre 7).

6.5.1 Système et comptes

`OperatingSystem.json`
: nom et domaine de la machine, édition et version de Windows, date d'installation, heure courante, **heure de démarrage**, durée d'activité, fuseau horaire. `BootTimeSource` dit d'où vient l'heure de démarrage (noyau, ou estimation de repli) ; `ClockAdjustedSinceBootMs` donne le cumul des corrections d'horloge depuis le démarrage — quelques secondes sont un recalage ordinaire, un grand écart signale une horloge modifiée. `ShutdownTime` : le dernier arrêt ordonné. `NtfsLastAccessUpdated` : faux, les dates d'accès NTFS ne sont pas mises à jour et ne disent rien. `Settings` : les réglages qui conditionnent la lecture des autres artefacts, tels que stockés — dates d'accès et noms courts NTFS, Prefetch actif, fichier d'échange et vidages mémoire, jeux de contrôle, bureau à distance, chronologie Windows, dernier utilisateur connecté.

`security.json`
: ruche `SECURITY` : **noms** des secrets LSA et dates de leurs valeurs (`DefaultPassword` signale

un mot de passe de connexion automatique enregistré, `_SC_<service>` un service qui tourne sous un compte), nombre d'emplacements de connexions de domaine en cache et combien sont occupés, politique d'audit (`PolAdtEv`, en octets : son format n'est pas publié), et les droits des comptes (`AccountRights`, lus dans `Policy\Accounts`) : pour chaque compte ou groupe, ses privilèges (`Privileges`, p. ex. `SeDebugPrivilege`, `SeBackupPrivilege`) et ses droits de connexion (`LogonRights`). Un privilège sensible donné à un compte inattendu est une persistance. **Aucune valeur de secret** n'est lue, ni aucune donnée d'authentification en cache. Les copies `config\RegBack` des ruches sont prélevées telles quelles.

`users.json`
: comptes locaux, lus dans la ruche `SAM` : SID et RID, état, drapeaux, nombre de connexions et d'échecs, dates de dernière connexion, de dernier échec, de changement de mot de passe et de modification du compte, profil, et les condensats NT et LM du mot de passe (`NtHash`, `LmHash`, en hexadécimal), avec leur historique (`NtHashHistory`, `LmHashHistory`). Un condensat est à sens unique : il désigne un mot de passe sans le révéler. Un mot de passe vide ou faible utilisé par un intrus se lit comme une connexion légitime ; **WAC ne juge rien**, il convertit, et c'est WAE qui confronte les condensats, hors ligne, à ceux des mots de passe faibles connus. Les condensats sont déchiffrés sur place (clé de démarrage de `SYSTEM`, clé de hachage de la `SAM`, schéma AES de Windows 10 et suivants ou RC4 plus ancien, dont la somme de contrôle est vérifiée : une clé fausse ne donne aucun condensat plutôt que des condensats faux, puis couche DES par RID, déchiffrée par WAC lui-même pour valoir de XP à Windows 11). Si le compte porte des questions de sécurité de réinitialisation (`Security Questions`, valeur `ResetData`), elles sont converties telles quelles, questions ET réponses : stockées en clair, une réponse triviale est une faiblesse que WAE juge comme un mot de passe faible.

`local_groups.json`
: groupes locaux lus dans la `SAM` : groupes intégrés (`Scope Builtin`, `S-1-5-32-…`) et groupes créés sur la machine (`Local`), chacun avec son RID, son SID, son nom, sa description et ses membres directs (SID et nom résolu hors ligne). C'est l'appartenance qui dit qui est administrateur ou membre des Bureaux à distance ; un compte ajouté à un groupe privilégié par un intrus y figure.

`Sessions.json`
: sessions ouvertes au moment de la collecte : utilisateur, type d'ouverture, paquet d'authentification, heure de début.

`processes.json`
: processus en cours : nom, identifiants (et parent), propriétaire, session, modules chargés, et, avec `--binary`, empreintes de l'exécutable.

`network.json`
: état réseau au moment de la collecte : `Connections` (TCP et UDP, IPv4 et IPv6 : adresses, ports, état, processus propriétaire et son nom, heure de création locale et UTC), `Neighbours` (caches ARP et NDP : les machines du réseau local contactées récemment), `Interfaces` (adresses, passerelles, serveurs DNS, DHCP) et `Routes`. `TablesNotRead` nomme une table que Windows a refusé de donner, ce qui n'est pas une table vide. `TablesIpv4Only` nomme les tables lues par les fonctions de Windows XP et Server 2003, qui ne connaissent qu'IPv4 : l'absence de ligne IPv6 n'y signifie pas l'absence de connexion IPv6.

Sur ces versions, une connexion n'a pas d'heure de création, et un voisin porte un `Type` (entrée ARP dynamique ou statique) à la place d'un `State`.

`services.json`
services et pilotes, configuration lue dans la ruche : type, démarrage, compte, binaire et DLL hébergée, commande en cas d'échec, dépendances, dernière écriture de la clé ; état courant ; avec `--binary`, empreintes du binaire et de la DLL.

`ScheduledTasks.json`
tâches planifiées, lues dans leurs définitions XML : auteur, compte d'exécution, état, dernière exécution et son résultat, déclencheurs et actions (avec `--binary`, empreintes de la commande). Sous Windows XP et Server 2003, d'après leurs fichiers `.job` (`SourceJob`) : programme, paramètres, dossier de travail, auteur, commentaire, dernière exécution (heure locale convertie), code de sortie, déclencheurs, et état du planificateur (`JobStatus` : `HasNotRun`, `Ready`…) ; le compte d'exécution n'est pas dans le fichier.

6.5.2 Exécution de programmes

`prefetchs.json`
fichiers Prefetch : programme, nombre et dates d'exécution, volumes, et liste des fichiers chargés avec leur référence `$MFT` (et leurs empreintes avec `--binary`). Une référence absente n'est pas une lecture ratée : Windows l'écrit à zéro, pour tous les fichiers d'un programme lancé une seule fois et pour une partie des autres. Versions décodées : 17 (Windows XP, qui n'enregistre aucune référence `$MFT` et une seule date d'exécution), 23 (Vista et 7, une seule date), 26 (Windows 8.1), 30 et 31 (Windows 10 et 11, compressées) ; les autres sont déclarées « format non pris en charge » (section 7.10).

`amcache_application_files.json`, `amcache_applications.json`, `amcache_inventory.json`
inventaire Amcache, table par table et valeur par valeur, avec la date de dernière écriture de chaque entrée : les exécutables (chemin, éditeur, version, taille, date d'édition de liens, `AmcacheSha1`), les applications installées, et dans le troisième fichier les pilotes (avec leur `AmcacheSha1`), périphériques, conteneurs, paquets de pilotes et raccourcis, chacun avec sa `Table`. **Attention** : pour un fichier de plus de 30 Mio, Amcache ne hache que les 31 457 280 premiers octets ; `AmcacheSha1CoversBytes` le signale, et ce SHA-1 ne se compare pas à celui du fichier entier. Sous Windows 8, 8.1 et les premiers Windows 10, l'Amcache est faite d'autres tables, `File` et `Programs`, dont les valeurs sont numérotées et non nommées : WAC ne nomme que celles dont le sens a été confirmé — pour un fichier, `LongPath`, `AmcacheSha1`, `ProgramId`, sa référence `$MFT` (`MftReference`, lue dans le nom de sa clé) et le GUID de son volume ; pour un programme, `Name`, `Version`, `Publisher`, `UninstallKeys` et `Files` —, et rend les autres telles que lues (`UnnamedValues` : nom, type, valeur). Sous Windows 7, les exécutables lancés depuis le dernier passage de sa tâche d'inventaire, que liste `RecentFileCache.bcf`, sont donnés dans `amcache_application_files.json` avec `Table` `RecentFileCache` : leur chemin seulement, le fichier ne porte aucune date.

`shimcache.json`
cache de compatibilité : chemin et date de modification du fichier, position dans le cache (`CachePosition`, 0 pour la plus récente). Une entrée prouve que le binaire était présent, pas qu'il a été exécuté. L'indicateur d'exécution que d'autres outils publient n'est *pas* relevé :

mesuré contre le Prefetch, son « oui » est presque toujours juste, mais son « non » est faux pour 9 binaires exécutés sur 10 — il ferait conclure qu'un programme n'a jamais tourné. Formats décodés : Windows XP, Vista, 7, 8, 8.1, 10 et 11 ; sous Windows 8.1, le chemin commence par `SYSVOL\`, le volume système, et il est publié tel quel. Windows XP y note aussi la taille du fichier (`FileSize`) et le moment où il a mis l'entrée à jour (`LastUpdateUtc`). Server 2003 écrit la même signature que Vista (`0xBADC0FFE`), la taille du fichier (`FileSize`) à la place de ses indicateurs : la valeur ne les distingue pas, la version examinée les départage, et sans version lue la taille n'est pas donnée (mesuré sur Server 2003 Enterprise x86, build 3790).

`bams.json`
: dernière exécution par programme et par utilisateur (Background Activity Moderator). Les ruches étant lues sur le disque, une exécution de la ou des dernières minutes peut manquer : Windows garde une modification du registre en mémoire avant de l'écrire (de 56 à 82 s mesurées sur la machine de test, que `RegFlushKey` n'a pas avancées), et `WAC` ne force pas cette écriture, qui modifierait le disque examiné.

`userassists.json`
: programmes lancés depuis l'interface : compteur, nombre et durée de premier plan (`FocusTimeMs`), dernière exécution. Sous Windows XP, la valeur ne garde pas le premier plan, et son compteur part de 5 (mesuré : deux lancements donnent 7) : `Count` est le compteur moins 5.

`muicache.json`
: noms d'affichage des programmes exécutés (sous Windows XP, dans `ShellNoRoam\MUICache` de NTUSER.DAT).

`run.json`
: programmes lancés au démarrage ou à l'ouverture de session (clés Run et RunOnce), avec la dernière écriture de la clé : dans SOFTWARE (toute la machine), dans la ruche DEFAULT (le compte système) et dans le NTUSER.DAT de chaque profil (`Hive`). `Type` `REG_EXPAND_SZ` : la commande contient des variables (`%SystemRoot%`…), données telles qu'écrites. Aussi : les clés Run des programmes 32 bits (`SOFTWARE (32-bit)`) ; `StartupApproved` (`Key` `StartupApproved\Run`, `Run32`, `StartupFolder`), l'état d'une entrée tel que l'utilisateur l'a réglé dans le Gestionnaire des tâches ou les Paramètres : `Status`, les 4 premiers octets tels que stockés (leur sens n'est pas mesuré, donc pas interprété), et `Changed`, l'instant du changement ; `RunNotification`, une entrée de démarrage que Windows a remarquée (`Notification`). Un nom sans valeur Run correspondante est une entrée retirée depuis.

`persistence.json`
: persistance du registre au-delà des clés Run : IFEO (`Debugger`, `GlobalFlag`), `SilentProcessExit`, AppInit_DLLs et AppCertDlls, Winlogon (`Shell`, `Userinit`, `Taskman`, `Notify`), paquets LSA, moniteurs d'impression, `BootExecute`, Active Setup, assistants NetSh, commandes d'ouverture des types exécutables (`exefile`, `batfile`, `cmdfile`…, machine et utilisateur), et détournements COM des classes de chaque utilisateur (UsrClass.dat). Pour chaque valeur : mécanisme (`Mechanism`), ruche, clé, nom, type, valeur telle qu'écrite, dernière écriture de la clé, et avec `--binary` les empreintes du fichier visé. Toutes les valeurs sont données, celles de Windows comprises : distinguer l'ajouté du défaut revient à l'analyse. Les tâches BITS ne sont pas encore lues.

`office_trust_records.json`
documents Office qu'un utilisateur a approuvés (« Activer le contenu ») : application, version, document, date ; `MacrosEnabled` quand il a activé les macros — la porte de la plupart des infections par macro.

`typed_paths.json`, `word_wheel_query.json`
chemins tapés dans la barre d'adresse de l'Explorateur et termes qui y ont été cherchés, par utilisateur et par position (`MruPosition`, 0 = le plus récent, comme dans `shellbags.json`).

`root_certificates.json`
certificats des magasins racines de la machine et de chaque utilisateur : sujet, émetteur, validité ; une racine ajoutée fait accepter tout ce qu'elle signe (interception TLS, logiciel signé). `ThumbprintMismatch` signale un certificat dont le SHA-1 n'est pas le nom de sa clé.

`internet_settings.json`
proxy et script de configuration automatique de chaque utilisateur.

`intune_logs.json`
journaux de l'extension de gestion Intune (`ProgramData\Microsoft\IntuneManagement Extension\Logs`), sur une machine gérée par Microsoft Intune : les scripts PowerShell et les applications que le locataire a poussés et exécutés en SYSTEM, leur résultat, l'inscription de l'appareil. Une fiche par message (format CMTrace, sur une ou plusieurs lignes) : `Message`, `Component`, `Type`, `Thread`, la date, écrite en heure locale, en UTC et en heure locale — exacte quand le journal écrit son décalage (`BiasMinutes`).

`cloud_identity.json`
inscription de la machine à Entra ID (`Control\CloudDomainJoin` de SYSTEM : `TenantId`, `User Email`, `IdpDomain`, l'empreinte du certificat de l'appareil en `KeyName`, et les informations du locataire, URL d'inscription MDM comprises) et comptes professionnels ajoutés par chaque utilisateur (`WorkplaceJoin` de son NTUSER.DAT) ; toutes les valeurs telles que stockées, `KeyLastWrite` datant l'inscription. Les caches de jetons des comptes (`Token Broker`, `Microsoft.AAD.BrokerPlugin`) sont prélevés bruts, chiffrés par DPAPI comme Windows les garde, et jamais déchiffrés par WAC. Prélevés bruts aussi, sans conversion : les fichiers de trace ETW que Windows écrit sur son disque (`--etw_logs`), les journaux et réglages de OneDrive (`--onedrive`, sauf ceux de la synchronisation des listes SharePoint) et le dossier local de Defender for Endpoint (`--defender_endpoint`, sans les binaires du capteur).

`firewall.json`
règles du pare-feu (nom, action, sens, ports, programme ; tous les champs dans `Fields`) et réglages des profils domaine, privé et public. Sous Windows XP SP2 et suivants et Server 2003 SP1, les exceptions de chaque profil — programmes autorisés et ports ouverts — en règles de `Version` `XP`, avec leur `Profile` et leur `Scope` ; XP SP1 n'a pas de politique de pare-feu.

`firewall_log.json`
journal du pare-feu (`pfirewall.log` et sa rotation `.old`), là où le `LogFilePath` de chaque profil le place — réglages du pare-feu et stratégie de groupe — et à son emplacement par défaut : une fiche par connexion acceptée ou rejetée, avec `Action`, `Protocol`, `Source`

`Address`, `DestinationAddress`, les ports, `Size`, `Direction` et, depuis Windows 10, `Process Id` ; la date, écrite en heure locale, donnée en UTC et en heure locale. C'est la seule trace des flux réseau qu'un Windows garde sur son disque ; la journalisation est désactivée par défaut.

`installed_programs.json`
programmes installés, lus dans les clés `Uninstall` de la machine (64 et 32 bits) et de chaque utilisateur, et produits Windows Installer (`Scope MSI product`, avec le paquet mis en cache `LocalPackage`) : toutes leurs valeurs (nom, éditeur, version, dossier, commande de désinstallation, `InstallDate` — le jour seul, tel qu'écrit) et la date d'écriture de la clé. Un outil d'accès à distance installé, puis désinstallé, y laisse souvent sa trace.

`defender_configuration.json`
configuration de Microsoft Defender lue dans le registre : les **exclusions** (chemins, extensions, processus, adresses ; `Source Settings` ou `Policy`), qu'un intrus ajoute pour que ses outils ne soient pas analysés, et les valeurs qui règlent ou coupent la protection (temps réel, protection contre la falsification, envoi d'échantillons…).

`explorer_recent_docs.json`
fichiers ouverts par chaque utilisateur, tels que l'Explorateur les retient (`RecentDocs`) : par extension (`Extension`) et pour toutes extensions confondues, avec leur rang (`MruPosition`, 0 pour le plus récent), l'élément de shell du raccourci créé dans Recent, et la date d'écriture de la liste, qui est celle de l'ouverture du fichier en position 0. Le nom survit au fichier et au raccourci.

`mount_points.json`
volumes et partages réseau qu'a vus l'Explorateur de chaque utilisateur (`MountPoints2`) : volume (GUID, à rapprocher de `mounted_device.json` pour une clé USB) ou partage (`Share`, `\\serveur\partage`), date d'écriture de l'entrée — la dernière fois que l'Explorateur l'a vue —, ses valeurs, et ses sous-clés avec leur commande (`Shell\AutoRun\command` : une persistance ancienne).

`known_networks.json`
réseaux auxquels la machine s'est connectée (`NetworkList`) : nom, type, catégorie, adresse matérielle de la passerelle et suffixe DNS de chaque signature, première et dernière connexion. Ces deux dates sont écrites par Windows en heure locale : WAC les donne en UTC avec le décalage en vigueur à leur date (mesuré contre le journal `NetworkProfile`). Elles disent où la machine a été.

`program_usage.json`
ce que chaque utilisateur a lancé ou utilisé, source par source (`Source`) : `Feature Usage` (lancements, bascules, clics sur la barre des tâches, par application, `Count`), `JumplistData` (dernière utilisation, `LastUsed`), le magasin de l'Assistant de compatibilité des programmes (programmes exécutés), les modes de compatibilité (`Layers`), les programmes diagnostiqués par RADAR, les éléments épinglés à la barre des tâches (`Taskband`), chacun décodé comme un raccourci complet, et les programmes choisis pour ouvrir chaque type de fichier (`FileExts` : `OpenWithList` avec `Extension` et `MruPosition`, 0 pour le plus récent ; `UserChoice`, le programme par défaut). `OpenWithProgids` n'est pas relevé : les installeurs le remplissent, l'utilisateur n'y est pour rien.

`devices.json`
: périphériques que la machine a connus (`Source`) : USB et SCSI (`Enum`), avec leurs valeurs et les dates d'installation, de première installation, de dernière arrivée et de dernier retrait (`Install`, `FirstInstall`, `LastArrival`, `LastRemoval`, en UTC) ; périphériques portables (téléphone, appareil photo) ; noms de volume que la recherche Windows a retenus par lettre de lecteur (`VolumeInfoCache`) ; interfaces de disque, de volume et de périphérique portable (`DeviceClasses`), écrites à leur première apparition.

`network_shares.json`
: dossiers que la machine partage (`LanmanServer\Shares`) : nom, chemin, réglages tels que stockés, et descripteur de sécurité décodé — propriétaire, et pour chaque entrée de la liste de contrôle d'accès son type, son masque, le SID et son nom. Les partages administratifs (`C$`, `ADMIN$`) n'y figurent pas : Windows ne les stocke pas.

`network_configuration.json`
: configuration réseau lue dans les ruches : nom d'hôte et domaine (`Global`), et pour chaque carte sa description et son pilote, une adresse matérielle forcée (`NetworkAddress`), son adresse matérielle courante et d'origine, sa dernière configuration IPv4 et IPv6, et son bail DHCP (obtention et fin, en UTC). Elle reste lisible quand la machine est débranchée ou le bail expiré, ce que le relevé vivant ne montre plus.

`event_log_configuration.json`
: configuration de chaque journal d'événements lue dans les ruches : les canaux (`WINEVT\Channels` : `Enabled`, `Type`, `OwningPublisher`, droits d'accès `ChannelAccess`, taille et rétention si elles sont fixées) et les journaux classiques (`Services\EventLog` : fichier, `MaxSize`, `Retention`, accès) et les sessions ETW lancées au démarrage (`WMI\Autologger` : `Start`, fichier, taille). Un intrus qui coupe un canal, réduit un journal, en ouvre l'accès ou arrête une session comme `EventLog-Security` y laisse sa trace ; `KeyLastWrite` date la dernière modification de chacun.

`boot_records.json`
: ce qui s'exécute avant Windows, lu en brut sur chaque disque physique (le disque du support de collecte excepté, s'il ne porte pas le système) : son premier mébioctet copié — code d'amorçage du MBR, table de partitions, en-tête et entrées GPT, et l'espace avant la première partition, où se cache le code d'un bootkit —, les 16 premiers secteurs de chaque partition, et chaque fichier de la partition système EFI, lue par le lecteur FAT de WAC (chargeurs, BCD). Une fiche par disque : `PartitionStyle` (`MBR`, `GPT`, ou `Volume` pour un disque formaté en entier, sans table, comme une clé USB : son premier secteur est celui du volume, `BootSectorOemId`), `DiskId`, `MbrBootCodeSha256`, et chaque partition en secteurs comme la table la stocke (`FirstSector`, `Sectors`), avec son type, le SHA-256 de ses premiers secteurs, l'OEM de son secteur de démarrage et, pour la partition EFI, ses fichiers (`Efi Files` : chemin, taille, SHA-256). Les partitions logiques d'une partition étendue ne sont pas suivies.

`fat_volumes.json`
: volumes FAT12, FAT16, FAT32 et exFAT, clés USB et cartes du suspect comprises (le volume de la collecte excepté) : chaque entrée, présente ou **supprimée** (`Deleted`), son chemin, son nom 8.3, sa taille, ses attributs et ses dates. Un nom supprimé en FAT garde son nom long s'il n'a pas été écrasé ; sinon son nom 8.3, première lettre perdue (`NameIncomplete`). Chaque date est donnée en UTC et en heure locale dérivée de l'UTC, comme dans tous

les artefacts : un même instant se lit partout à la même heure. FAT stocke l'heure locale calculée par Windows au moment où il l'écrit : une date de modification antérieure à la création est une date conservée par une copie, écrite au moment de la copie, et convertie avec le décalage de ce moment (`ModifiedWrittenAt Copy`, `ModifiedOffsetApplied`, `Modified AsStored`) — sans quoi un fichier d'hiver copié en été serait décalé d'une heure. exFAT garde son décalage (`ModifiedOffset`…), la date UTC est exacte. La collecte ne copie que les métadonnées du volume (secteur de démarrage, FAT, répertoires), pas le contenu des fichiers ; avec `--binary`, un programme de ces volumes cité par un artefact (lancé depuis une clé, par exemple) est empreinté et prélevé comme sur NTFS.

`shadow_copies.json`
clichés instantanés (« versions précédentes », points de restauration) de chaque volume NTFS fixe, **lus en brut** — catalogue et magasins de la zone de différences, sans le service VSS ni son interface : WAC ne crée, ne monte et n'expose aucun cliché. Un objet par cliché, du plus ancien au plus récent : `Volume`, `ShadowCopyId` (l'identifiant que montre `vssadmin`), `ShadowCopySetId`, `StoreId`, `CreationTimeUtc`, `Context`, `Attributes`, `OperatingMachine`, `Stored Blocks` (blocs de 16 Kio dont le magasin garde l'état d'avant) et `ExhibitFolder`, le dossier de la consigne où sont rangés ses fichiers (`vss\<instant>_<identifiant>`). Pour chaque cliché, la collecte relit à travers lui les ruches (système, Amcache, profils de `\Users`) et les journaux `.evtx` ; un fichier identique (SHA-256) à sa version plus récente — sur le volume tel qu'il est, ou dans le cliché suivant — n'est pas recopié : le manifeste l'inscrit comme partagé avec elle. Chaque pièce lue dans un cliché porte `ShadowCopyId` au manifeste. Une ruche effacée ou modifiée par un intrus, un journal vidé, s'y retrouvent dans leur état d'avant. Windows continue d'écrire pendant la lecture : chaque fichier est lu deux fois, les listes de blocs relues avant chaque lecture, et refait si les deux lectures diffèrent. Windows XP ne garde pas de cliché. Un cliché tient ce qui était *sur le disque* à son instant, et Windows écrit le registre et les journaux d'événements en retard : une valeur ou un événement écrits moins d'une minute environ avant le cliché peuvent ne pas y être — Windows lui-même ne les y lit pas (mesuré : absents 15 s après, présents 90 s après).

`shadow_copy_events.json`
les événements que les journaux d'un cliché gardent et que les journaux actuels n'ont plus (journal vidé, enregistrements écrasés par la rotation) : même forme que `events.json`, plus `ShadowCopyId` et `ShadowCopyCreationTimeUtc`, le cliché le plus récent qui le garde. Chaque événement une fois, quel que soit le nombre de clichés.

`shadow_copy_registry.json`
les entrées des artefacts de registre (clés Run, services, comptes et groupes, ShimCache, Amcache, BAM, UserAssist, MUICache, ShellBags, MRU, périphériques USB, programmes installés, réseaux connus…) convertis depuis les ruches d'un cliché, que la conversion du volume ne donne pas : une valeur Run, un service, un compte supprimés depuis, la trace d'un outil effacé. `Artefact` (le fichier JSON de l'artefact), `ShadowCopyId`, `ShadowCopyCreation TimeUtc` et `Entry`, l'entrée telle que l'artefact la donne. Chaque entrée une fois. Une entrée modifiée depuis y figure aussi, dans son état d'alors. Le cliché entier n'est pas converti : il répéterait, presque identiques, les milliers d'entrées du volume.

`srum.json`
usage des ressources (SRUM, `sru\SRUDB.dat`), une entrée par ligne de chaque table : octets envoyés et reçus par application et par interface (une exfiltration s'y lit, avec le

programme), cycles processeur, octets lus et écrits, connexions ; 30 à 60 jours. `Table` (GUID du fournisseur), `Provider` (nom documenté : `NetworkUsage`, `ApplicationResourceUsage`…), puis les colonnes par leur nom (lettres et chiffres seuls), et `App`, `UserSID`, `UserName` résolus par `SruDbIdMapTable`. La base est copiée vivante : la dernière heure est dans ses journaux `SRU*.log`, copiés dans la consigne pour un rejeu (`esentutl /r`) sur une copie.

`bits_jobs.json`

travaux BITS (`qmgr.db`) : nom, description, type, état, propriétaire, **commande de notification** (lancée à la fin ou à l'échec du transfert : une persistance), délais, dates, et chaque fichier : URL source, destination, fichier temporaire, tailles. Le blob complet est donné en hexadécimal : BITS 10 y garde plus (le programme créateur) à des endroits non décrits. Avant Windows 10, les travaux sont dans les fichiers de file d'attente `qmgr0.dat` et `qmgr1.dat`, que BITS écrit tour à tour : chaque travail dit dans lequel il a été lu (`QueueFile`), car le plus ancien peut garder un état antérieur du même travail. Sous Vista (BITS 3.0), le séparateur des travaux n'est connu d'aucun outil publié ; il est mesuré et lu de même.

`wmi_persistence.json`

abonnements aux événements WMI, lus **hors ligne** dans le dépôt CIM copié en brut (`wbem\Repository`) : aucune requête WMI, rien exécuté sur la machine. Dans *chaque* espace de noms, chaque filtre (requête WQL), consommateur (ligne de commande, script VBScript ou JScript, journal, courriel, et toute classe qu'un intrus aurait définie), liaison et minuterie, avec toutes ses propriétés (`Properties` : nom et valeur ; `CreatorSID` en clair). C'est une persistance invisible au registre. Les deux dates d'en-tête d'une instance ne sont pas émises : ce ne sont pas des dates de création. Sous Windows XP, le dépôt est dans `wbem\Repository\FS` et suit une autre disposition (condensats MD5, entrées de page de 4 octets, deux fichiers de correspondance au SP1) : il est lu de la même façon.

`web_client.json`

navigation de chaque utilisateur dans les navigateurs Chromium (Edge, Chrome, Brave, Vivaldi, Chromium, Opera), lue dans la base `History` de chaque profil de navigateur. Un enregistrement par URL (`Type Url` : titre, nombre de visites et de saisies, dernière visite), par visite (`Visit` : instant, `Transition` — `typed` saisie, `link`, `reload`… — et ses qualificatifs, visite d'origine `FromUrl`, durée), par téléchargement (`Download` : chaîne complète des redirections `UrlChain`, page et onglet d'origine, chemin de destination, octets reçus, état, ouverture, `DangerType` brut, SHA-256 quand le navigateur l'a calculé) et par terme cherché dans la barre d'adresse (`SearchTerm`). Chaque enregistrement porte `Browser`, `Browser Profile`, `SID` et `Source`. Les dates de Chromium (microsecondes depuis 1601, UTC) sont converties exactement. Les bases sont lues sur une copie privée avec leur journal, rejoué comme le ferait le navigateur. À côté de `History`, la consigne reçoit brutes les autres bases du profil (`Login Data`, `Web Data`, cookies, raccourcis, signets, préférences) et `Local State`, dont la clé protégée par DPAPI chiffre les mots de passe enregistrés ; jamais les caches. Firefox (`Browser Firefox`, chaque dossier de `Profiles`) donne les mêmes enregistrements depuis `places.sqlite` — visites avec leur type (`typed`, `link`, `download`…), téléchargements avec leur destination, leur fin et leur taille —, plus `TypedInput` (ce qui a été tapé dans la barre d'adresse et la page atteinte) et, depuis `formhistory.sqlite`, `FormEntry` : chaque valeur saisie dans un formulaire ou la barre de recherche (`FieldName searchbar-history`), avec ses dates. Sont copiés bruts à côté : cookies, `logins.json` et `key4.db` (identifiants enregistrés, chiffrés), extensions, préférences, session. Enfin `TypedUrl` : chaque adresse

tapée dans Internet Explorer ou la barre d'adresse de l'Explorateur (`TypedURLs` du `NTUSER.DAT`, `MruPosition` 0 pour la plus récente) avec l'instant de sa saisie (`TypedURLsTime`, `TypedUtc`). `SavedLogin` : chaque identifiant enregistré (`Login Data` de Chromium, `logins.json` de Firefox) — site, compte (en clair chez Chromium, chiffré et donc absent chez Firefox), création, dernier usage, nombre d'usages, refus d'enregistrer (`NeverSave`) ; `Cookie` : site, nom, chemin, dates, drapeaux. Les mots de passe et les valeurs des cookies, chiffrés, ne sont jamais décodés : les bases sont dans la consigne pour une analyse qui détiendrait la clé DPAPI de l'utilisateur. Ce qui était *ouvert* : `SessionEntry`, chaque page de chaque onglet des fichiers de session — `Sessions\Session_*` et `Tabs_*` de Chromium, `sessionstore.jsonlz4` et ses sauvegardes chez Firefox —, avec `State` (`Open`, `Closed`, `ClosedTab`, `ClosedWindow`), l'instant où la page a été affichée (`NavigatedUtc`, Chromium, celui de sa visite dans `History`) ou le dernier accès à l'onglet (`TabLastAccessedUtc`, Firefox), l'instant de la fermeture (`ClosedUtc`) et la page affichée (`Shown`) : un onglet d'une fenêtre privée, ou d'un historique effacé, peut n'être plus que là ; `Favicon`, l'icône de chaque page visitée, conservée après l'effacement de l'historique ; `AddressBarPrediction` (Chromium), ce qui a été tapé dans la barre d'adresse et l'adresse alors choisie. Les sauvegardes de favoris de Firefox, le dossier local d'Internet Explorer et les conteneurs de WinINet (`INetCookies`, `IEDownloadHistory`) sont copiés bruts. Et `CacheEntry` (`Browser WebCache`) : chaque entrée de la base `WebCacheV01.dat` d'Internet Explorer, d'Edge Legacy et des applications WinINet, par conteneur (`Container` : `History`, `MSHist…`, `Content`, `iedownload`, `BackgroundTransferApi`) — URL, compte et page d'une entrée d'historique (`User`, `Page`), nombre d'accès, dates, fichier du cache (`CachedFile` : son chemin, par le sous-dossier que l'entrée désigne). Le moteur d'Internet Explorer (WinINet) reste utilisé par `mshta.exe` et des scripts même quand IE est redirigé vers Edge : le cache de chaque profil (`INetCache\IE`) est collecté, limité aux types utiles à l'enquête — HTA, scripts, exécutables, archives, documents à macros, PDF —, et l'HTA qu'une attaque a fait exécuter s'y retrouve avec son entrée. Un téléchargement d'Internet Explorer 10 et 11 (conteneur `iedownload`) donne en outre `Download` : son URL, son type MIME, sa copie temporaire dans le cache, le chemin où il a été enregistré (`TargetPath`, ou seulement `SuggestedName` s'il ne l'a pas été), sa taille et l'adresse du serveur — format non documenté par Microsoft, établi sur un vrai téléchargement d'IE 11 dans les machines de test Windows 11 et Windows 8.1, dont les dispositions diffèrent ; une autre disposition n'est pas devinée. Sur Windows 7 et antérieurs, Internet Explorer 5 à 9 tenait ces mêmes données dans des `index.dat` (cache, historique et ses dossiers `MSHist01`, cookies, téléchargements) : ils sont collectés et convertis en `Cache Entry` (`Browser Internet Explorer`), entrées récupérées (`Recovered`) et abandonnées (`RecordKind LEAK`) comprises ; l'heure de visite d'un historique hebdomadaire, locale, est convertie, l'expiration et la dernière vérification, dates FAT sans fuseau, sont données telles qu'écrites. La date de modification des conteneurs `MSHist`, que Windows écrit en heure locale, est convertie en UTC avec le décalage en vigueur à sa date. La base est copiée vivante avec ses journaux : ses dernières entrées peuvent n'être que dans ceux-ci (rejeu `esentutl /r` par l'analyste).

`web_server_sites.json`, `web_server_application_pools.json`, `web_server_modules.json`
le serveur web IIS (7 et suivants) ; sa configuration est lue dans `inetsrv\config\applicationHost.config`. Chaque site avec ses attributs tels qu'IIS les stocke (nom, `Id`…), ses liaisons (`Bindings` : protocole, adresse et port, nom d'hôte), ses applications et leurs répertoires virtuels : `PhysicalPath` tel qu'écrit, et `PhysicalPathResolved`, le même avec ses variables d'environnement développées ; ses réglages de journal et le dossier de ses

journaux (`LogFolder`); chaque pool d'applications avec son `ProcessModel` (l'identité sous laquelle tournent ses processus : un pool passé en `LocalSystem` exécute chaque page qu'il sert en tant que système); chaque module natif chargé dans chaque processus (`globalModules` : nom, `Image`, `PreCondition`) — un module ajouté là voit chaque requête, une porte dérobée connue. Un mot de passe de pool ou de répertoire virtuel reste tel qu'IIS l'a chiffré. La collecte prend la configuration d'abord, puis le dossier de chaque site en entier (le code, les données, et ce qu'un intrus y a déposé : un web shell), sauf un site posé à la racine d'un volume, l'historique de la configuration (`inetpub\history`, les états antérieurs qu'IIS garde lui-même) et les dossiers des journaux. IIS 5.1 et 6 (Windows XP, Server 2003) gardent leur configuration dans `MetaBase.xml` : elle est copiée brute, non convertie.

`web_server_logs.json`
un objet par requête des journaux d'IIS (`W3SVC<id>\u_ex<date>.log`, format W3C étendu) et de ceux de HTTP.sys (`System32\LogFiles\HTTPERR`, les requêtes refusées avant qu'un site les voie) : `Time` (UTC, comme IIS l'écrit), `Method`, `UriStem`, `UriQuery`, `ClientAddress`, `UserName`, `UserAgent`, `Status`, `TimeTakenMs`, `Reason` (HTTPERR)…, `File` et `Line`. Un champ qu'IIS écrit `-` n'est pas émis ; un champ que WAC ne nomme pas garde son propre nom. Un journal dans un autre format (IIS, NCSA, personnalisé) est copié brut, non converti.

`web_server_connection_strings.json`
chaque chaîne de connexion des `web.config` des sites, telle qu'écrite (`Name`, `Connection String`, `ProviderName`, `File`) : son mot de passe y est en clair quand le site le garde ainsi. `DatabaseFile` donne le fichier de base qu'elle désigne (`Data Source`, `AttachDbFilename`, `Dbq`, `Filename`; `|DataDirectory|` est le dossier `App_Data` du site), jamais pour un serveur, un partage ou un chemin relatif. Une section chiffrée (`aspnet_regiis -pe`) est dite par son fournisseur (`ProtectedBy`), non déchiffrée. Une base désignée hors des dossiers des sites est collectée avec son `-wal` et son `-journal`, jusqu'à `max_database_mib` ; au-delà, sa taille est consignée au journal d'investigation et elle n'est pas copiée. Une base dans le dossier d'un site vient avec lui, entière.

`web_server_accounts.json`
les comptes des applications web : dans chaque base SQLite des sites ou désignée par eux, chaque table dont le nom dit des comptes (`users`, `wp_users`, `AspNetUsers`, `members`…) et qui a une colonne de mot de passe, une ligne par compte : `Table`, `PasswordColumn`, `PasswordHash`, `HashFormat` — le schéma que dit sa forme : bcrypt, phpass, argon2, PBKDF2 (Django, ASP.NET Core Identity v3), crypt ; un condensat hexadécimal nu est dit par sa taille, l'algorithme n'étant pas certain —, les autres colonnes dans `Fields`, chacune `Column` et `Value` (identifiant, courriel, dates telles qu'écrites : un nom venu de la base n'est jamais une clé), et `Database`. Un nom de table qui n'est pas fait de lettres, de chiffres et de _ n'est pas lu. Juger un mot de passe faible revient à l'analyse (WAE).

`web_server_installations.json`
Apache et nginx, trouvés sans rien exécuter : par leur service (`ImagePath`, ou `Parameters\Application` pour un service enveloppé par NSSM), ou, lancés à la main, par le dossier où leur archive se déplie : par exemple `\Apache24`, `\xampp\apache`, le dossier `ApacheSoftwareFoundation` ou `\nginx-1.28.0`. Chacun avec `Product`, `Source`, `Root`, `Config File`, ses modules (`Modules` : `LoadModule`, `load_module`, un module ajouté voit chaque requête) et ses hôtes (`Hosts` : noms, adresses d'écoute, racines des documents, alias,

journaux). La configuration est lue telle qu'écrite, `Define` développé, `Include` et `include` suivis ; une section conditionnelle est lue aussi : ce qu'elle porte est sur le disque, que la condition ait valu ou non. Une inclusion hors de la collecte est dite (`IncludesNotRead`). La collecte prend le dossier de la configuration, chaque racine de documents entière (sauf la racine d'un volume), et les journaux nommés avec leurs rotations. Les lignes des journaux d'accès, aux formats `common` et `combined`, rejoignent `web_server_logs.json` : la date porte son décalage, elle est convertie exactement en UTC. Une ligne d'un autre format est donnée telle quelle (`Raw`), jamais perdue.

`web_server_error_logs.json`
: une ligne par erreur d'Apache 2.2, d'Apache 2.4 et de nginx : `Time` (la date locale de la machine, sans décalage écrit, convertie avec son fuseau), `Level`, `Module`, `ProcessId`, `Thread Id`, `ClientAddress`, `Message`, `File` et `Line`. Une autre disposition (`ErrorLogFormat`) est donnée en `Raw`.

`web_server_database_engines.json`
: les moteurs de bases que nomment les services : MySQL et MariaDB (`mysqld.exe` ou `mariadbd.exe`, le dossier de données lu dans le fichier d'options, `datadir`), PostgreSQL (`pg_ctl runservice -D`). `Product`, `Service`, `Binary`, `ConfigFile`, `DataFolder`, `DataFolderCopied`. Le dossier de données est pris brut, entier, jusqu'à `max_database_mib` ; au-delà, seuls sa configuration et ses journaux, et le journal d'investigation le dit. Un moteur en marche ne bloque rien : le volume est lu directement. Ses fichiers ne sont pas décodés.

`dpapi_master_keys.json`
: clés maîtresses DPAPI du système (`System32\Microsoft\Protect`, `Scope System`) et de chaque profil (`AppData\Roaming\Microsoft\Protect`, `User`). Les fichiers sont copiés tels quels dans la consigne, toujours chiffrés par le mot de passe de l'utilisateur ou par le secret DPAPI du système, avec `Preferred` et `CREDHIST` : avec ce mot de passe, l'analyse peut déchiffrer plus tard les secrets enregistrés (mots de passe des navigateurs, Wi-Fi, gestionnaire d'identifiants). Le JSON n'en donne que les métadonnées, jamais la clé : propriétaire (`SID`), `Guid` (celui de l'en-tête, vérifié égal au nom du fichier), version, politique, taille de chaque bloc — `DomainKeyLength` non nul : compte d'un domaine —, dates du fichier, et `Preferred` avec `PreferredExpiresUtc` pour la clé en usage.

`startup_folders.json`
: contenu des dossiers de démarrage, de tous les utilisateurs (`Scope` `AllUsers`) et de chaque profil (`User`, avec `SID`) : chaque fichier, ses dates et ses empreintes ; un raccourci est décodé (`Shortcut`, comme dans `recentdocs.json`).

`shortcuts.json`
: raccourcis posés pour être cliqués, hors du dossier Recent : menu Démarrer de tous les utilisateurs et de chaque profil, Bureau de chaque profil et Bureau public, Quick Launch et éléments épinglés à la barre des tâches (`Location` : `StartMenu`, `Desktop`, `Quick Launch`, `TaskBar` ; `Scope` et `SID` comme ci-dessus). Chaque raccourci est décodé comme dans `recentdocs.json` : cible, arguments, volume et machine d'origine. Un programme désinstallé y laisse son raccourci ; un intrus y pose une ligne de commande, ou redirige un raccourci existant vers son binaire en gardant l'icône. Le dossier Startup du menu Démarrer n'y figure pas : il relève de `startup_folders.json`. Un Bureau redirigé ailleurs sur un disque local (OneDrive, un autre volume, voir `user_shell_folders.json`) est lu là

aussi.

`user_shell_folders.json`
dossiers de chaque utilisateur (`UserShellFolders` de son `NTUSER.DAT`) : nom (`Name`, et `Folder Name` pour ceux qu'un GUID désigne, comme les Téléchargements), valeur telle qu'écrite (`Value`), chemin résolu (`Path`, absent pour un partage réseau) et date de dernière écriture de la clé. OneDrive déplace le Bureau et les Documents, une entreprise les redirige vers un partage : c'est là qu'il faut chercher les fichiers de l'utilisateur, et une redirection posée en silence est elle-même une trace.

`powershell_history.json`
historique PSReadLine de chaque profil et de chaque hôte (`Host` : `ConsoleHost`, l'ISE, VS Code) : les commandes dans l'ordre où elles ont été tapées, une commande sur plusieurs lignes réunie. Aucune date par commande : `SourceModified` date la dernière. Cherché dès Windows 7 : par défaut depuis Windows 10, il existe sous 7, 8 et 8.1 dès que PowerShell 3 ou plus et PSReadLine y ont été installés. Vista et XP s'arrêtent à PowerShell 2, sans historique sur disque ; leurs journaux PowerShell sont dans `events.json`.

`powershell_profiles.json`
profils PowerShell, les scripts que PowerShell exécute à chaque démarrage — une persistance qui ne passe ni par le registre, ni par une tâche, ni par un service. Ceux de la machine, pour tous les utilisateurs (dossier de Windows PowerShell dans `System32` et `SysWOW64`, de PowerShell 6 et 7 dans `ProgramFiles\PowerShell`), et ceux de chaque utilisateur (`Documents\WindowsPowerShell` et `Documents\PowerShell`, là où User Shell Folders place les documents s'ils sont déplacés) : `Scope` (`Machine`, `User`), `Edition`, `Host` (`AllHosts` pour `profile.ps1`, sinon la console, l'ISE, VS Code), `Path`, dates, `Size`, `Sha256` et `Content`, le texte du script. Cherchés sur toutes les versions : sous XP et Vista, PowerShell n'est présent qu'une fois installé.

`activities_cache.json`
Chronologie Windows (`ActivitiesCache.db` de chaque compte de chaque profil, avec son journal `-wal`) : chaque ligne des tables `Activity` et `ActivityOperation` (`Table`), toutes ses colonnes. Sous Windows 10, les applications ouvertes, les fichiers et pages vus (`Payload` : titre, URI), le temps passé au premier plan (`Payload` `UserEngaged`, de `StartTime` à `EndTime`) et, si l'historique du presse-papiers est actif, ce qui a été copié (`ClipboardPayload`) ; sous Windows 11, qui a retiré la Chronologie, les seuls réglages synchronisés. Les dates sont des temps Unix en UTC, donnés en UTC et en heure locale ; les GUID dans l'ordre d'octets de .NET ; une date ou un GUID nul n'est pas émis. Les codes (`ActivityType`, `ActivityStatus`, `OperationType`) sont donnés tels quels : Microsoft ne les documente pas.

`search_index.json`
index de recherche Windows (Windows 11 : `Windows.db` et `Windows-gather.db`, avec leurs journaux `-wal`). Une fiche par élément indexé du magasin de propriétés (`Table` `Property Store`, `WorkId`) : chemin (`ItemUrl`, et `ItemPathDisplay` dans la langue de l'interface), dates de création, de modification, d'accès et d'indexation, taille, propriétaire, type et, pour un document, le début de son texte (`SearchAutoSummary`) ; chaque propriété sous son nom Windows sans le préfixe `System.`. Une fiche par document vu par le collecteur (`Table` `Gather`, `DocumentId`, le même numéro que `WorkId`) : son URL et sa dernière modification

au passage du collecteur (sous Vista, son chemin `ItemPath` en capitales, tiré de la table des chemins, et toutes ses dates en FILETIME). L'enregistrement survit au fichier jusqu'au passage suivant : un document supprimé peut encore y paraître. Jusqu'à Windows 10, l'index est une base ESE, `Windows.edb` : ses journaux y sont rejoués dans la copie de travail, puis il est converti sous la même forme ; le type d'une propriété, que cette base ne dit pas, vient de la table que Windows 11 déclare, et une propriété qu'elle ne connaît pas est donnée en hexadécimal. Mesuré sous Windows 8.1, et sous Windows 7, dont la base nomme autrement ses tables et ses colonnes, écrit ses nombres et ses dates en gros-boutiste et brouille son texte ; une valeur de Windows 10 compressée autrement que sur 7 bits (XPRESS) est signalée, non décodée. **Attention** : sous Windows 7 et 8, l'indexeur garde en mémoire plusieurs minutes ce qu'il vient d'indexer avant de l'écrire dans sa base ; WAC lit la base telle qu'elle est sur le disque — il ne redémarre aucun service sur la machine examinée — et les dernières minutes peuvent y manquer.

`rdp_client.json`
: connexions Bureau à distance *sortantes* de chaque profil : connexions récentes (`Recent Connection`, `MruPosition` 0 pour la plus récente) et serveurs contactés (`Server` : nom d'utilisateur proposé `UsernameHint`, empreinte du certificat accepté `CertHash`, dernière écriture de la clé), fichiers `.rdp` des Documents (`ConnectionFile`, `Default.rdp` compris, chaque réglage), et cache bitmap (`BitmapCache`) : chaque tuile de `Cache*.bin`, morceau de l'écran distant tel que le client l'a dessiné, est exportée en image dans `rdp_cache\<SID>\<fichier>`, avec sa taille, ses clés et le SHA-256 de ses pixels. L'ancien format `bcache22.bmc`, celui du client de Windows XP, est décodé de même : tuiles compressées par le RLE entrelacé de RDP, 16 bits par pixel (`BitsPerPixel`), `UnreadTiles` comptant celles qui n'ont pu l'être. Ce cache garde ce qui s'est affiché à l'écran, y compris un texte saisi en clair dans un champ visible. Un autre `.bmc`, d'une profondeur dont WAC n'a pas d'échantillon, est compté sans être décodé (`DecodingStatus UnsupportedFormat`).

`rdp_server.json`
: connexions Bureau à distance *entrantes*, lues dans les journaux collectés : connexion reçue et authentification (RemoteConnectionManager 261 et 1149), connexion TCP acceptée (RdpCoreTS 131), sessions (LocalSessionManager 21 à 25, 39 à 41), ouvertures et fermetures de session de type 10 et reconnexions (Security 4624, 4625, 4634, 4778, 4779). Chaque événement garde ses données et reçoit `User`, `Domain`, `SourceAddress` et `SessionId`, quel que soit le nom que lui donne son journal. Peu utile sur un poste, c'est la première question sur un serveur.

`emails.json`
: courriels d'Outlook de chaque profil : les PST (`Documents\Outlook Files`) et les OST, copie hors connexion d'un compte Exchange ou Microsoft 365 (`AppData\Local\Microsoft\Outlook`), lus par le code de WAC — PST Unicode d'Outlook 2003 et suivants, OST d'Outlook 2013 et suivants (pages de 4 Kio, blocs compressés). Une fiche par boîte (type, version, nombre de messages lus, de messages illisibles) et une par message : dossier, classe, objet, expéditeur, destinataires (`Recipients` : nom, adresse, To/Cc/Bcc), dates d'envoi, de remise, de création et de modification, identifiant Internet du message, en-têtes de transport. Le corps (texte, HTML) et chaque pièce jointe sont écrits à côté du JSON (`emails\<SID>\<fichier>\<message>\`), avec leur taille et leur SHA-256 ; le nom d'une pièce jointe, choisi par l'expéditeur, n'est jamais pris pour un chemin : le fichier porte son rang et, si

elle est simple, son extension. Un message joint (un rendez-vous, un courriel transféré en pièce jointe) donne son objet et sa classe. Les fichiers qu'Outlook a ouverts depuis un message (`INetCache\Content.Outlook`) et les données locales du nouvel Outlook (`AppData\Local\Microsoft\Olk`) sont prélevés tels quels. Non décodés : le chiffrement « élevé » (cyclique) et le format ANSI d'Outlook 97 à 2002, sans échantillon pour les vérifier — la fiche de la boîte le dit. Thunderbird : ses profils (`AppData\Roaming\Thunderbird\Profiles`) sont prélevés entiers — carnet d'adresses et réglages des comptes compris — et chaque dossier de courrier, un fichier mbox, est décodé : en-têtes (mots encodés compris), date d'envoi en UTC, corps texte et HTML, pièces jointes (base64, quoted-printable, noms RFC 2231), dans les mêmes champs que pour Outlook, avec `Offset` (position du message dans le fichier) au lieu de `NodeId`.

`sysmon.json`
: Sysmon (Sysinternals), là où il est installé : une fiche par installation, reconnue au lien entre le service (`Parameters\DriverName`) et la clé de son pilote qui porte des règles — trouvée même renommée. Le service et le pilote avec leur `ImagePath`, la date de dernière écriture de `Parameters` (dernier changement de configuration), toutes les valeurs de cette clé telles que stockées (`Parameters` : `Rules`, `HashingAlgorithm`, `Options`, `Archive Directory`, `ConfigFile`, `ConfigHash`), `RulesSha256`. Le format binaire des règles n'est pas documenté par Sysinternals : elles sont données en hexadécimal, non décodées. Le dossier d'archives (`ArchiveFolder`, `C:\Sysmon` par défaut) est prélevé entier : ce sont les fichiers supprimés sous une règle `FileDelete`, nommés par leur empreinte — le contenu même d'un outil effacé —, listés dans `ArchivedFiles` avec leur taille et leur SHA-256 ; le fichier de configuration XML, s'il est encore là, est prélevé et son SHA-256 donné (`Config FileSha256`, à comparer à `ConfigHash`). Le journal de Sysmon est converti avec les autres dans `events.json`, même déplacé ; sa version y est (événement 4).

`thumbcache.json`
: caches des miniatures et des icônes de l'Explorateur, de chaque profil (`thumbcache_*.db`, `iconcache_*.db`) : ce que l'utilisateur a eu sous les yeux, même si le fichier a été supprimé depuis. Pour chaque entrée : son cache et sa taille, sa clé (`Hash`), son identifiant, le `ThumbnailCacheId` que Windows donne pour le fichier, ce qui relie une miniature à son fichier (la clé depuis Windows 7, l'identifiant sous Vista), ses dimensions, son format (`png`, `jpg`, `bmp`), le SHA-256 de l'image, et l'image elle-même, exportée à côté du JSON (`thumbcache\<SID>\<cache>\<ThumbnailCacheId>.<ext>`). Les deux sommes de contrôle que Windows écrit pour chaque entrée sont recalculées (`HeaderChecksumValid`, `Data ChecksumValid`) : une entrée lue de travers les échoue. Vista à Windows 11 (Vista et 7 ne donnent pas les dimensions ; Vista donne l'extension du fichier d'origine, `Source Extension`). Après la première place libre que l'en-tête annonce, des entrées d'un état antérieur peuvent rester entières : elles sont rendues comme vestiges (`Remnant`) si leur somme d'en-tête est juste, leur image seulement si sa somme l'est aussi (`<ThumbnailCache Id>-remnant-<position>.<ext>`) ; la lecture s'arrête à la première qui ne l'est pas. Sous Windows XP et Server 2003, les `Thumbs.db` que l'Explorateur écrit dans chaque dossier affiché en miniatures, cherchés par leur nom sur chaque volume NTFS fixe : pour chaque miniature, le nom et le chemin de l'image, sa date de modification au moment de la miniature (`SourceModifiedUtc`), même une fois l'image effacée, la taille et le SHA-256 de la miniature, et ses octets exportés tels quels (`.bin`) : un JPEG à quatre composantes sans ses tables, qu'aucun visualiseur ordinaire n'affiche.

`error_reports.json`
rapports d'erreurs Windows (`Report.wer`) de la machine et de chaque profil : programme, chemin, instant (`EventTimeUtc`), signature du problème (module fautif, version), toutes les valeurs du rapport dans `Values`.

`defender_log.json`
journaux de support de Microsoft Defender (`MPLog-*.log`) : chaque ligne telle qu'écrite, datée quand elle commence par une date UTC. Analyses à l'exécution, détections, exclusions, mises à jour. Les traces WPP du même dossier (`MpWpp*.bin`, traces ETW binaires que seuls les symboles privés de Microsoft décodent) sont copiées brutes dans la consigne, sans conversion.

`defender_detections.json`
une entrée par fichier DetectionHistory, qui survit à l'effacement du journal d'événements : menace, ressource (fichier, clé), compte, processus, date de détection initiale, entrée de quarantaine, et les propriétés nommées `ThreatTracking` (empreintes, taille, heure de début). Le format n'est pas documenté : seuls les champs dont la position a été confirmée contre `MSFT_MpThreatDetection` sont nommés ; les autres sont rendus tels que lus (`UnnamedFields` : position, type, valeur), sans signification inventée. Defender sous Windows 8.1 écrit les mêmes champs sans le marqueur `MagicVersion` de Windows 10 et 11 : ils sont nommés de la même façon, confirmés de la même façon.

`defender_quarantine.json`
entrées de la quarantaine : menace, instant de mise en quarantaine, chemin d'origine du fichier retiré, ses dates de création et de modification, ses attributs, sa taille, et l'empreinte de son contenu (`ContentSha256`). La quarantaine est copiée **chiffrée**, telle que Defender la garde : rien de malveillant n'est écrit en clair, ni sur le support de collecte ni sur le poste d'analyse. Le contenu n'est déchiffré qu'en mémoire, pour son empreinte.

`hosts.json`
entrées du fichier `hosts` : adresse et noms, numéro de ligne ; un Windows intact n'en a aucune. Un fichier de plus de 16 Mio n'est pas lu (journalisé) : il reste dans la consigne.

6.5.3 Fichiers et navigation

`mft.json`
chaque fichier et répertoire de chaque volume NTFS fixe (`Volume` : `C:`, `G:`…; un numéro d'enregistrement n'a de sens que dans son volume), présent ou **supprimé** (`Deleted`) tant que NTFS n'a pas réutilisé son enregistrement : chemin reconstitué, taille, attributs, et deux jeux de quatre dates en UTC — `CreatedUtc`… (attribut `$STANDARD_INFORMATION`, ce que montre l'Explorateur et que réécrit un outil d'antidatage) et `FileNameCreatedUtc`… (attribut `$FILE_NAME`, écrit par NTFS seul). Une date `$STANDARD_INFORMATION` antérieure à celle de `$FILE_NAME` trahit un antidatage. `PathIncomplete` marque un chemin dont un répertoire parent a été réutilisé depuis : le chemin n'est pas inventé. Un fichier à liens physiques (`HardLinks`) est un seul enregistrement sous plusieurs noms — `System32\crypt32.dll` et son original sous `WinSxS` : `HardLinkPaths` donne ses autres chemins, chacun cherchable. `SecurityId` est le numéro de son descripteur de sécurité dans `security_descriptors.json`.

`security_descriptors.json`
: descripteurs de sécurité de chaque volume NTFS fixe (`$Secure:$SDS`) : chacun une seule fois, quel que soit le nombre de fichiers qui le partagent, repéré par `Volume` et `SecurityId` — la clé que porte chaque fichier de `mft.json`. Il donne le **propriétaire** (`Owner`, `OwnerName`), le groupe, et chaque entrée de la DACL et de la SACL (type, drapeaux, masque d'accès, SID et nom). L'outil d'un intrus déposé par son compte, un fichier système qui n'appartient plus à TrustedInstaller, une ACL ouverte à Everyone s'y lisent, fichiers supprimés compris. NTFS range avec chaque descripteur un condensat de ses octets : WAC le recalcule, et `Hash Mismatch` signale un descripteur qui ne lui correspond pas.

`deleted_files.json`
: le **contenu** des fichiers supprimés, là où `mft.json` n'en donne que l'enregistrement : l'outil, le script ou l'archive qu'un intrus a effacés restent dans leurs clusters tant que NTFS ne les a pas donnés à un autre fichier. Lus en brut, sans rien ouvrir ni écrire sur la machine examinée. Le contenu résident (quelques centaines d'octets, dans l'enregistrement même) est rendu tel quel ; chaque cluster d'un contenu non résident est vérifié libre dans `$Bitmap` avant et après sa lecture : donné entre-temps à un autre fichier, il devient un trou de zéros. Un cluster pris puis rendu depuis la suppression se relit libre : les plages de tous les enregistrements supprimés de la `$MFT` sont donc croisées, et un cluster qu'un fichier supprimé plus récent a tenu devient un trou (`Reason` « clusters claimed since by a later deleted file »). Un fichier intermédiaire dont l'enregistrement a été réutilisé ne laisse pas de plages : ce qu'il a écrit ne se voit pas ainsi. `State` : `Recovered` (tout le contenu), `Partial` (trous listés dans `MissingRanges`, octet de début et longueur), `Overwritten` (plus rien), `Zeroed` (clusters relus à zéro : un disque SSD qui a reçu TRIM), `NotRecoverable` et `Reason` (compressé, chiffré par EFS, plages en partie dans un enregistrement d'extension, plages incohérentes), `RecordReused` (l'enregistrement a servi à un autre fichier depuis la copie de la `$MFT`), `NotRead` (non lu : `max_total_mib` atteint). Avec le chemin (`Path`), les dates de l'enregistrement, l'heure de la suppression (`DeletedUtc`, lue dans le journal des modifications : aucun enregistrement ne la porte), `RecoveredBytes` et les empreintes `RecoveredSHA256`… du contenu *tel que récupéré*, trous compris : celles du fichier d'origine seulement si l'état est `Recovered`. Le contenu copié est dans la consigne sous `X:\$DeletedFiles\<enregistrement>-<séquence>_<nom>` (`Exhibit`), jamais sous son ancien chemin, où un autre fichier peut se trouver. Depuis Windows 10, un fichier supprimé encore ouvert est d'abord renommé dans `$Extend\$Deleted` : son enregistrement l'y nomme, et `FormerPath` donne le chemin qu'il avait, lu dans le journal des modifications. Un exécutable récupéré suit la règle des autres (`binary`) : authentique, il est relevé avec son `Signature` sans être copié (`Copied false`, `NotCopied`) ; sous `binary: none`, rien n'est jugé et il est copié. Les plus récents d'abord (dernière date de l'enregistrement, celle de la suppression n'y figurant pas), jusqu'à `max_total_mib` (`unlimited` lève ce plafond et celui de `max_file_mib` ; un fichier taillé reste vérifié en mémoire, jusqu'à 4 Gio, 1 Gio avec `WAC32.exe` ; le support doit alors pouvoir tout contenir, WAC ne le réserve plus) ; les dossiers `excluded_folders` (mises à jour de Windows) ne sont pas lus, ni le dossier de sortie. Le réglage est une structure sous `artefacts:` ; `deleted_files: false` suffit pour l'écarter. `Source` dit d'où vient l'entrée : `MftRecord`, un enregistrement supprimé de la `$MFT` (niveau 1) ; `IndexSlack` (niveau 2), un nom resté dans la marge d'un bloc d'index de répertoire (`$I30`) ou dans un bloc que le répertoire a libéré : le nom, le répertoire (`Path`), l'enregistrement du fichier (`Entry`, `Sequence`), sa taille et les quatre dates de son `$FILE_NAME` (`FileNameCreatedUtc`…), `State NameOnly` — aucun contenu. Seul un nom

qu'aucun répertoire ne porte plus est donné, et pas un nom déjà rendu par le niveau 1. `CurrentPath` : le même fichier existe encore sous ce chemin, déplacé ou renommé et non supprimé. `Carved` (niveau 3, `level: 3` seulement : tout l'espace libre est relu, aussi longtemps qu'il faut) : un fichier taillé dans les clusters libres que le niveau 1 n'a pas rendus, sans nom ni date — `Format` (`PE`, `ZIP` et les documents Office, `PDF`), premier cluster (`Lcn`), `Size`. Seuls ces formats, dont la structure donne la taille exacte et se vérifie, sont taillés ; un fichier n'est pris que si tous ses clusters sont contigus, libres et non rendus par le niveau 1 — un fichier fragmenté n'est pas trouvé, jamais coupé faux. Pièce sous `X:\$CarvedFiles\<cluster>.<ext>`. `FatEntry` : une entrée supprimée d'un volume FAT ou exFAT (une clé du suspect) : son premier cluster (`FirstCluster`), sa taille, ses dates telles que stockées (`ModifiedAsStored`, heure locale de la machine qui l'a écrite, ou `Modified Utc` quand exFAT en donne le décalage). La chaîne de la FAT est effacée : le contenu est relu comme les clusters qui suivent le premier, chacun vérifié libre — `State` `Recovered IfContiguous` et `ContiguityAssumed`, sauf quand exFAT savait le fichier d'un seul tenant (`Recovered`). Le niveau 3 taille aussi l'espace libre de ces volumes. Sur un volume BitLocker, la lecture brute voit le volume déchiffré par le système, comme pour les autres artefacts : les clusters libres aussi.

`usn_journal.json`

journal des modifications de chaque volume NTFS fixe (`$UsnJrnl:$J`, `Volume` ; les numéros `Usn` croissent au sein d'un volume) : chaque création, écriture, renommage et **suppression**, daté, avec ses motifs (`Reasons`, un bit inconnu rendu en hexadécimal) et le chemin reconstitué par la `$MFT`. Il garde la trace de fichiers déposés puis effacés, que la `$MFT` a déjà oubliés. La `$MFT`, le journal des modifications, `$LogFile` (journal de transactions, non converti) et `$TxfLog` (journal de NTFS transactionnel, journal CLFS, ses conteneurs et le flux `$Tops:$T`, non converti) sont en outre copiés bruts dans la consigne, pour refaire plus tard la chronologie avec un autre outil.

`ntfs_volumes.json`

chaque volume NTFS fixe, lu dans son secteur d'amorçage (`$Boot`, reconnu à ses signatures avant tout champ) : numéro de série (`VolumeSerial`, les 32 bits que les raccourcis, listes de sauts et fichiers Prefetch relèvent du volume de leur cible, et `VolumeSerial64` en entier), tailles de secteur, de cluster, d'enregistrement et de bloc d'index, emplacement de la `$MFT` et de son miroir. Puis l'en-tête du journal des modifications (`$UsnJrnl:$Max`, `UsnJournal`) : taille maximale, pas d'allocation, `UsnJournalId`, qui est la date de **création** du journal (`Created`), et premier numéro encore présent. Un journal supprimé puis recréé (`fsutil usn deletejournal`) se lit à une création bien postérieure à l'installation.

`recentdocs.json`

raccourcis de documents récents : cible, horodatages du raccourci et de la cible, taille de la cible, volume (type, numéro de série, nom), chemin réseau, liste d'éléments du shell, et les blocs de données supplémentaires : le bloc de suivi (`Tracker` : nom NetBIOS de la machine où était la cible, identifiants de suivi, et ce qu'en révèle le GUID de version 1 — la date de début du suivi et l'adresse matérielle de la machine), chemins avec variables d'environnement, dossier connu, propriétés de la cible.

`jumplistAutomaticDestinations.json`, `jumplistCustomDestinations.json`

listes de sauts par application, avec les raccourcis qu'elles contiennent ; pour les listes automatiques, chaque raccourci est accompagné de son entrée DestList (dernier accès,

nom de la machine, identifiants de suivi avec leur date et leur adresse matérielle, épinglage, nombre d'ouvertures `InteractionCount`, rang `MruPosition` — 0 pour le plus récent —, nom du dossier connu visé) ; pour les listes personnalisées, leurs catégories dans l'ordre (`Categories` : une catégorie propre et son nom, une catégorie connue — fréquents ou récents —, les tâches).

`shellbags.json`
dossiers parcourus dans l'Explorateur, arborescence reconstituée ; pour chaque dossier, son rang parmi ses voisins (`MruPosition`, 0 pour le plus récent), la date d'écriture de la clé qui le porte (`LastWriteTime`) et de sa propre clé (`KeyLastWriteTime` : son ajout, ou le parcours d'un sous-dossier), et son `NodeSlot`. Les « première » et « dernière interactions » d'autres outils ne sont que des lectures de ces deux dates. Sous Windows XP, les dossiers de l'Explorateur sont dans NTUSER.DAT (`ShellNoRoam\BagMRU`), lus de même.

`mrus.json`, `mruApps.json`
listes de fichiers et d'applications récemment utilisés. Sous Windows XP, en texte : le chemin du fichier (`Path`), le programme et son dossier (`Folder`), rangés par `ID`, 0 pour le plus récent.

`zone_identifiers.json`
flux `Zone.Identifier` (« marque du web ») que navigateurs et messageries écrivent sur ce qu'ils téléchargent : fichier, zone (`ZoneId`, `ZoneName` — `Internet` pour 3), page d'origine (`ReferrerUrl`) et adresse du fichier (`HostUrl`), toutes les valeurs dans `Values`. Lus dans la copie de la `$MFT`, où ce petit flux est résident : aucune lecture de plus sur la machine examinée ; un flux non résident, rare, est signalé (`ContentNotResident`).

`recycle_bin.json`
fichiers mis à la corbeille, sur chaque volume NTFS fixe, d'après leur index `$I` : chemin d'origine, taille, instant de la suppression (`DeletedUtc`), et l'utilisateur (`SID`). Seuls les `$I` sont prélevés ; les fichiers eux-mêmes (`$R`) figurent dans `mft.json`. Sous Windows XP et Server 2003, d'après `INFO2` de `RECYCLER\<SID>`, une entrée par enregistrement, avec le nom de la copie dans la corbeille (`RecycledName`, `D<lecteur><n>.<ext>`, `Dc1.txt` pour un fichier de C:) ; la taille est l'espace qu'occupait le fichier.

6.5.4 Périphériques

`setupapi.json`
sections de `setupapi.dev.log` et de ses rotations : opération (`Operation`), périphérique ou pilote (`Subject`, avec le numéro de série d'une clé USB), début et fin convertis depuis l'heure locale de la machine examinée, résultat. La première connexion d'un périphérique s'y date, même quand la clé `USBSTOR` a été nettoyée. Sous Windows XP, d'après `setupapi.log` : une section par ligne datée, son premier message en opération, le périphérique qu'une ligne `#I121` nomme en sujet, `SUCCESS` par cette ligne ou la première erreur en résultat ; pas de date de fin.

`Usbstor.json`
périphériques de stockage USB connus du système : `Vendor`, `Product`, `Revision` et `Type`, tirés de `DeviceId` ; `SerialNumber`, tiré de `InstanceId` — absent quand l'identifiant a été fabriqué par Windows (deuxième caractère « & »), le périphérique n'en déclarant aucun ; dates de

première et de dernière insertion.

`mounted_device.json`
correspondance entre lettres de lecteur et périphériques. `Drive` nomme la lettre ou le volume ; `Type` dit la forme de la valeur : `GPT partition` (`PartitionGuid`), `MBR partition` (`Disk Signature` et `PartitionOffset`, en octets), `Device path` (`Device`, chemin du périphérique, avec le numéro de série d'une clé USB) ou `Unknown` (`Data`, les octets en hexadécimal).

6.5.5 Journaux d'événements (`--events`)

`events.json` contient un objet par événement, de la même forme quel que soit le format du journal : les journaux `.evt` de Windows XP et Server 2003 sont lus dans les champs que Windows rend pour une source classique dans un `.evtx` (identifiant et ses qualificateurs, niveau d'après le type d'événement, catégorie en tâche, chaînes en `Data`, données binaires en `Binary`), leur zone circulaire parcourue depuis l'enregistrement le plus ancien. Un journal que le registre de la machine place hors de son dossier (valeur `File` de son canal, `wevtutil sl /lfn`, ou de son journal classique) — déplacé pour la place, ou par un intrus pour qu'on ne le cherche pas — est prélevé là où il est et converti avec les autres :

`EvtSystem*`
la section système : fournisseur, identifiant, niveau, tâche, mots-clés, horodatage de création, numéro d'enregistrement, processus et fil, canal, machine, utilisateur ;

`EvtEventData`
les données de l'événement, en liste de paires `Name` / `Value` ;

`EvtEventMessage`
le message lisible, tel que l'Observateur d'événements l'afficherait, reconstitué à partir des ressources du fournisseur prélevées sur la machine : manifeste (`WEVT_TEMPLATE`, ou son jumeau `SystemResources\*.mun` sous Windows 11) et table de messages dans la langue de l'interface de la machine, ou fichiers `EventMessageFile` d'une source classique. Chaque donnée y est montrée comme Windows la montre : libellé de sa table de valeurs ou de ses drapeaux, hexadécimal, texte d'un code d'erreur Win32, HRESULT ou NTSTATUS, adresse réseau ;

`EvtSourceLog`
le fichier journal dont provient l'événement. Un même canal peut être porté par plusieurs fichiers (journal courant et archives) dont les numéros d'enregistrement se recouvrent ;

`EvtRecovered`
`true` pour un événement d'un journal *vidé* : Windows remet l'en-tête du bloc à zéro sans effacer les enregistrements, que WAC relit. L'Observateur d'événements ne les montre plus ; ils restent la trace de ce que l'effacement voulait faire disparaître, et ne doivent jamais être lus comme des événements vivants.

> **Messages incomplets**
> Une marque `%1`, `%2`… restée dans un message signifie que l'événement ne porte pas la

> donnée attendue : elle est laissée visible pour que la phrase ne paraisse pas complète. Tous les fournisseurs ne livrent pas de ressources de messages : leurs événements n'ont alors pas de `EvtEventMessage`, mais leurs données restent complètes.

CHAPITRE 7

Lire les données

7.1 Horodatages

Chaque date est émise deux fois : `X` en heure locale et `XUtc` en UTC. L'heure locale est celle du **suspect**, calculée avec le fuseau lu dans sa ruche `SYSTEM` — pas celle de la machine de collecte. Les dates portent la fraction de seconde quand la source la fournit, et seulement dans la précision de cette source : sept chiffres pour un `FILETIME`, trois pour une date à la milliseconde, aucun pour une date à la seconde (date FAT, précise à deux secondes, date lue en texte, temps Unix).

Chaque date porte le décalage en vigueur **à cette date**, heure d'été comprise : une date de janvier relevée en été à Paris est à `+01:00`, une date de juillet à `+02:00`. WAC applique les règles de la machine examinée année par année (clé `Dynamic DST` de la ruche `SOFTWARE`) : une date de 2005 aux États-Unis suit les règles de 2005. L'heure locale écrite est toujours dérivée de l'heure UTC : un même instant se lit à la même heure dans tous les artefacts. Deux dates n'ont aucun fuseau connaissable et sont données **telles qu'écrites**, sans décalage ni UTC, avec `TimeZone` qui le dit : la date d'un élément vu dans une archive ZIP (`ModifiedAsWritten` ; une entrée ZIP garde l'heure d'horloge de la machine qui a fait l'archive — les archives que Windows 11 lit lui-même, 7z ou tar, portent au contraire un instant UTC, donné comme tel) et une date OLE d'un magasin de propriétés (`AsWritten`). Les placer sur une chronologie demande de connaître leur origine : WAC ne l'invente pas. Pour une valeur que l'artefact enregistre en heure locale (volume FAT, `setupapi.dev.log`), l'heure répétée à l'automne est lue comme sa première occurrence, et l'heure sautée au printemps avec le décalage d'avant le changement, comme le fait Windows.

7.2 Noms des comptes

Les champs qui nomment un compte (`SIDName`, `Owner`, `RunAs`) sont tirés des preuves seules, sans interroger le système ni le réseau : comptes et groupes locaux de la base SAM (dans la langue de la machine examinée), SID bien connus sous leur nom canonique (`SYSTEM`, `LOCAL SERVICE`…), SID de services (`NT SERVICE\…`), et dossier de profil pour un compte de domaine. Un SID qu'aucune de ces sources ne nomme est publié sans nom : le SID reste la preuve, à rapprocher du domaine par l'analyste.

Le profil Default. `C:\Users\Default` est le modèle copié pour chaque nouveau profil : une valeur Run, un raccourci ou une association posés là atteignent tous les comptes créés ensuite. WAC le lit comme un profil (ruches `NTUSER.DAT` et `UsrClass.dat`, Bureau, menu Démarrer, Recent…). N'appartenant à aucun compte, il n'a pas de SID : ses entrées portent le SID `S-0-0-0`, fait de zéros, au format d'un SID pour que les outils les lient, mais qui n'en

est pas un (révision 0), et `Profile` : `Default` à la place de `SIDName`. Ce n'est pas `S-1-0-0`, le vrai SID nul de Windows.

7.3 Noms des GUID

Les champs qui nomment un GUID (classe COM, dossier connu, gestionnaire d'une tâche) viennent d'abord d'une table de référence, puis des ruches de la machine examinée : ses classes (`SOFTWARE\Classes\CLSID`), ses dossiers connus (`FolderDescriptions`), et les classes de chaque utilisateur (`UsrClass.dat`). Un composant installé sur la machine y est donc nommé, avec le module qui le sert. Un GUID que rien ne nomme est publié sans nom.

7.4 Un champ absent n'est pas un champ vide

Une valeur sans source n'est pas émise du tout. Une clé vide se lirait comme une lecture ratée, et ne se distinguerait pas de « la ruche ne contient pas cette valeur ». Ce qui n'a pas pu être lu est consigné dans `investigation.json`.

7.5 Artefact non collecté

Quand un artefact entier n'a pas pu être lu, son fichier le dit explicitement :

```
{
  "Artifact": "USBSTOR (registre)",
  "CollectionStatus": "NotCollected",
  "Error": "0x02 Le fichier spécifié est introuvable.",
  "Note": "La lecture de cet artefact a échoué : l'absence de données
           ci-dessus ne signifie PAS qu'aucune trace n'existe sur le système."
}
```

> **Attention**
> Un fichier d'artefact vide et un fichier « NotCollected » ne disent pas la même chose. Le premier indique qu'aucune entrée n'a été trouvée ; le second, que la source n'a pas pu être lue.

7.6 Artefact absent de la version de Windows

Un artefact n'existe pas sur toutes les versions de Windows : pas de SRUM avant Windows 8, pas de jump lists avant Windows 7. WAC lit la version de la machine examinée dans sa ruche `SOFTWARE`, l'annonce au début de la collecte et la consigne dans `investigation.json`. Un artefact demandé que cette version ne porte pas n'est ni lancé ni compté en échec, et son fichier le dit :

```
{
  "Artifact": "RESOURCE USAGE (SRUM)",
  "CollectionStatus": "AbsentFromThisVersion",
  "WindowsRelease": "Windows 7",
  "WindowsBuild": 7601,
  "Note": "This artefact does not exist on this version of Windows
           (artefacts.srum): there was nothing to collect. Its absence is
           normal and says nothing about what happened on the system."
}
```

Une version plus récente que celles que WAC connaît est traitée comme la dernière connue, et signalée comme telle : la collecte se fait. Le tableau ci-dessous donne, pour chaque clé de `wac.yml`, le premier Windows qui porte l'artefact, son emplacement et son format selon la version. Il est produit depuis la table du programme, la seule que WAC consulte. La version 3 de WAC est éprouvée sur chaque poste client, de Windows XP SP1 à Windows 11 ; la version 4, en plus, sur Server 2003 (Enterprise x86, par le cycle de Windows XP), Server 2008 R2, 2012, 2012 R2, 2016, 2019, 2022 et 2025 avec leur interface, et sur Server 2022 en installation Server Core.

Les emplacements sont écrits comme Windows Vista et suivants rangent leurs dossiers. Sous Windows XP et Server 2003, WAC les cherche là où ces versions gardent les mêmes données : `AppData\Local` devient `LocalSettings\ApplicationData`, `AppData\Roaming` devient `ApplicationData`, les documents récents et le menu Démarrer sont à la racine du profil, `Documents` devient `MyDocuments`, et `ProgramData` devient `AllUsers\ApplicationData`, à côté des profils.

Artefact	Depuis	Emplacement	Format	Converti
`activities_cache`	Windows 10	`Users\<user>\AppData\Local\ConnectedDevicesPlatform\<account>\ActivitiesCache.db`	SQLite	oui
	Toujours écrit par Windows 11 (mesuré, build 26200), bien que la Chronologie n'y soit plus affichée. Éditions serveur à mesurer.			
`amcache`	Windows 7	`Windows\AppCompat\Programs\RecentFileCache.bcf`	BCF	oui
	Windows 8	`Windows\AppCompat\Programs\Amcache.hve`	ruche, clés File et Programs	oui
	Windows 10	`Windows\AppCompat\Programs\Amcache.hve`	ruche, clés Inventory*	oui
	toujours présent depuis Windows 10; Amcache.hve aussi sous Windows 7 avec la mise à jour KB2952664.			
`bam`	Windows 10	`SYSTEM\CurrentControlSet\Services\bam\UserSettings`	valeurs de registre, FILETIME	oui
	Windows 10	`SYSTEM\CurrentControlSet\Services\bam\State\UserSettings`	valeurs de registre, FILETIME	oui
	Éditions serveur à mesurer.			

Artefact	Depuis	Emplacement	Format	Converti
`bits`	Windows XP	`ProgramData\Microsoft\Network\Downloader\qmgr0.dat, qmgr1.dat`	fichier d'état de BITS	oui
	Windows 10	`ProgramData\Microsoft\Network\Downloader\qmgr.db`	ESE	oui
	Avant Vista : Documents and Settings\All Users\Application Data\Microsoft\Network\Downloader. Les fichiers de file d'attente mesurés sous Windows 8.1 ; les versions antérieures lues comme bits_parser (ANSSI) les décrit, sans mesure.			
`boot_records`	Windows XP	`chaque disque physique : son premier mébioctet, les 16 premiers secteurs de chaque partition ; les fichiers de la partition système EFI`	MBR, GPT, secteurs de démarrage ; FAT	oui
	La partition système EFI n'existe que sur une machine démarrée en UEFI (Windows 8 et suivants, certains 7 et Vista).			
`cloud_identity`	Windows 10	`SYSTEM\CurrentControlSet\Control\CloudDomainJoin, NTUSER.DAT\…\WorkplaceJoin ; TokenBroker, AAD.Broker Plugin de chaque profil`	clés du registre ; caches de jetons copiés chiffrés	oui
	Présent seulement sur une machine inscrite à Entra ID ou portant un compte professionnel.			
`defender`	Windows 8	`ProgramData\Microsoft\Windows Defender\Support\MPLog-*.log`	texte UTF-16	oui
	Windows 10	`ProgramData\Microsoft\Windows Defender\Scans\History\Service\DetectionHistory`	enregistrements DetectionHistory	oui
	Server 2012 et 2012 R2 n'ont pas de Defender intégré ; la quarantaine est copiée chiffrée sur toutes les versions.			
`defender_configuration`	Windows Vista	`SOFTWARE\Microsoft\Windows Defender, SOFTWARE\Policies\Microsoft\Windows Defender`	valeurs de registre	oui
`defender_endpoint`	Windows 10	`ProgramData\Microsoft\Windows Defender Advanced Threat Protection`	fichiers, copie brute	non
	Présent seulement là où Defender for Endpoint est inscrit ; ses formats ne sont pas documentés : copié brut, non converti.			

Artefact	Depuis	Emplacement	Format	Converti
`deleted_files`	Windows XP	`$MFT, $Bitmap, $I30, clusters libres ; volumes FAT et exFAT`	enregistrements de fichier NTFS 3.1, data runs, index blocks ; entrées de répertoire FAT ; PE, ZIP, PDF carved	oui
	Ce qu'un fichier supprimé garde dépend du disque : un SSD qui a reçu TRIM rend des zéros pour les clusters libérés, déclarés comme tels, jamais comme le contenu.			
`devices`	Windows XP	`SYSTEM\CurrentControlSet\Enum\USB, SCSI`	clés de registre	oui
	Windows Vista	`…et SOFTWARE\Microsoft\Windows Portable Devices, propriétés des périphériques avec leurs dates (Windows 8)`	clés de registre	oui
`dpapi_master_keys`	Windows XP	`<profile>\Application Data\Microsoft\Protect\<SID>`	fichier de clé maîtresse, version 2	oui
	Windows Vista	`Users\<user>\AppData\Roaming\Microsoft\Protect\<SID>`	fichier de clé maîtresse, version 2	oui
`emails`	Windows XP	`PST et OST d'Outlook, mbox de Thunderbird, dans chaque profil`	PST/OST (ANSI et Unicode), mbox	oui
	Dépend du logiciel de messagerie installé, non de Windows.			
`error_reports`	Windows Vista	`ProgramData\Microsoft\Windows\WER, Users\<user>\AppData\Local\Microsoft\Windows\WER`	Report.wer, texte UTF-16	oui
	Windows XP : Dr Watson (drwtsn32.log, user.dmp), non collecté.			
`etw_logs`	Windows Vista	`Windows\System32\LogFiles\WMI, Windows\System32\WDI, Windows\Logs, ProgramData\Microsoft\Diagnosis\ETLLogs`	fichiers de trace ETL, copie brute	non
	Copié brut : non converti (événements du noyau au format MOF, manifestes des fournisseurs).			
`event_log_configuration`	Windows XP	`SYSTEM\CurrentControlSet\Services\EventLog`	clés de registre	oui
	Windows Vista	`…et SOFTWARE\Microsoft\Windows\CurrentVersion\WINEVT\Channels, Control\WMI\Autologger`	clés de registre	oui
	toujours présent depuis Windows XP			
`events`	Windows XP	`Windows\System32\config\*.evt`	EVT (enregistrements LfLe)	oui

Artefact	Depuis	Emplacement	Format	Converti
	Windows Vista	`Windows\System32\winevt\Logs\*.evtx`	EVTX, XML binaire	oui
	toujours présent depuis Windows XP			
`explorer_recent_docs`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs`	MRUListEx et éléments de shell	oui
`fat_volumes`	Windows XP	`volumes FAT12, FAT16, FAT32`	entrées de répertoire FAT	oui
	Windows Vista	`…et volumes exFAT (Vista SP1 ; XP avec KB955704)`	entrées de répertoire exFAT	oui
`firewall`	Windows XP	`SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy (XP SP2)`	valeurs de registre, profils et listes de ports	oui
	Windows Vista	`…\FirewallPolicy\FirewallRules`	valeurs de registre, règles en chaînes v2.x	oui
`firewall_log`	Windows XP	`Windows\pfirewall.log (XP SP2), ou le LogFilePath du profil`	texte W3C, heure locale	oui
	Windows Vista	`Windows\System32\LogFiles\Firewall\pfirewall.log, ou le LogFilePath du profil`	texte W3C, heure locale ; processus depuis Windows 10	oui
	Désactivé par défaut : présent seulement là où la journalisation du pare-feu a été activée.			
`hosts`	Windows XP	`Windows\System32\drivers\etc\hosts`	texte	oui
	toujours présent depuis Windows XP			
`installed_programs`	Windows XP	`SOFTWARE et NTUSER.DAT …\CurrentVersion\Uninstall, Installer\UserData`	clés de registre	oui
	toujours présent depuis Windows XP			
`internet_settings`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Internet Settings`	valeurs de registre	oui
`intune_logs`	Windows 10	`ProgramData\Microsoft\IntuneManagementExtension\Logs\*.log`	texte CMTrace, heure locale	oui
	Présent seulement sur une machine gérée par Microsoft Intune, une fois l'extension de gestion installée.			

Artefact	Depuis	Emplacement	Format	Converti
`jump_lists`	Windows 7	`Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\Automatic Destinations, Custom Destinations`	fichier composé OLE, DestList version 1	oui
	Windows 10	`les mêmes dossiers`	fichier composé OLE, DestList version 3 et 4	oui
	Windows 11	`les mêmes dossiers`	fichier composé OLE, DestList version 6	oui
	DestList version 6 mesurée sur le build 26200; les listes personnalisées (CustomDestinations) ne portent pas de version.			
`known_networks`	Windows Vista	`SOFTWARE\Microsoft\Windows NT\CurrentVersion\Network List`	clés de registre, dates SYSTEMTIME	oui
`local_groups`	Windows XP	`SAM\SAM\Domains\Builtin\Aliases`	enregistrements d'alias de la SAM	oui
	toujours présent depuis Windows XP			
`log_file`	Windows XP	`$LogFile`	journal NTFS, copie brute	non
	Windows Vista	`…et $Extend\$RmMetadata\$TxfLog`	journal NTFS transactionnel, copie brute	non
	Copié brut sur toutes les versions : non converti.			
`mft`	Windows XP	`$MFT`	enregistrements de fichier NTFS 3.1	oui
`mounted_devices`	Windows XP	`SYSTEM\MountedDevices`	valeurs de registre	oui
	toujours présent depuis Windows XP			
`mount_points`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2`	clés de registre	oui
`mru`	Windows XP	`NTUSER.DAT …\Explorer\ComDlg32\OpenSaveMRU, LastVisitedMRU`	MRUList et chaînes	oui
	Windows Vista	`NTUSER.DAT …\Explorer\ComDlg32\OpenSavePidlMRU, LastVisitedPidlMRU`	MRUListEx et éléments de shell	oui
`mru_apps`	Windows XP	`NTUSER.DAT …\Explorer\ComDlg32\LastVisitedMRU`	MRUList et chaînes	oui

Artefact	Depuis	Emplacement	Format	Converti
	Windows Vista	`NTUSER.DAT …\Explorer\ComDlg32\LastVisitedPidlMRU`	MRUListEx et éléments de shell	oui
`muicache`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\ShellNoRoam\MUICache`	valeurs de registre	oui
	Windows Vista	`UsrClass.dat\Local Settings\Software\Microsoft\Windows\Shell\MuiCache`	valeurs de registre	oui
`network`	Windows XP	`en direct : tables TCP/IP du système en marche`	observation	oui
`network_configuration`	Windows XP	`SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces, Control\Network`	valeurs de registre	oui
	Windows 10	`…et Control\NetworkSetup2\Interfaces\<guid>\Kernel : adresses matérielles`	valeurs de registre	oui
	toujours présent depuis Windows XP ; Avant Windows 10, les adresses matérielles des cartes ne sont dans aucune clé documentée des ruches : non données.			
`network_shares`	Windows XP	`SYSTEM\CurrentControlSet\Services\LanmanServer\Shares`	valeurs de registre, descripteurs de sécurité	oui
`ntfs_volumes`	Windows XP	`$Boot, $Extend\$UsnJrnl:$Max`	secteur de démarrage NTFS, en-tête du journal	oui
`office_trust_records`	Windows XP	`NTUSER.DAT\Software\Microsoft\Office\<version>\<program>\Security\Trusted Documents\TrustRecords`	valeurs de registre	oui
	Dépend d'Office (2010 et suivants), non de Windows.			
`onedrive`	Windows 7	`Users\<user>\AppData\Local\Microsoft\OneDrive\logs et settings`	journaux ODL, base de synchronisation, copie brute	non
	Présent seulement là où OneDrive a tourné ; journaux de la synchronisation des listes SharePoint (ListSync) écartés. Copié brut : non converti.			
`operating_system`	Windows XP	`SOFTWARE\Microsoft\Windows NT\CurrentVersion, SYSTEM\…\Control`	valeurs de registre	oui
	toujours présent depuis Windows XP			
`persistence`	Windows XP	`SOFTWARE, SYSTEM et NTUSER.DAT : IFEO, AppInit_DLLs, Winlogon, LSA, Active Setup…`	valeurs de registre	oui
	toujours présent depuis Windows XP			

Artefact	Depuis	Emplacement	Format	Converti
`powershell_history`	Windows 7	`Users\<user>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`	texte UTF-8	oui
	Écrit par PSReadLine : par défaut depuis Windows 10 ; sous Windows 7, 8 et 8.1 une fois PowerShell 3 ou plus et PSReadLine installés. Vista et XP s'arrêtent à PowerShell 2, sur lequel PSReadLine ne fonctionne pas.			
`powershell_profiles`	Windows XP	`Windows\System32\WindowsPowerShell\v1.0\*profile.ps1, Windows\SysWOW64\WindowsPowerShell\v1.0, Users\<user>\Documents\WindowsPowerShell`	script PowerShell	oui
	Windows 7	`…et Program Files\PowerShell\<version>, Users\<user>\Documents\PowerShell`	script PowerShell	oui
	Windows XP et Vista : seulement une fois PowerShell installé, absent par défaut. Les documents d'un utilisateur aussi là où User Shell Folders les déplace.			
`prefetch`	Windows XP	`Windows\Prefetch\*.pf`	SCCA version 17	oui
	Windows Vista	`Windows\Prefetch\*.pf`	SCCA version 23	oui
	Windows 8	`Windows\Prefetch\*.pf`	SCCA version 26	oui
	Windows 10	`Windows\Prefetch\*.pf`	SCCA version 30 et 31, compressé (MAM)	oui
	Désactivé par défaut sur les éditions serveur.			
`processes`	Windows XP	`en direct : processus du système en marche`	observation	oui
`program_usage`	Windows XP	`NTUSER.DAT …\AppCompatFlags\Layers`	valeurs de registre	oui
	Windows Vista	`…et AppCompatFlags\Compatibility Assistant\Store, Explorer\Taskband`	valeurs de registre	oui
	Windows 10	`…et Explorer\FeatureUsage, Search\JumplistData`	valeurs de registre	oui
`rdp_client`	Windows XP	`NTUSER.DAT\Software\Microsoft\Terminal Server Client; …\Terminal Server Client\Cache\bcache*.bmc`	valeurs de registre ; cache d'images BMC	oui
	Windows 7	`…et Cache\Cache????.bin`	cache d'images BIN	oui
	bcache22.bmc décodé (16 bits par pixel, mesuré sous XP SP1) ; un autre .bmc est compté, ses tuiles non exportées.			

Artefact	Depuis	Emplacement	Format	Converti
`rdp_server`	Windows Vista	`Security.evtx, journaux TerminalServices-Local SessionManager et Remote ConnectionManager`	événements EVTX	oui
	Windows XP et Server 2003 : événements 528 et 682 de Security.evt, non convertis.			
`recent_documents`	Windows XP	`<profile>\Recent\*.lnk`	raccourci	oui
	Windows Vista	`Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\*.lnk`	raccourci	oui
`recycle_bin`	Windows XP	`RECYCLER\<SID>\INFO2`	enregistrements INFO2, version 5	oui
	Windows Vista	`$Recycle.Bin\<SID>\$I*`	$I version 1	oui
	Windows 10	`$Recycle.Bin\<SID>\$I*`	$I version 2	oui
`registry_backups`	Windows XP	`Windows\repair`	ruches	non
	Windows Vista	`Windows\System32\config\RegBack`	ruches, copie brute	non
	Vide depuis Windows 10 1803, sauf si la tâche de sauvegarde a été réactivée. Copié brut : non converti.			
`root_certificates`	Windows XP	`SOFTWARE et NTUSER.DAT …\SystemCertificates\Root, AuthRoot`	certificats (blobs)	oui
	toujours présent depuis Windows XP			
`run_keys`	Windows XP	`SOFTWARE et NTUSER.DAT …\CurrentVersion\Run, RunOnce`	valeurs de registre	oui
	toujours présent depuis Windows XP			
`scheduled_tasks`	Windows XP	`Windows\Tasks\*.job`	binaire JOB	oui
	Windows Vista	`Windows\System32\Tasks, SOFTWARE …\Schedule\TaskCache`	XML, valeurs de registre	oui
	toujours présent depuis Windows Vista			
`search_index`	Windows Vista	`ProgramData\Microsoft\Search\Data\Applications\Windows\Windows.edb`	ESE	oui
	Windows 11	`…\Windows\Windows.db, Windows-gather.db`	SQLite	oui

Artefact	Depuis	Emplacement	Format	Converti
	Windows.edb converti, avec son texte compressé sur 7 bits (mesuré sous Windows 8.1), et la disposition de Windows 7 : ses propres noms de table et de colonnes, ses nombres gros-boutistes, son texte brouillé (mesurés) ; une valeur compressée autrement (XPRESS) est signalée, non décodée. Windows XP : Windows Desktop Search, une installation facultative.			
`security`	Windows XP	`ruche SECURITY : Policy\Secrets, Cache, PolAdtEv`	valeurs de registre	oui
	toujours présent depuis Windows XP			
`security_descriptors`	Windows XP	`$Secure:$SDS`	flux des descripteurs de sécurité NTFS	oui
`services`	Windows XP	`SYSTEM\CurrentControlSet\Services`	clés de registre	oui
	toujours présent depuis Windows XP			
`sessions`	Windows XP	`en direct : sessions ouvertes sur le système en marche`	observation	oui
`setupapi`	Windows XP	`Windows\setupapi.log`	texte, disposition de XP	oui
	Windows Vista	`Windows\INF\setupapi.dev.log`	texte, sections	oui
	toujours présent depuis Windows XP			
`shadow_copies`	Windows Vista	`System Volume Information\{3808876b-c176-4e48-b7ae-04046e6cc752}, {GUID}{3808876b-…}`	catalogue et magasins VSS, blocs de 16 Kio	oui
	Windows XP ne garde aucun cliché persistant : ses points de restauration sont des dossiers à part. Server 2003 ne catalogue aucun magasin (pas d'entrée de type 3) : ses clichés sont listés, pas lus.			
`shellbags`	Windows XP	`NTUSER.DAT\Software\Microsoft\Windows\Shell et ShellNoRoam`	éléments de shell de BagMRU	oui
	Windows Vista	`UsrClass.dat\Local Settings\Software\Microsoft\Windows\Shell, et NTUSER.DAT`	éléments de shell de BagMRU	oui
`shimcache`	Windows XP	`SYSTEM\…\Control\Session Manager\AppCompatibility\AppCompatCache`	en-tête 0xDEADBEEF, 96 entrées	oui
	Windows XP	`SYSTEM\…\Control\Session Manager\AppCompatCache\AppCompatCache`	en-tête 0xBADC0FFE, taille du fichier	oui

Artefact	Depuis	Emplacement	Format	Converti
	Windows Vista	`la même valeur`	en-tête 0xBADC0FFE, entrées de 24 ou 32 octets	oui
	Windows 7	`la même valeur`	en-tête 0xBADC0FEE	oui
	Windows 8	`la même valeur`	en-tête 0x80, entrées ”00ts”	oui
	Windows 8.1	`la même valeur`	en-tête de 0x80 octets commençant par 0, entrées ”10ts”	oui
	Windows 10	`la même valeur`	en-tête 0x30 ou 0x34, entrées ”10ts”	oui
	toujours présent depuis Windows XP ; Windows 8 et 8.1 ont un en-tête de 0x80 octets : ils se distinguent par la marque de leur première entrée, qui tient ici lieu de signature. Windows 8.1 mesuré sur le build 9600. Server 2003 et Vista partagent leur signature : la version choisit la forme ; Vista mesuré sur SP2 x86, Server 2003 sur le build 3790 x86.			
`shortcuts`	Windows XP	`menu Démarrer, bureaux, Lancement rapide de chaque profil et de All Users`	raccourci	oui
`srum`	Windows 8	`Windows\System32\sru\SRUDB.dat`	ESE	oui
	toujours présent depuis Windows 8			
`startup_folders`	Windows XP	`Start Menu\Programs\Startup de chaque profil et de All Users`	fichiers	oui
`sysmon`	Windows 7	`SYSTEM\CurrentControlSet\Services\<pilote de Sysmon>\Parameters, son Archive Directory`	clés du registre ; Rules telles que stockées	oui
	Absent de Windows : présent seulement là où Sysmon a été installé. Le format binaire de ses règles n’est pas documenté : donné tel que stocké, avec son empreinte.			
`thumbcache`	Windows XP	`Thumbs.db dans chaque dossier`	fichier composé OLE	oui
	Windows Vista	`Users\<user>\AppData\Local\Microsoft\Windows\Explorer\thumbcache_*.db`	CMMM, disposition de Vista	oui
	Windows 7	`le même dossier`	CMMM, disposition de Windows 7	oui
	Windows 8	`le même dossier, avec iconcache_*.db`	CMMM, disposition de Windows 8 et suivants	oui

Artefact	Depuis	Emplacement	Format	Converti
typed_paths	Windows Vista	NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\TypedPaths	valeurs de registre	oui
usbstor	Windows XP	SYSTEM\CurrentControlSet\Enum\USBSTOR	clés de registre	oui
	Windows 8	…avec les dates des propriétés du périphérique (première installation, dernière connexion, dernier retrait)	clés de registre, FILETIME	oui
userassist	Windows XP	NTUSER.DAT …\Explorer\UserAssist\{5E6AB780-…}, {75048700-…}	noms en ROT13, valeurs de 16 octets	oui
	Windows 7	NTUSER.DAT …\Explorer\UserAssist\{CEBFF5CD-…}, {F4E57C4B-…}	noms en ROT13, valeurs de 72 octets	oui
users	Windows XP	SAM\SAM\Domains\Account\Users	enregistrements F et V, condensats sous RC4 (clé de démarrage)	oui
	Windows 10	les mêmes clés	enregistrements F et V, condensats sous AES	oui
	toujours présent depuis Windows XP			
user_shell_folders	Windows XP	NTUSER.DAT …\Explorer\User Shell Folders	valeurs de registre	oui
usn_journal	Windows XP	$Extend\$UsnJrnl:$J	USN_RECORD_V2	oui
	Windows 8	le même flux	USN_RECORD_V2, V3 (références de 128 bits)	oui
	Windows 8.1	le même flux	USN_RECORD_V2, V3, V4 (suivi des plages)	oui
	Le journal n'existe que là où il a été créé : toujours sur le volume système à partir de Vista.			
web_client	Windows XP	Internet Explorer 5 à 9 : index.dat de l'historique, du cache, des cookies, des téléchargements	MSIE Cache File 5.2	oui
	Windows 8	Internet Explorer 10 et 11, Edge Legacy : …\WebCache\WebCacheV01.dat	ESE	oui

Artefact	Depuis	Emplacement	Format	Converti
	Windows 10	…et navigateurs Chromium, Firefox : History, places.sqlite, fichiers de session	SQLite, SNSS, mozLz4	oui
	Chromium et Firefox dépendent du navigateur installé, non de Windows.			
web_server	Windows XP	IIS 5.1 et 6 : Windows\System32\inetsrv\MetaBase.xml, journaux W3SVC<n>	métabase d'IIS en XML	non
	Windows Vista	IIS 7 et suivants : Windows\System32\inetsrv\config\applicationHost.config, inetpub\logs\LogFiles, web.config et bases SQLite de chaque site	XML ; texte W3C, UTC ; SQLite	oui
	Présent seulement là où un serveur web est installé. Aussi Apache et nginx, trouvés par leur service ou leur dossier, et les dossiers de données de MySQL, MariaDB et PostgreSQL, pris bruts.			
wmi	Windows XP	Windows\System32\wbem\Repository\FS	dépôt CIM de Windows XP (INDEX.BTR, OBJECTS.DATA, MAPPING, dans FS)	oui
	Windows Vista	Windows\System32\wbem\Repository	dépôt CIM (INDEX.BTR, OBJECTS.DATA, MAPPING)	oui
	toujours présent depuis Windows XP ; Windows XP range le dépôt dans un sous-dossier FS, avec des condensats MD5 et des entrées de page de 4 octets (mesuré, SP1 et SP3).			
word_wheel_query	Windows 7	NTUSER.DAT …\Explorer\WordWheelQuery	MRUListEx et chaînes UTF-16	oui
	Windows XP : Search Assistant\ACMru, non collecté.			
zone_identifiers	Windows XP	<fichier>:Zone.Identifier (XP SP2)	flux de données alterné, texte INI	oui

Une ligne par forme de chaque artefact : le premier Windows qui la porte, son emplacement, son format, et si WAC convertit cette forme (une forme non convertie est au plus collectée brute).

7.7 Couverture de MITRE ATT&CK

ATT&CK décrit, technique par technique, comment la détecter : des analytiques, chacune nommant les sources dont elle a besoin — un composant de données (« Process Creation ») et la source de journal qui le porte (« WinEventLog:Sysmon EventCode=1 »). C'est une liste tenue hors de WAC, contre laquelle sa couverture se mesure. Les tableaux ci-dessous en

sont produits : d'abord ce que WAC fait de chaque source de journal, puis les artefacts qui gardent sur le disque la trace de chaque composant de données hors des journaux, enfin, pour chaque technique Windows, si un journal que WAC prend ou un artefact du disque en porte une trace, et les sources qu'il ne prend pas, avec leur raison. Une trace portée n'est pas une détection : la rechercher dans les fichiers JSON relève de l'analyse.

Source de journal	WAC
`wineventlog:sysmon…`	`events`, `sysmon`
`wineventlog:iis…`	rôle serveur (IIS, AD FS, contrôleur de domaine) : version 4.0
`wineventlog:microsoft-iis…`	rôle serveur (IIS, AD FS, contrôleur de domaine) : version 4.0
`wineventlog:adfs…`	rôle serveur (IIS, AD FS, contrôleur de domaine) : version 4.0
`wineventlog:…`	`events`
`autoruns:registryscan…`	`persistence`, `run_keys`, `services`, `scheduled_tasks`, `startup_folders`
`windows firewall log…`	`firewall_log`
`applicationlog:intune…`	`intune_logs`, `events`
`applicationlog:iis…`	rôle serveur (IIS, AD FS, contrôleur de domaine) : version 4.0
`applicationlog:callrecords…`	service en nuage : dans les journaux du locataire ; le côté de la machine est pris (cloud_identity, onedrive, intune_logs)
`etw…`	télémétrie vivante (ETW, appels d'API, compteurs) : lue par un consommateur à l'instant ; les fichiers de trace que Windows écrit sur son disque sont pris bruts (etw_logs)
`windows:perfmon…`	télémétrie vivante (ETW, appels d'API, compteurs) : lue par un consommateur à l'instant ; les fichiers de trace que Windows écrit sur son disque sont pris bruts (etw_logs)
`windows:osquery…`	télémétrie vivante (ETW, appels d'API, compteurs) : lue par un consommateur à l'instant ; les fichiers de trace que Windows écrit sur son disque sont pris bruts (etw_logs)
`edr…`	télémétrie propre à un EDR : dans sa console ; le dossier local de Defender for Endpoint est pris brut (defender_endpoint)
`m365defender…`	télémétrie propre à un EDR : dans sa console ; le dossier local de Defender for Endpoint est pris brut (defender_endpoint)
`m365:defender…`	télémétrie propre à un EDR : dans sa console ; le dossier local de Defender for Endpoint est pris brut (defender_endpoint)
`m365:…`	service en nuage : dans les journaux du locataire ; le côté de la machine est pris (cloud_identity, onedrive, intune_logs)

Source de journal	WAC
`azure:…`	service en nuage : dans les journaux du locataire ; le côté de la machine est pris (cloud_identity, onedrive, intune_logs)
`aws:…`	service en nuage : dans les journaux du locataire ; le côté de la machine est pris (cloud_identity, onedrive, intune_logs)
`nsm:…`	capture réseau : sur le réseau, pas sur la machine
`network traffic…`	capture réseau : sur le réseau, pas sur la machine
`dns:query…`	capture réseau : sur le réseau, pas sur la machine
`firmware:…`	micrologiciel : non lu, WAC lit le disque

Composant de données	Sur le disque, hors des journaux
Process Creation	`prefetch`, `amcache`, `shimcache`, `bam`, `userassist`, `muicache`, `srum`, `program_usage`
Process Access	—
Process Modification	—
Process Metadata	`processes`
Process Termination	—
Command Execution	`powershell_history`, `prefetch`
Script Execution	`powershell_history`, `powershell_profiles`
OS API Execution	—
Module Load	—
File Creation	`mft`, `usn_journal`, `zone_identifiers`
File Modification	`mft`, `usn_journal`, `log_file`
File Deletion	`usn_journal`, `deleted_files`, `recycle_bin`
File Access	`jump_lists`, `recent_documents`, `shortcuts`, `shellbags`, `explorer_recent_docs`
File Metadata	`mft`, `security_descriptors`, `zone_identifiers`
Network Connection Creation	`srum`, `network`, `firewall_log`, `rdp_client`, `bits`, `web_client`
Network Traffic Content	—
Network Traffic Flow	`srum`, `firewall_log`
Network Share Access	`mount_points`, `network_shares`, `shellbags`
Windows Registry Key Modification	`run_keys`, `persistence`, `services`, `registry_backups`, `shadow_copies`
Windows Registry Key Creation	`run_keys`, `persistence`, `services`, `registry_backups`, `shadow_copies`
Windows Registry Key Access	—
Service Creation	`services`
Service Modification	`services`
Service Metadata	`services`
Driver Load	`services`

Composant de données	Sur le disque, hors des journaux
Scheduled Job Creation	`scheduled_tasks`
Scheduled Job Modification	`scheduled_tasks`
Scheduled Job Metadata	`scheduled_tasks`
WMI Creation	`wmi`
Logon Session Creation	`users`, `rdp_server`
Logon Session Metadata	`users`, `sessions`, `rdp_server`
User Account Authentication	`users`
User Account Creation	`users`
User Account Modification	`users`, `local_groups`
User Account Deletion	—
User Account Metadata	`users`, `local_groups`, `security`
Group Enumeration	—
Firewall Rule Modification	`firewall`
Drive Creation	`usbstor`, `mounted_devices`, `devices`, `setupapi`, `mount_points`
Drive Access	`mount_points`, `shellbags`, `fat_volumes`
Drive Modification	`ntfs_volumes`, `boot_records`
Volume Creation	`mounted_devices`, `ntfs_volumes`, `shadow_copies`
Host Status	—
Named Pipe Metadata	—
Application Log Content	`defender`, `setupapi`, `error_reports`, `intune_logs`, `onedrive`, `web_server`
Web Credential Creation	`web_client`, `dpapi_master_keys`, `cloud_identity`
Firmware Modification	—
Active Directory Object Modification	—
Active Directory Object Access	—
Active Directory Object Creation	—
Active Directory Object Deletion	—
Active Directory Credential Request	—
Cloud Service Metadata	`cloud_identity`
Cloud Storage Access	`onedrive`
Instance Creation	—
Instance Start	—
Domain Registration	—

ID	Technique	Journaux	Disque	Sources non prises
T1001	Data Obfuscation	—	—	NSM
T1001.001	Junk Data	oui	oui	NSM
T1001.002	Steganography	oui	oui	NSM
T1001.003	Protocol or Service Impersonation	oui	oui	NSM
T1003	OS Credential Dumping	oui	oui	
T1003.001	LSASS Memory	oui	oui	
T1003.002	Security Account Manager	oui	oui	
T1003.003	NTDS	oui	oui	
T1003.004	LSA Secrets	oui	oui	
T1003.005	Cached Domain Credentials	oui	oui	
T1003.006	DCSync	oui	—	NSM
T1005	Data from Local System	oui	oui	
T1006	Direct Volume Access	oui	oui	
T1007	System Service Discovery	oui	oui	
T1008	Fallback Channels	oui	oui	NSM
T1010	Application Window Discovery	oui	oui	
T1011	Exfiltration Over Other Network Medium	oui	oui	
T1011.001	Exfiltration Over Bluetooth	oui	oui	
T1012	Query Registry	oui	oui	
T1014	Rootkit	oui	oui	
T1016	System Network Configuration Discovery	oui	oui	
T1016.001	Internet Connection Discovery	oui	oui	
T1016.002	Wi-Fi Discovery	oui	oui	
T1018	Remote System Discovery	oui	oui	
T1020	Automated Exfiltration	oui	oui	
T1021	Remote Services	oui	oui	
T1021.001	Remote Desktop Protocol	oui	oui	
T1021.002	SMB/Windows Admin Shares	oui	oui	
T1021.003	Distributed Component Object Model	oui	oui	
T1021.004	SSH*	oui	—	
T1021.005	VNC	oui	oui	NSM
T1021.006	Windows Remote Management	oui	oui	NSM
T1025	Data from Removable Media	oui	oui	
T1027	Obfuscated Files or Information	oui	oui	
T1027.001	Binary Padding	oui	oui	
T1027.002	Software Packing	oui	oui	
T1027.003	Steganography	oui	oui	
T1027.004	Compile After Delivery	oui	oui	
T1027.005	Indicator Removal from Tools	oui	oui	

ID	Technique	Journaux	Disque	Sources non prises
T1027.006	HTML Smuggling	oui	oui	EDR, NSM
T1027.007	Dynamic API Resolution	oui	oui	ETW
T1027.008	Stripped Payloads	oui	oui	EDR
T1027.009	Embedded Payloads	oui	oui	EDR
T1027.010	Command Obfuscation	oui	oui	
T1027.011	Fileless Storage	oui	oui	
T1027.012	LNK Icon Smuggling	oui	oui	
T1027.013	Encrypted/Encoded File	oui	oui	
T1027.014	Polymorphic Code	oui	oui	
T1027.015	Compression	oui	oui	
T1027.016	Junk Code Insertion	oui	oui	
T1027.017	SVG Smuggling	oui	oui	
T1027.018	Invisible Unicode	oui	oui	
T1029	Scheduled Transfer	oui	oui	
T1030	Data Transfer Size Limits	oui	oui	NSM
T1033	System Owner/User Discovery	oui	oui	
T1036	Masquerading	oui	oui	
T1036.001	Invalid Code Signature	oui	oui	
T1036.002	Right-to-Left Override	oui	oui	
T1036.003	Rename Legitimate Utilities	oui	oui	EDR
T1036.004	Masquerade Task or Service	oui	oui	
T1036.005	Match Legitimate Resource Name or Location	oui	oui	
T1036.007	Double File Extension	oui	oui	
T1036.008	Masquerade File Type	oui	oui	
T1036.010	Masquerade Account Name	oui	oui	ETW
T1036.012	Browser Fingerprint	oui	oui	ETW, NSM
T1037	Boot or Logon Initialization Scripts	oui	oui	
T1037.001	Logon Script (Windows)	oui	oui	
T1037.003	Network Logon Script	oui	oui	
T1039	Data from Network Shared Drive	oui	oui	
T1040	Network Sniffing	oui	oui	
T1041	Exfiltration Over C2 Channel	oui	oui	NSM
T1046	Network Service Discovery	oui	oui	
T1047	Windows Management Instrumentation	oui	oui	
T1048	Exfiltration Over Alternative Protocol	oui	oui	
T1048.001	Exfiltration Over Symmetric Encrypted Non-C2 Protocol	oui	oui	
T1048.002	Exfiltration Over Asymmetric Encrypted Non-C2 Protocol	oui	oui	NSM

ID	Technique	Journaux	Disque	Sources non prises
T1048.003	Exfiltration Over Unencrypted Non-C2 Protocol	oui	oui	NSM
T1049	System Network Connections Discovery	oui	oui	
T1052	Exfiltration Over Physical Medium	oui	oui	
T1052.001	Exfiltration over USB	oui	oui	
T1053	Scheduled Task/Job	oui	oui	
T1053.002	At	oui	oui	
T1053.005	Scheduled Task	oui	oui	
T1055	Process Injection	oui	oui	ETW
T1055.001	Dynamic-link Library Injection	oui	oui	
T1055.002	Portable Executable Injection	oui	oui	
T1055.003	Thread Execution Hijacking	oui	oui	ETW
T1055.004	Asynchronous Procedure Call	oui	oui	ETW
T1055.005	Thread Local Storage	oui	—	EDR
T1055.011	Extra Window Memory Injection	oui	oui	ETW
T1055.012	Process Hollowing	oui	oui	ETW
T1055.013	Process Doppelgänging	oui	oui	ETW
T1055.015	ListPlanting	oui	oui	ETW
T1056	Input Capture	oui	oui	
T1056.001	Keylogging	oui	oui	
T1056.002	GUI Input Capture	oui	oui	
T1056.003	Web Portal Capture	oui	oui	4.0
T1056.004	Credential API Hooking	oui	—	
T1057	Process Discovery	oui	oui	
T1059	Command and Scripting Interpreter	oui	oui	
T1059.001	PowerShell	oui	oui	
T1059.003	Windows Command Shell	oui	oui	EDR
T1059.005	Visual Basic	oui	oui	
T1059.006	Python	oui	oui	EDR
T1059.007	JavaScript	oui	oui	EDR
T1059.010	AutoHotKey & AutoIT	oui	oui	
T1059.011	Lua	oui	oui	
T1068	Exploitation for Privilege Escalation	oui	oui	
T1069	Permission Groups Discovery	oui	oui	
T1069.001	Local Groups	oui	oui	
T1069.002	Domain Groups	oui	oui	
T1070	Indicator Removal	oui	oui	
T1070.003	Clear Command History	oui	oui	
T1070.004	File Deletion	oui	oui	
T1070.005	Network Share Connection Removal	oui	oui	NSM

ID	Technique	Journaux	Disque	Sources non prises
T1070.006	Timestomp	oui	oui	EDR
T1070.007	Clear Network Connection History and Configurations	oui	oui	EDR
T1070.008	Clear Mailbox Data	oui	oui	cloud
T1070.009	Clear Persistence	oui	oui	
T1070.010	Relocate Malware	oui	oui	
T1071	Application Layer Protocol	oui	oui	NSM
T1071.001	Web Protocols	oui	oui	NSM
T1071.002	File Transfer Protocols	oui	oui	NSM
T1071.003	Mail Protocols	oui	oui	NSM
T1071.004	DNS	oui	oui	NSM
T1071.005	Publish/Subscribe Protocols	oui	oui	NSM
T1072	Software Deployment Tools	oui	oui	
T1074	Data Staged	oui	oui	
T1074.001	Local Data Staging	oui	oui	
T1074.002	Remote Data Staging	oui	oui	
T1078	Valid Accounts	oui	oui	
T1078.001	Default Accounts	oui	oui	
T1078.002	Domain Accounts	oui	oui	
T1078.003	Local Accounts	oui	oui	
T1080	Taint Shared Content	oui	oui	
T1082	System Information Discovery	oui	oui	
T1083	File and Directory Discovery	oui	oui	
T1087	Account Discovery	oui	oui	
T1087.001	Local Account	oui	oui	
T1087.002	Domain Account	oui	oui	NSM
T1087.003	Email Account	oui	oui	
T1090	Proxy	oui	oui	NSM
T1090.001	Internal Proxy	oui	oui	
T1090.002	External Proxy	oui	oui	
T1090.003	Multi-hop Proxy	oui	oui	NSM
T1090.004	Domain Fronting	oui	oui	NSM
T1091	Replication Through Removable Media	oui	oui	
T1092	Communication Through Removable Media	oui	oui	
T1095	Non-Application Layer Protocol	oui	oui	NSM
T1098	Account Manipulation	oui	oui	
T1098.002	Additional Email Delegate Permissions	oui	oui	cloud
T1098.005	Device Registration	oui	oui	

ID	Technique	Journaux	Disque	Sources non prises
T1098.007	Additional Local or Domain Groups	oui	oui	
T1102	Web Service	oui	oui	NSM
T1102.001	Dead Drop Resolver	oui	oui	ETW
T1102.002	Bidirectional Communication	oui	oui	ETW
T1102.003	One-Way Communication	oui	oui	ETW
T1104	Multi-Stage Channels	oui	oui	
T1105	Ingress Tool Transfer	oui	oui	
T1106	Native API	oui	oui	
T1110	Brute Force	oui	oui	
T1110.001	Password Guessing	oui	oui	
T1110.002	Password Cracking	oui	oui	
T1110.003	Password Spraying	oui	oui	
T1110.004	Credential Stuffing	oui	oui	
T1111	Multi-Factor Authentication Interception	oui	oui	
T1112	Modify Registry	oui	oui	
T1113	Screen Capture	oui	oui	
T1114	Email Collection	oui	oui	
T1114.001	Local Email Collection	oui	oui	
T1114.002	Remote Email Collection	oui	oui	cloud
T1114.003	Email Forwarding Rule	oui	oui	cloud
T1115	Clipboard Data	oui	oui	
T1119	Automated Collection	oui	oui	
T1120	Peripheral Device Discovery	oui	oui	
T1123	Audio Capture	oui	oui	
T1124	System Time Discovery	oui	oui	EDR, ETW
T1125	Video Capture	oui	oui	
T1127	Trusted Developer Utilities Proxy Execution	oui	oui	
T1127.001	MSBuild	oui	oui	EDR
T1127.002	ClickOnce	oui	oui	
T1127.003	JamPlus	oui	oui	
T1129	Shared Modules	oui	oui	
T1132	Data Encoding	oui	oui	NSM
T1132.001	Standard Encoding	oui	oui	EDR
T1132.002	Non-Standard Encoding	oui	oui	EDR
T1133	External Remote Services	oui	oui	
T1134	Access Token Manipulation	oui	oui	ETW
T1134.001	Token Impersonation/Theft	oui	oui	ETW
T1134.002	Create Process with Token	oui	oui	ETW
T1134.003	Make and Impersonate Token	oui	oui	ETW

ID	Technique	Journaux	Disque	Sources non prises
T1134.004	Parent PID Spoofing	oui	oui	ETW
T1134.005	SID-History Injection	oui	oui	ETW
T1135	Network Share Discovery	oui	oui	ETW
T1136	Create Account	oui	oui	
T1136.001	Local Account	oui	oui	
T1136.002	Domain Account	oui	oui	
T1137	Office Application Startup	oui	oui	
T1137.001	Office Template Macros	oui	oui	
T1137.002	Office Test	oui	oui	
T1137.003	Outlook Forms	oui	oui	
T1137.004	Outlook Home Page	oui	oui	
T1137.005	Outlook Rules	oui	oui	
T1137.006	Add-ins	oui	oui	
T1140	Deobfuscate/Decode Files or Information	oui	oui	
T1176	Software Extensions	oui	oui	
T1176.001	Browser Extensions	oui	oui	
T1176.002	IDE Extensions	oui	oui	
T1185	Browser Session Hijacking	oui	oui	
T1187	Forced Authentication	oui	oui	NSM
T1189	Drive-by Compromise	oui	oui	ETW, NSM
T1190	Exploit Public-Facing Application	oui	oui	4.0
T1195	Supply Chain Compromise	oui	oui	NSM
T1195.001	Compromise Software Dependencies and Development Tools	oui	oui	NSM
T1195.002	Compromise Software Supply Chain	oui	oui	NSM
T1195.003	Compromise Hardware Supply Chain	oui	oui	
T1197	BITS Jobs	oui	oui	
T1199	Trusted Relationship	oui	oui	
T1200	Hardware Additions	oui	oui	
T1201	Password Policy Discovery	oui	oui	
T1202	Indirect Command Execution	oui	oui	
T1203	Exploitation for Client Execution	oui	oui	
T1204	User Execution	oui	oui	
T1204.001	Malicious Link	oui	oui	NSM
T1204.002	Malicious File	oui	oui	
T1204.004	Malicious Copy and Paste	oui	oui	NSM
T1204.005	Malicious Library	oui	oui	
T1205	Traffic Signaling	oui	oui	
T1205.001	Port Knocking	oui	oui	
T1205.002	Socket Filters	oui	oui	NSM

ID	Technique	Journaux	Disque	Sources non prises
T1207	Rogue Domain Controller	oui	—	cloud, NSM
T1210	Exploitation of Remote Services	oui	oui	NSM
T1211	Exploitation for Stealth	oui	oui	
T1212	Exploitation for Credential Access	oui	oui	
T1213	Data from Information Repositories	oui	oui	cloud
T1213.002	Sharepoint	—	oui	cloud
T1213.006	Databases	oui	oui	
T1216	System Script Proxy Execution	oui	oui	
T1216.001	PubPrn	oui	oui	
T1216.002	SyncAppvPublishingServer	oui	oui	
T1217	Browser Information Discovery	oui	oui	
T1218	System Binary Proxy Execution	oui	oui	
T1218.001	Compiled HTML File	oui	oui	
T1218.002	Control Panel	oui	oui	
T1218.003	CMSTP	oui	oui	
T1218.004	InstallUtil	oui	oui	
T1218.005	Mshta	oui	oui	
T1218.007	Msiexec	oui	oui	
T1218.008	Odbcconf	oui	oui	
T1218.009	Regsvcs/Regasm	oui	oui	
T1218.010	Regsvr32	oui	oui	
T1218.011	Rundll32	oui	oui	
T1218.012	Verclsid	oui	oui	
T1218.013	Mavinject	oui	oui	
T1218.014	MMC	oui	oui	
T1218.015	Electron Applications	oui	oui	
T1219	Remote Access Tools	oui	oui	
T1219.001	IDE Tunneling	oui	oui	NSM
T1219.002	Remote Desktop Software	oui	oui	
T1219.003	Remote Access Hardware	oui	oui	
T1220	XSL Script Processing	oui	oui	
T1221	Template Injection	oui	oui	
T1222	File and Directory Permissions Modification	oui	oui	
T1222.001	Windows Permissions	oui	oui	
T1480	Execution Guardrails	oui	oui	
T1480.001	Environmental Keying	oui	oui	
T1480.002	Mutual Exclusion	oui	oui	
T1482	Domain Trust Discovery	oui	oui	
T1484	Domain or Tenant Policy Modification	oui	oui	
T1484.001	Group Policy Modification	oui	oui	

ID	Technique	Journaux	Disque	Sources non prises
T1484.002	Trust Modification	oui	oui	
T1485	Data Destruction	oui	oui	
T1486	Data Encrypted for Impact	oui	oui	
T1489	Service Stop	oui	oui	
T1490	Inhibit System Recovery	oui	oui	
T1491	Defacement	oui	oui	
T1491.001	Internal Defacement	oui	oui	
T1491.002	External Defacement	oui	oui	NSM
T1495	Firmware Corruption	oui	oui	
T1496	Resource Hijacking	oui	oui	ETW
T1496.001	Compute Hijacking	oui	oui	
T1496.002	Bandwidth Hijacking	oui	oui	
T1497	Virtualization/Sandbox Evasion	oui	oui	
T1497.001	System Checks	oui	oui	
T1497.002	User Activity Based Checks	oui	oui	
T1497.003	Time Based Checks	oui	oui	
T1498	Network Denial of Service	oui	oui	
T1498.001	Direct Network Flood	oui	oui	
T1498.002	Reflection Amplification	oui	oui	ETW
T1499	Endpoint Denial of Service	oui	oui	
T1499.001	OS Exhaustion Flood	oui	oui	NSM
T1499.002	Service Exhaustion Flood	oui	oui	ETW
T1499.003	Application Exhaustion Flood	oui	oui	ETW
T1499.004	Application or System Exploitation	oui	oui	
T1505	Server Software Component	oui	oui	
T1505.001	SQL Stored Procedures	oui	oui	
T1505.002	Transport Agent	oui	oui	
T1505.003	Web Shell	oui	oui	NSM
T1505.004	IIS Components	oui	oui	4.0
T1505.005	Terminal Services DLL	oui	oui	
T1518	Software Discovery	oui	oui	
T1518.001	Security Software Discovery	oui	oui	
T1518.002	Backup Software Discovery	oui	oui	
T1529	System Shutdown/Reboot	oui	oui	
T1531	Account Access Removal	oui	oui	
T1534	Internal Spearphishing	oui	oui	cloud
T1539	Steal Web Session Cookie	oui	oui	
T1542	Pre-OS Boot	oui	oui	
T1542.001	System Firmware	oui	oui	
T1542.002	Component Firmware	oui	oui	firmware

ID	Technique	Journaux	Disque	Sources non prises
T1542.003	Bootkit	oui	oui	
T1543	Create or Modify System Process	oui	oui	
T1543.003	Windows Service	oui	oui	
T1546	Event Triggered Execution	oui	oui	
T1546.001	Change Default File Association	oui	oui	
T1546.002	Screensaver	oui	oui	
T1546.003	Windows Management Instrumentation Event Subscription	oui	oui	
T1546.007	Netsh Helper DLL	oui	oui	
T1546.008	Accessibility Features	oui	oui	
T1546.009	AppCert DLLs	oui	oui	
T1546.010	AppInit DLLs	oui	oui	
T1546.011	Application Shimming	oui	oui	
T1546.012	Image File Execution Options Injection	oui	oui	
T1546.013	PowerShell Profile	oui	oui	
T1546.015	Component Object Model Hijacking	oui	oui	
T1546.016	Installer Packages	oui	oui	
T1546.018	Python Startup Hooks*	—	oui	
T1547	Boot or Logon Autostart Execution	oui	oui	
T1547.001	Registry Run Keys / Startup Folder	oui	oui	
T1547.002	Authentication Package	oui	oui	
T1547.003	Time Providers	oui	oui	
T1547.004	Winlogon Helper DLL	oui	oui	
T1547.005	Security Support Provider	oui	oui	
T1547.008	LSASS Driver	oui	oui	
T1547.009	Shortcut Modification	oui	oui	
T1547.010	Port Monitors	oui	oui	
T1547.012	Print Processors	oui	oui	
T1547.014	Active Setup	oui	oui	
T1548	Abuse Elevation Control Mechanism	oui	oui	
T1548.002	Bypass User Account Control	oui	oui	
T1550	Use Alternate Authentication Material	oui	oui	
T1550.002	Pass the Hash	oui	oui	
T1550.003	Pass the Ticket	oui	oui	
T1552	Unsecured Credentials	oui	oui	
T1552.001	Credentials In Files	oui	oui	
T1552.002	Credentials in Registry	oui	oui	EDR
T1552.003	Shell History*	—	oui	
T1552.004	Private Keys	oui	oui	
T1552.006	Group Policy Preferences	oui	oui	
T1553	Subvert Trust Controls	oui	oui	

ID	Technique	Journaux	Disque	Sources non prises
T1553.002	Code Signing	oui	oui	
T1553.003	SIP and Trust Provider Hijacking	oui	oui	
T1553.004	Install Root Certificate	oui	oui	
T1553.005	Mark-of-the-Web Bypass	oui	oui	
T1553.006	Code Signing Policy Modification	oui	oui	
T1554	Compromise Host Software Binary	oui	oui	
T1555	Credentials from Password Stores	oui	oui	
T1555.003	Credentials from Web Browsers	oui	oui	
T1555.004	Windows Credential Manager	oui	oui	
T1555.005	Password Managers	oui	oui	
T1556	Modify Authentication Process	oui	oui	
T1556.001	Domain Controller Authentication	oui	oui	
T1556.002	Password Filter DLL	oui	oui	
T1556.005	Reversible Encryption	oui	oui	
T1556.006	Multi-Factor Authentication	oui	oui	
T1556.007	Hybrid Identity	oui	oui	
T1556.008	Network Provider DLL	oui	oui	
T1557	Adversary-in-the-Middle	oui	oui	
T1557.001	Name Resolution Poisoning and SMB Relay	oui	oui	NSM
T1557.002	ARP Cache Poisoning	oui	oui	
T1557.003	DHCP Spoofing	oui	oui	NSM
T1558	Steal or Forge Kerberos Tickets	oui	oui	
T1558.001	Golden Ticket	oui	oui	
T1558.002	Silver Ticket	oui	oui	
T1558.003	Kerberoasting	oui	oui	
T1558.004	AS-REP Roasting	oui	oui	
T1559	Inter-Process Communication	oui	—	
T1559.001	Component Object Model	oui	oui	
T1559.002	Dynamic Data Exchange	oui	oui	
T1560	Archive Collected Data	oui	oui	
T1560.001	Archive via Utility	oui	oui	
T1560.002	Archive via Library	oui	oui	
T1560.003	Archive via Custom Method	oui	oui	
T1561	Disk Wipe	oui	oui	
T1561.001	Disk Content Wipe	oui	oui	
T1561.002	Disk Structure Wipe	oui	oui	
T1563	Remote Service Session Hijacking	oui	oui	
T1563.002	RDP Hijacking	oui	oui	
T1564	Hide Artifacts	oui	oui	
T1564.001	Hidden Files and Directories	oui	oui	

ID	Technique	Journaux	Disque	Sources non prises
T1564.002	Hidden Users	oui	oui	
T1564.003	Hidden Window	oui	oui	
T1564.004	NTFS File Attributes	oui	oui	ETW
T1564.005	Hidden File System	oui	oui	ETW
T1564.006	Run Virtual Instance	oui	oui	
T1564.007	VBA Stomping	oui	oui	
T1564.008	Email Hiding Rules	oui	oui	cloud
T1564.010	Process Argument Spoofing	oui	oui	
T1564.011	Ignore Process Interrupts	oui	oui	
T1564.012	File/Path Exclusions	oui	oui	
T1565	Data Manipulation	oui	oui	
T1565.001	Stored Data Manipulation	oui	oui	
T1565.002	Transmitted Data Manipulation	oui	oui	
T1565.003	Runtime Data Manipulation	oui	oui	
T1566	Phishing	oui	oui	cloud
T1566.001	Spearphishing Attachment	oui	oui	cloud
T1566.002	Spearphishing Link	oui	oui	cloud
T1566.003	Spearphishing via Service	oui	oui	
T1566.004	Spearphishing Voice	—	oui	cloud
T1567	Exfiltration Over Web Service	oui	oui	
T1567.001	Exfiltration to Code Repository	oui	oui	
T1567.002	Exfiltration to Cloud Storage	oui	oui	
T1567.003	Exfiltration to Text Storage Sites	oui	oui	
T1567.004	Exfiltration Over Webhook	oui	oui	
T1568	Dynamic Resolution	oui	oui	
T1568.001	Fast Flux DNS	oui	oui	
T1568.002	Domain Generation Algorithms	oui	oui	
T1568.003	DNS Calculation	oui	oui	
T1569	System Services	oui	oui	
T1569.002	Service Execution	oui	oui	
T1570	Lateral Tool Transfer	oui	oui	
T1571	Non-Standard Port	oui	oui	
T1572	Protocol Tunneling	oui	oui	
T1573	Encrypted Channel	oui	oui	
T1573.001	Symmetric Cryptography	oui	oui	
T1573.002	Asymmetric Cryptography	oui	oui	
T1574	Hijack Execution Flow	oui	oui	
T1574.001	DLL	oui	oui	
T1574.005	Executable Installer File Permissions Weakness	oui	oui	

ID	Technique	Journaux	Disque	Sources non prises
T1574.007	Path Interception by PATH Environment Variable	oui	oui	
T1574.008	Path Interception by Search Order Hijacking	oui	oui	
T1574.009	Path Interception by Unquoted Path	oui	oui	
T1574.010	Services File Permissions Weakness	oui	oui	
T1574.011	Services Registry Permissions Weakness	oui	oui	
T1574.012	COR_PROFILER	oui	oui	
T1574.013	KernelCallbackTable	oui	oui	ETW
T1574.014	AppDomainManager	oui	oui	
T1606	Forge Web Credentials	oui	oui	
T1606.001	Web Cookies	oui	oui	
T1606.002	SAML Tokens	oui	oui	4.0
T1611	Escape to Host	oui	oui	
T1614	System Location Discovery	oui	oui	ETW
T1614.001	System Language Discovery	oui	oui	ETW
T1615	Group Policy Discovery	oui	oui	NSM
T1620	Reflective Code Loading	oui	oui	ETW
T1621	Multi-Factor Authentication Request Generation	oui	oui	
T1622	Debugger Evasion	oui	oui	ETW
T1649	Steal or Forge Authentication Certificates	oui	—	
T1652	Device Driver Discovery	oui	oui	
T1653	Power Settings	oui	oui	
T1654	Log Enumeration	oui	oui	
T1657	Financial Theft	oui	oui	
T1659	Content Injection	oui	oui	NSM
T1665	Hide Infrastructure	oui	oui	NSM
T1667	Email Bombing	oui	oui	cloud
T1668	Exclusive Control	oui	oui	
T1669	Wi-Fi Networks	oui	oui	
T1673	Virtual Machine Discovery	oui	oui	
T1674	Input Injection	oui	oui	
T1678	Delay Execution	oui	oui	
T1679	Selective Exclusion	oui	oui	
T1680	Local Storage Discovery	oui	oui	
T1684	Social Engineering	oui	oui	
T1684.001	Impersonation	oui	oui	cloud
T1684.002	Email Spoofing	—	oui	cloud

ID	Technique	Journaux	Disque	Sources non prises
T1685	Disable or Modify Tools	oui	oui	
T1685.001	Disable or Modify Windows Event Log	oui	oui	
T1685.003	Modify or Spoof Tool UI	oui	oui	
T1685.005	Clear Windows Event Logs	oui	oui	
T1685.007	Impair Autonomous Defenses*	—	—	IA
T1686	Disable or Modify System Firewall	oui	oui	
T1686.003	Windows Host Firewall	oui	oui	
T1687	Exploitation for Defense Impairment	oui	oui	
T1688	Safe Mode Boot	oui	oui	
T1689	Downgrade Attack	oui	oui	
T1690	Prevent Command History Logging	oui	oui	
T1698	Indirect Prompt Injection*	—	—	IA
T1699	Direct Prompt Injection*	—	—	IA
T1700	AI-Enabled Exfiltration	oui	oui	

ATT&CK Enterprise 20.0 (2026-10-13, SHA-256 81113BA98AFA2928…) : 479 techniques et sous-techniques Windows ; 475 avec au moins une trace que WAC prélève, 4 dont toutes les sources sont hors du disque ; 6 (marquées *) n'ont pas d'analytique Windows dans ATT&CK et sont décidées d'après la technique elle-même. *Journaux* : un journal que WAC prend porte une trace de la technique ; *Disque* : un artefact sur le disque en porte une, hors des journaux. Un journal d'événements n'est là que si la machine l'a tenu : WAC relève la configuration des canaux et la politique d'audit, qui le disent.

Sources non prises, et pourquoi : ETW, télémétrie vivante (ETW, appels d'API, compteurs) : lue par un consommateur à l'instant ; les fichiers de trace que Windows écrit sur son disque sont pris bruts (etw_logs) ; NSM, capture réseau : sur le réseau, pas sur la machine ; cloud, service en nuage : dans les journaux du locataire ; le côté de la machine est pris (cloud_identity, onedrive, intune_logs) ; EDR, télémétrie propre à un EDR : dans sa console ; le dossier local de Defender for Endpoint est pris brut (defender_endpoint) ; firmware, micrologiciel : non lu, WAC lit le disque ; 4.0, rôle serveur (IIS, AD FS, contrôleur de domaine) : version 4.0 ; IA, comportement d'un modèle d'IA : ATT&CK ne lui connaît aucune trace sous Windows

7.8 Source attendue mais introuvable

À l'inverse, certaines sources sont sur toute installation d'une version donnée, et Windows continue de fonctionner sans elles : le fichier `hosts`, `Amcache.hve` et `SRUDB.dat` depuis qu'ils existent, les journaux d'événements, la clé des services. Le tableau ci-dessus les signale par « toujours présent depuis ». La règle vaut pour les postes clients, et pour les serveurs que le cycle de test a mesurés : Server 2022 et suivants, avec leur interface graphique ou en installation Server Core ; sur un serveur plus ancien, une source absente reste une simple absence. Quand la collecte ne contient pas une telle source, ce n'est ni une lecture ratée ni une absence d'activité : c'est un constat, que le fichier de l'artefact énonce.

```
{
  "Artifact": "Hosts file",
  "CollectionStatus": "ExpectedButMissing",
  "WindowsRelease": "Windows 11",
  "WindowsBuild": 26200,
  "ExpectedLocation": "Windows\\System32\\drivers\\etc\\hosts",
  "Error": "0x80070002 Le fichier spécifié est introuvable.",
  "Note": "This version of Windows always has this artefact's source
           (artefacts.hosts) and the collection does not hold it. [...]"
}
```


La conversion l'annonce en console et le consigne dans `conversion.json`. Elle sait que la collecte scellée ne contient pas la source ; que la machine ne l'avait pas se lit dans `investigation.json`, où l'extraction du dossier est consignée avec son résultat et le nombre de fichiers pris. Un fichier effacé, un service arrêté ou un système endommagé sont alors à expliquer.

> **Attention**
>
> Le statut n'est posé que là où la présence est certaine. Il ne l'est pas pour ce qui n'existe qu'après un événement (la clé `USBSTOR` avant le premier support USB), pour ce qui dépend d'un programme installé (navigateurs, messagerie), ni sur les serveurs antérieurs à Server 2022, qui restent à mesurer : l'absence y reste un artefact non collecté ou une liste vide.

7.9 Données non décodées

Un élément que WAC ne sait pas décoder (élément du shell inconnu, bloc d'extension non reconnu) est accompagné de ses octets bruts en hexadécimal, pour qu'un analyste puisse le décoder plus tard. Avec `--dump`, la valeur brute des shellbags et des raccourcis est ajoutée systématiquement.

7.10 Format non pris en charge

Le format d'une pièce se lit sur la pièce elle-même — sa signature ou sa version interne —, jamais sur la version de Windows : un cache de miniatures de Vista retrouvé sur un Windows 10 reste un cache de Vista. Quand WAC n'a pas de décodeur pour le format rencontré, qu'il soit ancien ou qu'il n'ait encore jamais été vu, il ne convertit rien et le dit ; la pièce, elle, est dans la consigne, prête pour un autre outil.


```
{
  "SID": "S-1-5-21-1004336348-1177238915-682003330-1000",
  "SIDName": "jean",
  "Cache": "C:\\Users\\jean\\AppData\\Local\\Microsoft\\Windows\\Explorer\\thumbcache_96.
    db",
  "FormatVersion": 20,
  "DecodingStatus": "UnsupportedFormat",
```

```
  "FormatSignature": "0x00000014",
  "FormatName": "CMMM, Vista layout",
  "Note": "Collected raw, not converted: this version of WAC has no
           decoder for this format. The exhibit store holds the source."
}
```

`FormatName` nomme le format quand le tableau de la section 7.6 le connaît (colonne « Converti » à « non »); il est absent pour un format inconnu. Le statut porte sur une pièce (`DecodingStatus`) quand l'artefact en compte plusieurs — fichiers Prefetch, index `$I` de la corbeille, caches de miniatures, jump lists (dont les raccourcis restent décodés, sans leur entrée DestList) — et sur l'artefact entier (`CollectionStatus`) quand sa source est unique, comme la valeur du ShimCache.

Certains formats ne portent pas de signature : c'est alors la version de Windows examinée qui dit la forme de l'artefact. Avant Windows 10 1607, `Amcache.hve` n'a pas ses clés `Inventory`; avant Windows 10 1709, BITS tient ses travaux dans `qmgr0.dat`; avant Windows 11, l'index de recherche est une base ESE. La collecte prend la pièce comme d'habitude, et la conversion l'annonce au lieu de rendre une liste vide :

```
{
  "Artifact": "Amcache applications",
  "CollectionStatus": "UnsupportedFormat",
  "WindowsRelease": "Windows 8.1",
  "WindowsBuild": 9600,
  "FormatName": "registry hive, File and Programs keys",
  "Location": "Windows\\AppCompat\\Programs\\Amcache.hve",
  "Note": "On this version of Windows this artefact (artefacts.amcache) is
           in a format this version of WAC has no decoder for. [...]"
}
```

> **Attention**
> Une pièce « UnsupportedFormat » n'est ni une liste vide ni une lecture ratée : la trace existe, elle est prélevée, et son contenu reste à lire.

7.11 Chemins

Les chemins et valeurs sont restitués **tels qu'ils figurent sur le disque**. Rien n'est transformé : l'échappement n'intervient qu'à l'écriture du JSON.

CHAPITRE 8

Traces laissées sur la machine examinée

À lire avant d'employer WAC sur une affaire. Ce chapitre énumère ce que WAC fait à la machine, opération par opération, y compris ce qu'il ne peut pas éviter.

8.1 Ce que WAC fait encore

Opération	Trace
Lecture directe du volume	Un accès par volume effectivement lu (en général un seul). Aucun fichier ouvert, donc aucune date d'accès modifiée. Si l'audit d'accès aux objets est activé, l'ouverture du volume peut être journalisée (Security 4656/4663).
Écriture des résultats	Sur le support de collecte uniquement. Rien n'est écrit sur le disque examiné.
Rejeu des journaux de ruche	Sur la copie de travail uniquement, avec journal d'annulation.
État des services	Une requête en lecture au gestionnaire de services. Aucun service n'est ouvert ni modifié.
Processus	Un instantané du noyau. Relevé des propriétaires : une requête au service Terminal Services. Liste des modules : chaque processus ouvert en requête et lecture mémoire seules, sans droit d'écriture ni d'injection ; un EDR peut le relever.
Sessions	Une requête en lecture à LSASS.
État réseau	Lectures en mémoire des tables de la pile TCP/IP : aucun paquet émis, aucun nom résolu. `iphlpapi.dll` n'est chargée que si `artefacts.network` est activé.
Heure de démarrage	Une requête en mémoire au noyau.
Journaux d'événements (`--events`)	Lus par le même accès direct au volume : le service de journalisation n'est pas sollicité.
Fichiers cités (`--binary`)	Lus par le même accès direct. Aucun fichier ouvert.

Opération	Trace
Vérification d'authenticité (`--binary`)	Catalogues lus par le même accès direct ; vérification en mémoire. Aucun service sollicité, aucun accès au magasin de certificats, aucun accès réseau.

8.2 Ce que WAC ne fait plus

Les versions antérieures créaient un cliché instantané (VSS), passaient par COM et WMI, interrogeaient le planificateur de tâches et le service de journalisation, ouvraient chaque processus et chaque service, lisaient la liste des profils dans le registre vivant et ouvraient chaque fichier cité pour en calculer l'empreinte. Toutes ces opérations, qui laissaient des entrées dans les journaux ou modifiaient des dates d'accès, ont été supprimées.

8.3 Ce qu'aucun outil ne peut éviter

Ces traces sont listées pour que l'analyste reconnaisse l'empreinte de WAC au lieu de la prendre pour l'activité étudiée :

— **lancer `WAC.exe` est une exécution** : elle produit une entrée Amcache, une entrée ShimCache à l'arrêt et, si WAC est lancé du disque de la machine plutôt que d'une clé, un fichier Prefetch et une valeur BAM. WAC n'écrit lui-même aucun événement, mais une surveillance en place enregistre son exécution : Security 4688 et 4689 si l'audit de création de processus est actif, Sysmon (le processus, ses lectures brutes du volume, les fichiers qu'il crée sur la clé), un EDR. Attendez-vous à retrouver WAC dans les artefacts qu'il collecte ;

— **brancher la clé USB** l'inscrit dans `USBSTOR`, `MountedDevices` et les journaux associés ;

— **ouvrir une session** pour lancer l'outil crée une session et ses entrées de journal ;

— WAC **occupe de la mémoire** pendant la collecte, ce qui peut évincer des pages vers le fichier d'échange. Sa lecture du volume, elle, contourne le cache (lecture non tamponnée) : les gigaoctets lus ne remplacent pas le cache de fichiers de la machine.

CHAPITRE 9

Limites connues

Volumes ni NTFS ni FAT
Un fichier situé sur un CD-ROM n'est pas lu : il est signalé comme non lu.

Variables propres à un utilisateur
Un chemin exprimé avec `%APPDATA%`, `%LOCALAPPDATA%` ou `%USERPROFILE%` (dans une tâche planifiée, par exemple) n'est pas résolu : le compte concerné n'est pas connu à cet endroit, et deviner donnerait l'empreinte d'un autre fichier.

Exécutable supprimé
Un processus dont l'exécutable a été effacé du disque n'a pas d'empreinte : il ne reste que la mémoire, hors du périmètre de WAC.

Messages d'événements
Sur la machine de test, 99,6 % des événements reçoivent un message lisible, identique à celui de Windows pour 98 % des sortes d'événements. Le reste est expliqué dans le dossier de validation : écarts de Windows lui-même par rapport au manifeste, satellites de langue plus anciens que leur binaire. Les données, elles, sont toujours complètes.

Signés mais non authentifiés
Quelques sortes de fichiers réellement signés restent prélevées, dans le sens prudent : les applications du Microsoft Store (signées au niveau du paquet, et non fichier par fichier) et les scripts Windows Script Host signés seulement dans leur texte.

CHAPITRE 10

Dépannage

Message ou constat	Cause et remède
Échec de « Check administrator rights »	L'invite n'a pas été lancée en tant qu'administrateur. Relancez-la avec les droits d'administrateur.
« édition serveur » sous la version de Windows examinée	La collecte se fait, et chaque format que WAC ne décode pas sur cette version est déclaré (section 7.10) ; le cycle de test complet ne couvre pas ce serveur (il couvre les postes de XP SP1 à Windows 11, les serveurs 2003 et de 2008 R2 à 2025, et Server Core 2022).
Échec de « Checking the collection medium », répertoire non vide	Le dossier de sortie contient une collecte antérieure. Indiquez un dossier neuf avec `--output`.
Échec de « Checking the collection medium », place insuffisante	Le support est trop petit pour les options choisies (section 3.2). Utilisez un support plus grand ou retirez `--binary`.
Un fichier d'artefact porte `"CollectionStatus": "Not Collected"`	La source n'a pas pu être lue ; le motif est dans `Error`. Les autres artefacts ne sont pas affectés.
Des pièces en échec au manifeste	Fichiers cités absents du disque (logiciel désinstallé, profil de service sans ruche), volume non NTFS… Le motif est dans `Result`.
Le journal indique des binaires « hachés sans être copiés »	La place a manqué pendant le prélèvement : les empreintes sont complètes, les copies non.
Besoin de comprendre un échec de lecture	Relancez avec `--loglevel=2` (et, pour la lecture NTFS, `--debug`) et consultez `WAC.exe.log`.

ANNEXE A

Glossaire

Artefact
Trace laissée par l'activité du système ou d'un utilisateur, et exploitable en investigation.

Consigne
Répertoire des copies brutes, scellé, jamais réécrit.

Empreinte
Valeur calculée sur le contenu d'un fichier (MD5, SHA-1, SHA-256) : deux contenus différents ont des empreintes différentes.

Journal de transaction
Fichier `.LOG1`/`.LOG2` contenant les modifications récentes d'une ruche, pas encore écrites dans celle-ci.

Manifeste
Fichier qui identifie chaque pièce de la consigne.

`$MFT`
Table des fichiers d'un volume NTFS.

Ruche
Fichier constituant une partie de la base de registre (`SYSTEM`, `SOFTWARE`, `ntuser.dat`…).

Sceau
Empreinte du manifeste, conservée à part : elle révèle toute retouche du manifeste.

Travail
Répertoire des copies de travail, sur lesquelles portent l'analyse et les éventuelles modifications.

WOF / Compact OS
Mode de stockage compressé des fichiers système de Windows 10 et 11.